

FEMaster User Manual

Finite Elements in Mechanical Analysis and Simulation for Technical Engineering
Research

Version 2.6.13

10 August 2026

Contents

1	Overview	1
1.1	Model layers	1
1.2	Supported analysis types	1
1.3	Degree-of-freedom convention	2
1.4	Targets	2
2	Build and Command Line	3
2.1	Build	3
2.2	CMake presets	3
2.3	Running an input deck	4
2.4	DSL documentation mode	4
3	DSL Basics	6
3.1	Commands	6
3.2	Scopes	6
3.3	Aliases, defaults, and flags	6
3.4	Component order	6
3.5	Targets	7
3.6	Dependency-ordered parsing	7
3.7	Parser diagnostics	7
4	Command Reference	8
4.1	Geometry and topology	8
4.2	Properties	8
4.3	Features, fields, loads, and constraints	8
4.4	Loadcase controls	9
5	Nodes	10
5.1	Node IDs	10
5.2	Coordinate meaning	10
5.3	Active degrees of freedom	10
6	Elements	12

6.1	Registered element types	12
6.2	Solid element family	13
6.3	C3D4: 4-node tetrahedral solid	14
6.4	C3D5: 5-node pyramid input	15
6.5	C3D6: 6-node wedge solid	16
6.6	C3D8: 8-node hexahedral solid	17
6.7	C3D8R: reduced-integration 8-node hexahedral solid	18
6.8	C3D10: 10-node quadratic tetrahedral solid	20
6.9	C3D13: 13-node quadratic pyramid solid	21
6.10	C3D15: 15-node quadratic wedge solid	22
6.11	C3D20: 20-node quadratic hexahedral solid	23
6.12	C3D20R: 20-node reduced-integration hexahedral solid	24
6.13	Shell element family	25
6.14	S3: 3-node triangular shell	25
6.15	MITC3FRT: finite-rotation 3-node MITC shell	27
6.16	S4: 4-node quadrilateral shell	28
6.17	MITC4: quadrilateral shell with tied shear interpolation	29
6.18	MITC4FRT: finite-rotation 4-node MITC shell	30
6.19	S6: 6-node quadratic triangular shell	31
6.20	MITC6FRT: finite-rotation 6-node MITC6-b shell	32
6.21	S8: 8-node quadratic quadrilateral shell	33
6.22	MITC8: 8-node quadratic shell with tied shear interpolation	34
6.23	MITC8FRT: finite-rotation 8-node MITC shell	36
6.24	QSPT: quadrilateral shear-panel type	37
6.25	Beam and truss element family	38
6.26	B33: 3D beam element	38
6.27	T3: 2-node truss	39
6.28	Choosing an element	40
6.29	Shell directors and prescribed normals	41
7	Surfaces	42
7.1	Surface from one element side	42
7.2	Surface from an element set	42
7.3	Orientation and sign	42
8	Sets	44
8.1	*NSET: node sets	44
8.2	*ELSET: element sets	46
8.3	*SFSET: surface sets	47
8.4	Generated and explicit sets	48
8.5	Naming conventions	48

8.6	Checking set usage	48
9	Sections and Profiles	49
9.1	*SOLIDSECTION: solid sections	49
9.2	*SHELLSECTION: shell sections	51
9.3	*PROFILE: Beam profiles	53
9.4	*BEAMSECTION: beam sections	54
9.5	*TRUSSSECTION: truss sections	55
9.6	Choosing and checking sections	56
10	Features	57
10.1	Point Mass	57
11	Material Models	58
11.1	Material context: *MATERIAL	58
11.2	Density: *DENSITY	58
11.3	Thermal expansion: *THERMALEXPANSION	59
11.4	Isotropic elasticity	59
11.5	Generalised isotropic elasticity	60
11.6	Neo-Hooke hyperelasticity	60
11.7	Orthotropic elasticity	61
11.8	Prescribed ABD shell stiffness	61
12	Coordinate Systems and Orientations	63
12.1	Rectangular systems	63
12.2	Cylindrical systems	64
12.3	Where orientations are used	64
13	Fields	65
13.1	Supported domains	65
13.2	Typical references	66
13.3	Shell normal selection: *NORMAL	66
14	Loads, Supports, and Amplitudes	67
14.1	Support collectors	67
14.2	Concentrated loads	68
14.3	Surface traction and pressure	68
14.4	Volume loads	69
14.5	Thermal loads	69
14.6	Inertial loads	69
14.7	Amplitudes	70
15	Constraints	71

15.1 Rigid-body motion suppression: *RBM	71
15.2 Coupling: *COUPLING	72
15.2.1 Kinematic coupling equations	72
15.2.2 Structural coupling load distribution	73
15.3 Tie: *TIE	73
15.4 Connector: *CONNECTOR	73
15.5 Constraint summary	74
16 Analysis and Solver Controls	75
16.1 Activating support and load collectors	75
16.2 Solver	75
16.3 Constraint method	76
16.3.1 Nullspace method	76
16.3.2 Lagrange method	77
16.3.3 Elimination method	77
16.4 Inertia relief	77
16.5 Load rebalancing	78
16.6 Matrix and diagnostic requests	78
16.7 Eigenvalue parameters	79
16.8 Nonlinear static controls	79
16.8.1 Load control	80
16.8.2 Arc-length control	81
16.8.3 Adaptation and convergence	81
16.9 Transient-specific controls	82
16.10 Linear harmonic response controls	84
16.10.1 Frequency sweep	84
16.10.2 Rayleigh damping in harmonic analysis	84
17 Loadcases	85
17.1 General syntax	85
17.2 Common equation structure	85
17.3 Linear Static	85
17.3.1 Governing equation	85
17.3.2 Typical DSL	86
17.4 Linear Static Topology	86
17.4.1 Governing equation	86
17.4.2 Typical DSL	86
17.5 Nonlinear Static	87
17.5.1 Governing equation	87
17.5.2 Typical DSL	87
17.6 Eigenfrequency	88

17.6.1	Governing equation	88
17.6.2	Typical DSL	88
17.7	Linear Buckling	88
17.7.1	Governing equations	89
17.7.2	Sigma	89
17.7.3	Typical DSL	89
17.8	Linear Transient	89
17.8.1	Governing equation	90
17.8.2	Typical DSL	90
17.9	Typical command overview	90
17.10	Linear Harmonic	92
17.10.1	Governing equation	92
17.10.2	Typical DSL	92
17.10.3	Result fields	92
18	Contact	94
18.1	Contact definition	94
18.2	Master and slave surfaces	94
18.3	Gap and clearance	95
18.4	Dual-mortar discretization	95
18.5	Penalty and augmented-Lagrange update	95
18.6	Nonlinear solution behaviour	95
18.7	Current limitations	96
18.8	Minimal example	96
19	Result Format	97
19.1	Native .res files	97
19.1.1	Loadcase blocks	97
19.1.2	Dense fields	97
19.1.3	Indexed fields	98
19.1.4	Common field names	98
19.1.5	Linear harmonic result fields	98
19.2	FRD files	99
20	Complete Examples	101
20.1	Linear static truss	101
20.2	Topology-aware linear static solid	102
20.3	Shell with explicit ABD and transverse shear stiffness	103
20.4	Nonlinear load control	104
20.5	Snap-through with arc-length control	105
20.6	Eigenfrequency with a point mass	106

20.7 Linear buckling of a shallow truss	107
20.8 Linear transient ramp load	108
20.9 Linear harmonic response of a cantilever beam	110
20.10 Dual-mortar contact between two C3D8 blocks	110
A Mathematical Derivations	113
A.1 Element stiffness	113
A.2 Compliance	113
A.3 Density penalization	114
A.4 Orientation-dependent stiffness	114
A.5 Voigt transformation	114
A.6 Orientation sensitivity	115
A.7 Rotation derivatives	115

1 Overview

FEMaster is a finite-element solver for linear and nonlinear mechanical analyses. Models are described with an input-deck language in which commands start with an asterisk, command options are written as keywords, and numerical data follows on subsequent lines.

The manual is organized as a user reference. Command tables summarize the available DSL commands; the later chapters explain how model entities are connected, how collectors are activated in loadcases, and how analysis controls affect the solved equations.

1.1 Model layers

A model starts with nodes. Nodes define global coordinates and provide the reference points for all other model entities. Elements connect nodes into line, surface, or volume discretizations. Surfaces describe element sides used by pressure loads, distributed loads, ties, surface couplings, and surface-to-surface contact. Sets group nodes, elements, or surfaces so that loads, supports, sections, and constraints can refer to meaningful names instead of long ID lists.

Materials and sections give elements their physical meaning. An element by itself defines only topology and interpolation. A section assigns material, thickness, area, or profile data. Features such as point masses are additional model objects that contribute mass, inertia, or spring stiffness without being part of element connectivity. Contact pairs connect explicitly selected master and slave surfaces and contribute unilateral normal interaction during nonlinear static analysis.

Loads and supports are stored in named collectors. A loadcase decides which collectors are active. This allows one model to contain several analyses without repeating geometry, materials, sections, or load definitions.

1.2 Supported analysis types

Type	Purpose
LINEARSTATIC	Small-displacement static equilibrium with linear material behaviour.
LINEARSTATICTOPO	Static analysis with element-wise topology fields such as density, orientation, and penalization exponent.
NONLINEARSTATIC	Incremental static equilibrium with geometric and material nonlinearity, load control, arc-length path following, and non-linear surface contact.
EIGENFREQ	Natural-frequency analysis of the linear constrained system.
LINEARBUCKLING	Linear buckling analysis from a static preload and the resulting geometric stiffness.
LINEARTRANSIENT	Linear time integration with Newmark parameters, optional Rayleigh damping, and controlled result cadence.

Type	Purpose
LINEARHARMONIC	Direct steady-state harmonic response over a prescribed excitation-frequency sweep with optional Rayleigh damping.

1.3 Degree-of-freedom convention

Mechanical nodal vectors use the component order

Ux, Uy, Uz, Rx, Ry, Rz.

The first three components are translations and the last three are rotations. Not every element uses all six components. Solid elements usually contribute only translational displacement degrees of freedom. Beams, shells, constraints, connectors, and some features may introduce rotations.

1.4 Targets

Many commands accept a target that can be either a set name or a single numeric ID. For example, ***CLOAD** can target a node set or one node, ***DLOAD** can target a surface set or one surface, and ***VLOAD** can target an element set or one element. This makes small test decks compact while keeping large models readable through named sets.

2 Build and Command Line

FEMaster is built with CMake. A typical build produces an executable named **FEMaster** in the build output directory. The executable reads an input deck and writes result files using a common job base name. If no explicit output path is supplied, the job base is derived from the input file name. FEMaster then writes both the native text result file **.res** and the CalculiX/CGX result file **.frd**.

2.1 Build

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release
```

A release build is the normal choice for analysis work. A debug build is useful for small diagnostic models, assertions, and development checks, but it is usually too slow for larger finite-element models.

2.2 CMake presets

The repository provides CMake presets for the common supported toolchain combinations. Presets place their build directory under **build/<preset-name>** and set the normal release-oriented defaults: tests disabled, double precision enabled, Eigen compile-time reduction flags enabled, and OpenMP disabled unless the preset explicitly enables it.

```
cmake --preset linux
cmake --build --preset linux

cmake --preset linux-mkl-omp
cmake --build --preset linux-mkl-omp
```

The Linux presets cover CPU-only builds, OpenMP builds, oneMKL builds, CUDA builds, cuDSS builds, and combined CUDA/oneMKL configurations. Windows presets are split between MSVC and IntelLVM toolchains. macOS currently provides CPU presets. The exact preset list should be read from **CMakePresets.json** for the checked-out version.

CMake option	Meaning
FEMASTER_BUILD_TESTS	Build and register the GoogleTest executable.
FEMASTER_ENABLE_CUDA	Build CUDA solver backends.
FEMASTER_ENABLE_CUDSS	Use NVIDIA cuDSS for CUDA sparse direct solves. Requires CUDA.
FEMASTER_ENABLE_MKL	Use Intel oneMKL for Eigen operations and PARDISO.
FEMASTER_MKL_SEQUENTIAL	Use the sequential oneMKL threading layer.

CMake option	Meaning
FEMASTER_MKL_STATIC	Link oneMKL component libraries statically.
FEMASTER_ENABLE_OPENMP	Enable OpenMP for FEMaster CPU loops.
FEMASTER_DOUBLE_PRECISION	Use double precision arithmetic.
FEMASTER_FAST_EIGEN_COMPILE	Use Eigen compile-time reduction flags.
FEMASTER_STATIC_RUNTIME	Link compiler C/C++ runtimes statically where supported.
FEMASTER_FULLY_STATIC	Produce a fully static Linux executable. Incompatible with CUDA shared libraries.
FEMASTER_TIME_REPORT	Enable compiler timing diagnostics where supported.
FEMASTER_DEPS_INCLUDE_DIR	Include root containing header-only dependencies such as Eigen, Spectra, and argparse.
FEMASTER_CUDSS_ROOT	Root of an NVIDIA cuDSS installation. Defaults to the CUDSS_DIR environment variable.

2.3 Running an input deck

```
FEMaster model.inp
FEMaster model.inp --output model.res
FEMaster model.inp --ncpus 8
FEMaster model.inp --no-frd
FEMaster model.inp --no-res
```

Argument	Meaning
input_file	Path to the input deck. If no <code>.inp</code> suffix is supplied, it is added automatically.
-output FILE	Explicit output base or result file path. The suffix <code>.res</code> , <code>.frd</code> , or <code>.inp</code> is stripped before writers are opened, so <code>-output run.res</code> produces <code>run.res</code> and <code>run.frd</code> .
-ncpus N	Number of threads allowed for parallel assembly and compute sections.
-no-res	Disable native FEMaster <code>.res</code> result output. This is useful when only CalculiX/CGX <code>.frd</code> output is needed.
-no-frd	Disable CalculiX/CGX <code>.frd</code> result output. This is useful when only the native FEMaster <code>.res</code> output is needed.

By default both result writers are enabled. `-no-res` and `-no-frd` can be combined if a run should only parse, solve, and print console diagnostics without writing either result format.

At the end of a regular solver run, FEMaster prints a process timing summary in hours, minutes, seconds, milliseconds, and total milliseconds. This timing covers the full executable run, including parsing, solving, result writing, and shutdown.

2.4 DSL documentation mode

The executable can print DSL registration information for the installed version. This is useful when checking which commands, keywords, and variants a particular build accepts.

```

FEMaster --document --doc-list
FEMaster --document --doc-show CLOAD
FEMaster --document --doc-variants ELASTIC

```

Option	Meaning
-doc-list	Print the available DSL commands.
-doc-show CMD	Show details for one command.
-doc-tokens CMD	Show expected tokens for a command.
-doc-variants CMD	Show accepted data-line variants.
-doc-search TEXT	Search command names and descriptions.
-doc-where-token TOKEN	Find commands that use a token.
-doc-all	Print the full command overview.
-doc-format text md json	Select the documentation output format. Currently the implementation falls back to text for non-text formats.
-doc-verbosity index compact full	Select the detail level for command documentation.
-doc-width N	Set the wrapping width for documentation output. Use 0 for no wrapping.
-doc-no-wrap	Disable wrapping. Equivalent to -doc-width 0.
-doc-regex	Treat -doc-search as a regular expression.

3 DSL Basics

3.1 Commands

A FEMaster deck consists of commands and data lines. A command starts with an asterisk, followed by the command name and optional comma-separated keywords.

Syntax

```
*COMMAND, KEY=value, FLAG  
data, data, data
```

Command names and keywords are normalized by the parser. Case is not significant, and spaces in command names are removed before lookup. A consistent uppercase style is still recommended for readability.

3.2 Scopes

Some commands open an active context. ***MATERIAL** activates a material; subsequent material subcommands such as ***ELASTIC**, ***HYPERELASTIC**, ***DENSITY**, and ***THERMALEXPANSION** apply to it. ***LOADCASE** activates an analysis context; commands such as ***SUPPORTS**, ***LOADS**, ***SOLVER**, and ***CONSTRAINTMETHOD** configure that loadcase.

***MODEL**, **NAME=<name>** may be used as an explicit top-level model marker. ***END** is accepted inside a loadcase as an explicit closing marker. Neither marker is required: a context also ends when a following command no longer belongs to it or when the file ends.

3.3 Aliases, defaults, and flags

Many keywords have aliases. **NAME** and **NSET** can identify node sets; **MATERIAL** and **MAT** are equivalent in sections. Defaults reduce boilerplate: ***NODE** defaults to **NALL**, ***ELEMENT** defaults to **EALL**, and ***SOLVER** defaults to **DEVICE=CPU** and **METHOD=DIRECT**.

Flags are keywords without a required value. **GENERATE** is the most common example. Explicit false-like values such as **NO**, **OFF**, **FALSE**, or **0** are interpreted as inactive.

3.4 Component order

Mechanical nodal vectors use **Ux**, **Uy**, **Uz**, **Rx**, **Ry**, **Rz**. The first three values are translations. The last three values are rotations. Loads, supports, velocities, accelerations, displacements, and many internal reductions use this convention.

3.5 Targets

A target can often be a set name or a single numeric ID. If a matching set name exists, the set is used. Otherwise FEMaster attempts to interpret the token as one ID.

3.6 Dependency-ordered parsing

FEMaster processes an input deck in dependency-ordered stages. Topology is created before fields whose sizes depend on element-local enumeration. Enumeration-dependent fields such as `ELEMENT_NODAL`, `ELEMENT_IP`, and `ELEMENT_MP` are therefore available before later model construction uses them.

Commands that reference such fields should still appear after the corresponding `*FIELD` definition in the input deck. In particular, a shell-director field must be defined before `*NORMAL, FIELD=<name>` selects it. The repeated parser passes are an implementation detail; users should write decks in logical dependency order rather than relying on a later command to define data referenced earlier in the file.

3.7 Parser diagnostics

Input errors report the source path and one-based line number of the command or data record that caused the failure. This applies to unknown commands, invalid keywords, commands used in the wrong scope, malformed data, incomplete multiline records, and errors raised while binding parsed values to the model. If the failing line came from an included file, the diagnostic names that included file rather than only the main deck. Unknown commands are rejected instead of being treated as implicit structural scopes.

4 Command Reference

This chapter gives a compact overview of the DSL commands. The following chapters describe the commands in more detail.

4.1 Geometry and topology

Command	Purpose
*NODE	Define nodes and coordinates.
*ELEMENT	Define elements by type and connectivity.
*SURFACE	Define element sides or element-set sides as surfaces.
*NSET	Define node sets.
*ELSET	Define element sets.
*SFSET	Define surface sets.

4.2 Properties

Command	Purpose
*MATERIAL	Create or activate a material.
*ELASTIC	Assign elastic material behaviour.
*HYPERELASTIC	Assign Abaqus-style Neo-Hooke hyperelastic behaviour.
*DENSITY	Assign mass density.
*THERMALEXPANSION	Assign thermal expansion coefficient.
*SOLIDSECTION	Assign material and optional orientation to solid elements.
*SHELLSECTION	Assign integrated or prescribed ABD section behaviour to shell elements.
*BEAMSECTION	Assign material, profile, and local axis data to beam elements.
*TRUSSSECTION	Assign material and area to truss elements.
*PROFILE	Define beam profile constants.

4.3 Features, fields, loads, and constraints

Command	Purpose
*POINTMASS	Add concentrated mass, rotational inertia, and optional spring stiffness to node sets.
*FIELD	Define node, element, element-nodal, integration-point, or material-point data fields.

Command	Purpose
*NORMAL	Select a three-component element-nodal field for shell directors.
*ORIENTATION	Define a rectangular or cylindrical local coordinate system.
*SUPPORT	Add support entries to a support collector.
*CLOAD	Add concentrated nodal loads to a load collector.
*DLOAD	Add vector surface tractions to a load collector.
*PLOAD	Add pressure loads to a load collector.
*VLOAD	Add volume loads to a load collector.
*TLOAD	Add thermal loads from a temperature field.
*INERTIALOAD	Add rigid-body inertial loads.
*AMPLITUDE	Define a time function for transient load scaling.
*COUPLING	Define kinematic or structural couplings.
*TIE	Define tie constraints between slave nodes and master geometry.
*CONNECTOR	Define connector constraints between two node sets.
*RBM	Generate rigid-body-motion suppression equations.
*CONTACT	Define frictionless dual-mortar surface-to-surface contact for non-linear static analysis.

4.4 Loadcase controls

Command	Purpose
*LOADCASE	Start an analysis block.
*MODEL	Optionally start an explicit top-level model block.
*END	Optionally mark the end of a loadcase.
*SUPPORTS	Activate support collectors.
*LOADS	Activate load collectors.
*SOLVER	Select device and linear solution method.
*CONSTRAINTMETHOD	Select Nullspace, Lagrange, or elimination constraint handling.
*NONLINEAR	Configure path control, increment adaptation, Newton convergence, and tangent regularization for nonlinear static analysis.
*INERTIARELIEF	Enable inertia relief for free static models.
*REBALANCELOADS	Balance residual resultant force and moment.
*REQUESTSTIFFNESS	Request stiffness matrix output.
*REQUESTSTGEOM	Request geometric stiffness matrix output.
*NUMEIGENVALUES	Set the number of eigenpairs.
*SIGMA	Set the buckling eigenvalue shift.
*TOPODENSITY	Select the topology density field.
*TOPOORIENT	Select the topology orientation field.
*TOPOEXPONENT	Set the topology penalization exponent.
*TIME	Set transient time range and step size.
*NEWMARK	Set Newmark integration parameters.
*DAMPING	Set Rayleigh damping for transient or harmonic analysis.
*FREQUENCIES	Define the excitation-frequency sweep for linear harmonic analysis.
*WRITEEVERY	Set transient output cadence.
*INITIALVELOCITY	Set transient initial velocity from a field.
*CONSTRAINTSUMMARY	Request generated-constraint diagnostics.
*OVERVIEW	Print a model overview.

5 Nodes

Nodes are the geometric reference points of a FEMaster model. Each node has an ID and three global coordinates. Elements, loads, supports, constraints, and features refer to nodes either directly or through node sets.

Syntax

```
*NODE, NSET=<optional-set>  
<id>, <x>, <y>, <z>
```

NSET is optional. If omitted, nodes are added to NALL. Missing coordinate values are interpreted as zero, but production decks should normally write all three coordinates explicitly.

A node may have up to six mechanical degrees of freedom, U_x , U_y , U_z , R_x , R_y , R_z . Which of these are active depends on the connected elements and model objects. Solid elements usually need only translations. Shells, beams, point-mass springs, connectors, and constraints may require rotations.

5.1 Node IDs

Node IDs must be non-negative integers. The ID 0 is valid and can be used like any other node ID. Negative node IDs are not valid user-facing node identifiers in input decks because node IDs are used directly as row indices in nodal fields, connectivity, loads, supports, and result output.

FEMaster does not require a particular numbering strategy. Clear numbering is still useful for debugging, set construction, and result inspection. IDs should be unique and stable. If the same node ID is defined multiple times, the deck no longer gives a clear single coordinate for that node.

5.2 Coordinate meaning

Node coordinates are always global coordinates. Local coordinate systems do not change node positions; they only change how vector components are interpreted by commands such as *SUPPORT, *CLOAD, *DLOAD, *VLOAD, *SOLIDSECTION, *SHELLSECTION, and *CONNECTOR. This distinction matters: geometry is global, while loads, supports, material axes, and connector directions may be local.

5.3 Active degrees of freedom

FEMaster builds the active degree-of-freedom set from the model objects that reference each node. A node connected only to solid elements normally contributes three translational degrees of freedom. A shell or beam node contributes translations and rotations. A connector, coupling, support, or point-mass spring can also make rotational degrees of freedom relevant.

The presence of a node alone does not imply six active unknowns. The active set is determined when

the model is assembled. This keeps purely solid models smaller while still allowing mixed solid-shell-beam models to share the same nodal component convention.

6 Elements

Elements connect nodes and define the interpolation used over a line, surface, or volume region. They are the fundamental discretization objects of the finite-element model: once a mesh has been read, the selected element types determine how nodal degrees of freedom are converted into strains, curvatures, stresses, internal forces, mass contributions, geometric stiffness terms, and recoverable result fields.

An element alone does not define a complete physical model. A usable model also needs sections, materials, coordinate systems, loads, supports, and analysis controls. The element type defines the kinematic assumption and the numerical integration domain; the section and material then define how that kinematic state becomes stiffness, mass, stress, or section force.

Syntax

```
*ELEMENT, TYPE=<type>, ELSET=<optional-set>
<element-id>, <node-1>, <node-2>, ...
```

TYPE is required. ELSET is optional and defaults to EALL. Element IDs should be non-negative and unique because they are referenced by element sets, sections, loads, result fields, diagnostics, and postprocessing operations. Connectivity order matters: it defines the local reference orientation, face numbering, integration point layout, and, for structural elements, the local axes used for section behaviour.

Element quality has a direct influence on solution quality. Poorly shaped elements, excessive distortion, highly warped shell surfaces, inverted Jacobians, and inappropriate element choices can significantly reduce accuracy even if the solver converges successfully. A coarse model made from a suitable element family is often more useful than a fine model made from an element type that does not match the physical idealization.

6.1 Registered element types

Type	Nodes	Family
C3D4	4	Linear tetrahedral solid.
C3D5	5	Pyramid input mapped to a degenerated hexahedral solid.
C3D6	6	Linear wedge solid.
C3D8	8	Linear hexahedral solid.
C3D8R	8	Reduced-integration linear hexahedral solid with hourglass stabilization.
C3D10	10	Quadratic tetrahedral solid.
C3D13	13	Quadratic pyramid solid.
C3D15	15	Quadratic wedge solid.
C3D20	20	Quadratic hexahedral solid with full higher-order integration.
C3D20R	20	Quadratic hexahedral solid with reduced stiffness integration and higher-order mass integration.
S3	3	Three-node triangular shell.

Type	Nodes	Family
MITC3FRT	3	Three-node finite-rotation MITC shell for geometrically nonlinear analysis.
S4	4	Four-node quadrilateral shell.
MITC4	4	Four-node quadrilateral shell with MITC-style tied shear interpolation.
MITC4FRT	4	Four-node finite-rotation MITC shell for geometrically nonlinear analysis.
S6	6	Six-node quadratic triangular shell.
MITC6FRT	6	Six-node finite-rotation MITC6-b shell for geometrically nonlinear analysis.
S8	8	Eight-node quadratic quadrilateral shell.
MITC8	8	Eight-node quadratic quadrilateral shell with MITC-style tied shear interpolation.
MITC8FRT	8	Eight-node finite-rotation MITC8 shell for geometrically nonlinear analysis.
QSPT	4	Specialized four-node quadrilateral shear-panel type.
B33	2 or 3	Two-node 3D beam element with optional orientation node.
T3	2	Two-node truss element.

6.2 Solid element family

Solid elements describe three-dimensional continua. They use translational nodal degrees of freedom and require ***SOLIDSECTION**; the section assigns material data and, where needed, a material orientation. Solid elements are appropriate when the stress state is genuinely three-dimensional, when through-thickness stresses matter, or when local geometry, joints, load introduction, contact, or nonlinear local deformation cannot be represented by a shell or line idealization.

The main cost of solid modelling is the number of degrees of freedom required to resolve a body. A solid plate, for example, usually needs several elements through thickness before bending and transverse stress fields become credible. Low-order solids are inexpensive and robust, but they may become overly stiff in bending-dominated states. Higher-order solids improve stress gradients and curved geometry representation, but they are more sensitive to distortion and use larger element matrices.

For most engineering models, hexahedral elements are preferable when a high-quality structured or swept mesh is available. Tetrahedral elements are valuable for complex geometry and automated meshing, especially in their quadratic form. Wedges and pyramids are often transition elements: they are useful when mesh topology changes, but they should not be used casually in regions where they dominate the stiffness or stress result.

6.3 C3D4: 4-node tetrahedral solid

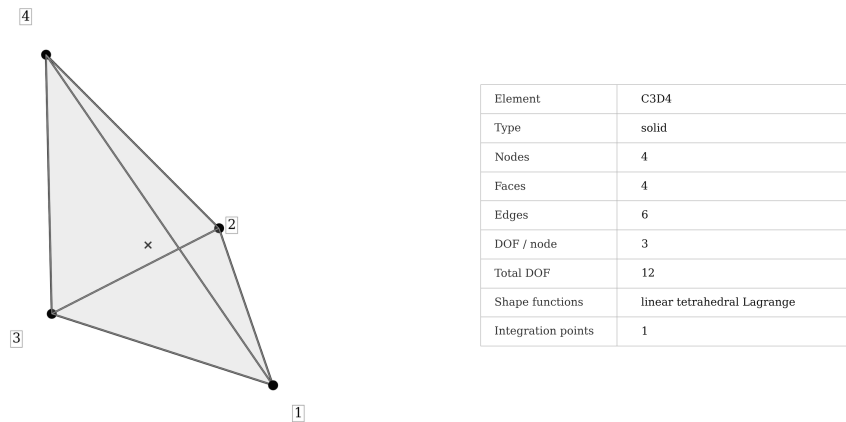


Figure 6.1: C3D4 element topology and integration scheme.

Syntax

```
*ELEMENT, TYPE=C3D4, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>
```

C3D4 is the simplest three-dimensional continuum element in FEMaster. It uses four corner nodes, a tetrahedral reference domain, and linear displacement interpolation. Because the interpolation is linear, the strain field inside a single element is constant. The element is therefore easy to integrate and very robust from a meshing perspective, but it has limited ability to represent bending, curvature, or stress gradients without substantial mesh refinement.

The strongest reason to use C3D4 is geometric robustness. Tetrahedra can fill complicated CAD volumes, imported parts, transition regions, and local details with relatively little manual meshing effort. For early model checks, contact-region filler meshes, stiffness estimates, or coarse load-path studies, this can be more important than high stress accuracy. A model built from C3D4 elements is often a useful first step because it can reveal gross constraint errors, load direction mistakes, disconnected parts, and order-of-magnitude stiffness behaviour.

The weakness of C3D4 is accuracy per degree of freedom. In bending-dominated problems, linear tetrahedra tend to be too stiff unless the mesh is very fine. Since each element carries only a constant strain state, stress contours can look patchy and stress gradients are represented mostly by element-to-element jumps. This is especially visible near holes, notches, fillets, and load introduction regions. If the goal is engineering stress recovery rather than only global stiffness, C3D10 is usually a better tetrahedral choice.

When using C3D4, keep the mesh quality conservative. Avoid sliver elements, inverted tetrahedra, and regions where many badly shaped elements align in the same stress path. Refinement should be guided by displacement convergence and by the stability of integrated quantities such as reactions and strain energy, not only by local peak stress values. The element is robust, but it should not be mistaken for a high-accuracy stress element.

6.4 C3D5: 5-node pyramid input

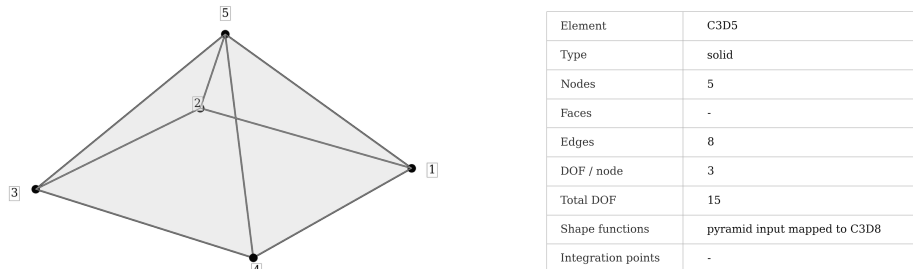


Figure 6.2: C3D5 pyramid input topology.

Syntax

```
*ELEMENT, TYPE=C3D5, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <apex>
```

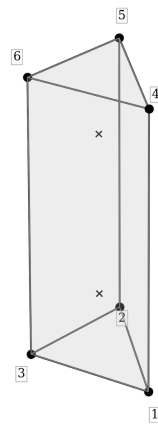
C3D5 accepts a five-node pyramid-style connectivity. In FEMaster this topology is treated primarily as an input convenience: the pyramid is mapped internally to a degenerated hexahedral representation by repeating the apex node in the upper hexahedral positions. This allows mixed meshes containing pyramid transition cells to be imported without requiring the user to remesh every transition region by hand.

The element is useful when a mesh transitions between a quadrilateral or hexahedral region and a tetrahedral region. Such transitions occur frequently in automatic mesh generation, especially around complex local features attached to otherwise block-like geometry. In that role, C3D5 is a compatibility element. It helps preserve mesh topology and lets the surrounding solid elements carry the main structural response.

It should not be interpreted as a dedicated high-quality pyramid formulation. Degenerated mappings can produce less predictable numerical behaviour than regular tetrahedra, wedges, or hexahedra. The quality of the base quadrilateral, the height of the pyramid, and the position of the apex strongly influence the Jacobian and therefore the stiffness contribution. Flat, skewed, or highly stretched pyramids can become poor transition cells even if the surrounding mesh looks acceptable.

Use C3D5 deliberately and preferably in small numbers. It is reasonable in local transition zones where it improves mesh compatibility, but it should not dominate a stress-critical region. If many pyramid elements appear in the primary load path, inspect the mesh and compare against a refined or alternative discretization. For stress-sensitive work, a clean tetrahedral or hexahedral mesh is normally more predictable than a large population of degenerated pyramid cells.

6.5 C3D6: 6-node wedge solid



Element	C3D6
Type	solid
Nodes	6
Faces	5
Edges	9
DOF / node	3
Total DOF	18
Shape functions	linear wedge / prism
Integration points	2

Figure 6.3: C3D6 wedge element.

Syntax

```
*ELEMENT, TYPE=C3D6, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>
```

C3D6 is a linear wedge, or prism, solid. It combines triangular topology in one parametric direction with quadrilateral side faces in the other directions. The element is useful for swept meshes, extruded triangular meshes, layered solids, and transition regions where a purely hexahedral topology is not available but a tetrahedral mesh would not align well with the geometry.

The usual strength of a wedge is directional structure. If a part has a natural extrusion direction, such as a layer stack, a swept rib, a boundary-layer-like region, or a prismatic transition, wedge elements can align the mesh with that direction more cleanly than tetrahedra. This can improve through-thickness consistency and make section-like stress interpretation easier. Wedges can also reduce the number of elements needed in regions where triangular surface meshing is unavoidable but the volume is still essentially swept.

As a low-order element, C3D6 still has limited bending performance. If only one or two linear wedge elements are used through thickness, bending-dominated states can be too stiff and stress recovery can be coarse. A wedge mesh should therefore be refined in the direction where gradients are expected, especially through thickness and near load introduction. The element is not a substitute for a higher-order formulation when curvature or stress gradients are central to the analysis.

Mesh quality is particularly important for wedges because their triangular and quadrilateral faces can distort in different ways. Collapsed triangular faces, strongly skewed side faces, and elements with a poor height-to-base relationship can produce unreliable Jacobians. A good wedge mesh should have consistent layer direction, limited twisting between the two triangular faces, and smooth size transitions. When used in that role, C3D6 is a practical compromise between fully structured hexahedral meshes and fully unstructured tetrahedral meshes.

6.6 C3D8: 8-node hexahedral solid

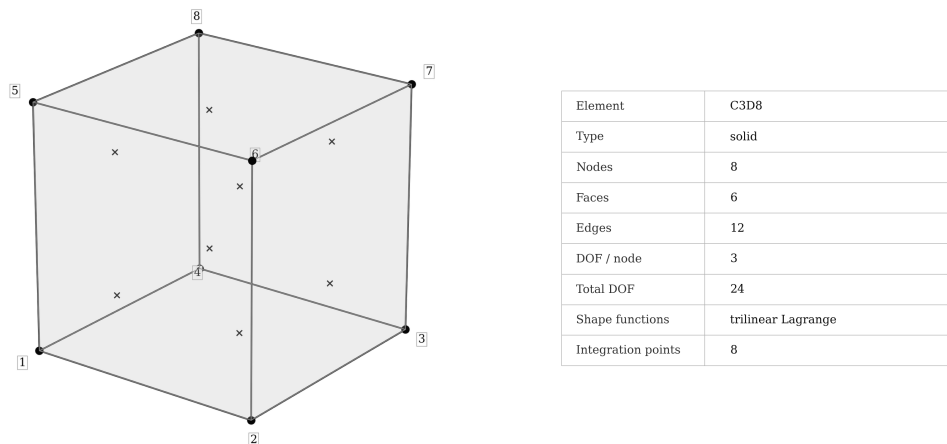


Figure 6.4: C3D8 hexahedral solid element.

Syntax

```
*ELEMENT, TYPE=C3D8, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>, <n7>, <n8>
```

C3D8 is the standard low-order hexahedral solid element. It uses eight corner nodes and a hexahedral reference domain. In a well-shaped mesh, the element is often more efficient than a comparable low-order tetrahedral discretization because it aligns naturally with block-like geometry, swept volumes, layered structures, and orthotropic material directions.

The element is a good default for regular solid meshes. It performs well when the mesh follows the structural load path and when element directions are meaningful: for example in plates modelled with solids, rectangular blocks, extruded parts, bonded layers, spars, ribs, and regions where material orientation should follow a local coordinate system. Hexahedral elements also make it easier to control the number of elements through thickness, which is important for bending and local stress gradients.

The main limitation is still the linear displacement interpolation. A coarse C3D8 mesh can be too stiff in bending, and one element through thickness is rarely enough for credible stress results in a bending-dominated solid model. Curved boundaries are represented only by straight element edges, so geometric approximation error can also become significant unless the mesh is refined or a higher-order hexahedral element is used.

Element distortion matters. Warped faces, skewed edges, inverted Jacobians, and extreme aspect ratios reduce the reliability of stiffness and stress recovery. A distorted hexahedron can be worse than a cleaner tetrahedral mesh. Use C3D8 when the mesh can remain structured and reasonably regular; for complex geometry where a high-quality hex mesh is not practical, a quadratic tetrahedral mesh may produce more predictable stress behaviour.

6.7 C3D8R: reduced-integration 8-node hexahedral solid

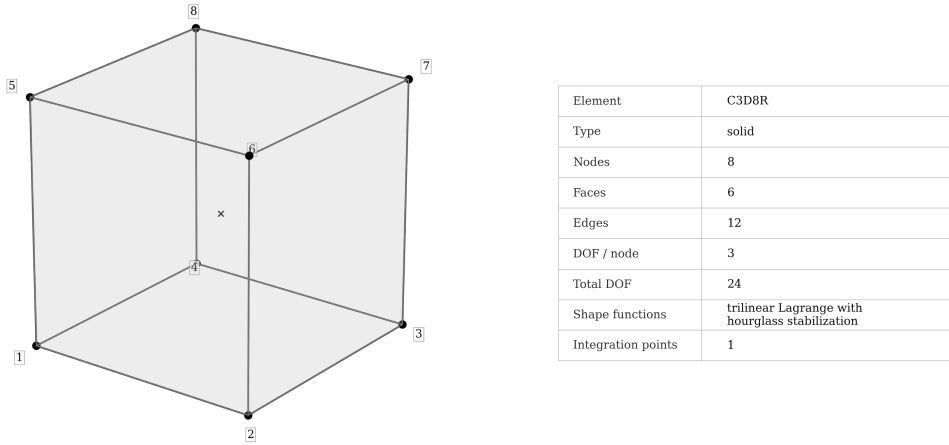


Figure 6.5: C3D8R reduced-integration hexahedral solid element.

Syntax

```
*ELEMENT, TYPE=C3D8R, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>, <n7>, <n8>
```

C3D8R uses the same eight-node hexahedral topology, connectivity, faces, sections, loads, and material models as C3D8, but evaluates its continuum contribution at the element centre with weight 8. Reduced integration lowers the element cost and avoids some of the excessive bending stiffness of a fully integrated low-order hexahedron. The one-point continuum stiffness alone has four scalar zero-energy hourglass modes in each translational direction, so FEMaster adds reference-configuration Flanagan-Belytschko hourglass stabilization.

The primitive scalar hourglass modes are

$$\gamma_1 = s t, \quad \gamma_2 = t r, \quad \gamma_3 = r s, \quad \gamma_4 = r s t,$$

evaluated at the eight local hexahedral nodes. FEMaster projects these modes against the affine displacement field with volume-averaged reference gradients from a full $2 \times 2 \times 2$ geometry integration:

$$G = (I - \bar{D} X^T) \gamma.$$

The stabilization scale is

$$k_{\text{hg}} = \alpha G_{\text{eff}} V_0 \sum_a \|\bar{\nabla} N_a\|^2, \quad \alpha = 3.0, \quad G_{\text{eff}} = \frac{C_{44} + C_{55} + C_{66}}{3},$$

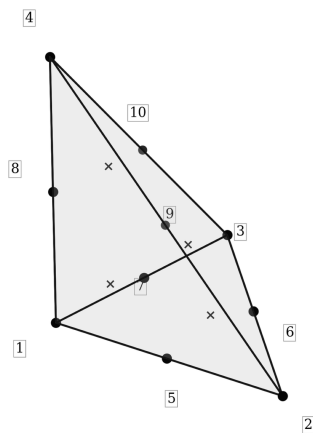
where C is the material tangent evaluated in the undeformed reference state and V_0 is the reference volume. The resulting tangent and internal force use the consistent pair

$$K_{\text{hg}} = k_{\text{hg}} (G G^T \otimes I_3), \quad f_{\text{hg}} = K_{\text{hg}} u_e.$$

The tangent is added to the inherited one-point C3D8 continuum tangent and symmetrized. The nonlinear internal force is assembled from the one-point Total-Lagrange continuum residual plus the constant reference hourglass force. The central integration-point stress and strain are repeated to all eight nodes for nodal result output.

The stabilization is fixed in the reference configuration. It is symmetric and Newton-consistent, but it is not fully objective under very large rigid-body rotations. Highly distorted elements can also make a fixed hourglass coefficient inappropriate, so deformation patterns and hourglass-sensitive bending modes should be checked by mesh refinement or comparison with **C3D8**. The current element also uses its one-point rule for mass integration; for eigenfrequency or transient work, prefer **C3D8** unless that reduced mass representation has been assessed deliberately.

6.8 C3D10: 10-node quadratic tetrahedral solid



Element	C3D10
Type	solid
Nodes	10
Faces	4
Edges	6
DOF / node	3
Total DOF	30
Shape functions	quadratic tetrahedral Lagrange
Integration points	4

Figure 6.6: C3D10 quadratic tetrahedral element.

Syntax

```
*ELEMENT, TYPE=C3D10, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>, <n7>, <n8>, <n9>, <n10>
```

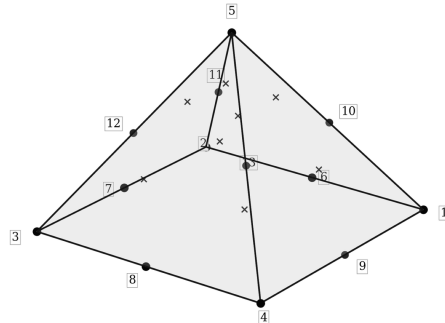
C3D10 is the quadratic tetrahedral solid element. It adds midside nodes to the four-node tetrahedral topology and uses quadratic interpolation over the tetrahedral reference domain. Compared with C3D4, it can represent curved displacement fields, bending deformation, and stress gradients much more effectively.

This element is usually the preferred tetrahedral choice for engineering stress analysis. It keeps the meshing flexibility of tetrahedra while avoiding many of the stiffness and stress-recovery limitations of linear tetrahedra. For complex castings, imported CAD geometry, organic shapes, local fillets, holes, and automatically generated meshes, C3D10 often gives a better balance between meshing robustness and numerical quality than a very fine C3D4 mesh.

The midside nodes are important. If they lie on curved geometric edges and surfaces, the element can represent curved boundaries more accurately than a linear tetrahedron. If they are badly placed, however, the mapping can become distorted. A quadratic tetrahedron with poor midside-node placement can have a poor Jacobian even when the corner-node tetrahedron looks acceptable. Mesh generation and geometry projection therefore matter more than they do for C3D4.

The cost per element is higher, but fewer elements are often required for a comparable displacement or stress result. In stress-critical regions, convergence should still be checked by refinement. Quadratic interpolation improves the element, but it does not remove the need for reasonable aspect ratios, clean transition sizes, and careful interpretation of peak stresses near singularities, sharp corners, and idealized point loads.

6.9 C3D13: 13-node quadratic pyramid solid



Element	C3D13
Type	solid
Nodes	13
Faces	-
Edges	8
DOF / node	3
Total DOF	39
Shape functions	quadratic pyramid
Integration points	8

Figure 6.7: C3D13 quadratic pyramid element.

Syntax

```
*ELEMENT, TYPE=C3D13, ELSET=<set>
<id>, <n1>, ..., <n13>
```

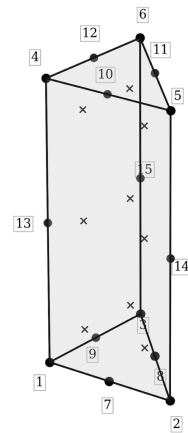
C3D13 is a higher-order pyramid transition element. It is intended for mixed meshes where a hexahedral or quadrilateral topology must connect to a tetrahedral topology. The additional midside nodes give the element more flexibility than a linear pyramid-style input and allow it to participate in higher-order solid meshes.

Pyramid elements are topologically useful but numerically delicate. Their reference-to-physical mapping is more complicated than the mapping of a tetrahedron, wedge, or hexahedron. The element has a quadrilateral base and triangular side faces, which makes it attractive as a transition cell, but also means that element quality depends on both the base shape and the apex position. Poor pyramid geometry can create strong Jacobian variation and unreliable stress recovery.

Use C3D13 as a transition element rather than as the primary building block of a model. It is reasonable around mesh interfaces where a structured hexahedral region joins an automatically meshed tetrahedral region. It is less attractive in large homogeneous blocks, in bending-critical regions, or in zones where the pyramid elements would carry a significant part of the stiffness response.

When a model contains C3D13 elements, inspect where they are located. If they appear only in local transition bands, they are usually acceptable. If they form long chains through the load path or cluster near stress-critical features, compare the result against a cleaner mesh. The element exists to preserve compatibility in mixed meshes, not to replace well-shaped hexahedral or tetrahedral discretizations.

6.10 C3D15: 15-node quadratic wedge solid



Element	C3D15
Type	solid
Nodes	15
Faces	5
Edges	9
DOF / node	3
Total DOF	45
Shape functions	quadratic wedge / prism
Integration points	9

Figure 6.8: C3D15 quadratic wedge element.

Syntax

```
*ELEMENT, TYPE=C3D15, ELSET=<set>
<id>, <n1>, ..., <n15>
```

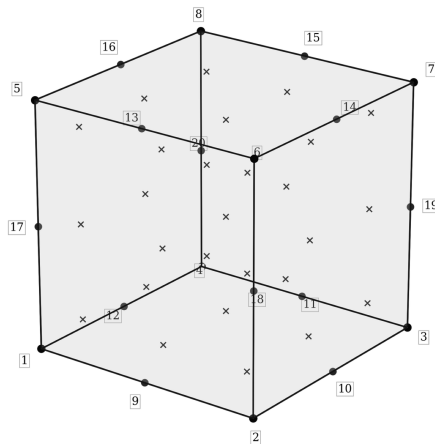
C3D15 is the quadratic counterpart to C3D6. It uses a wedge topology with midside nodes and higher-order interpolation. The element is useful when the geometry has a swept or layered character but the response requires smoother displacement and stress fields than a linear wedge can provide.

In practice, C3D15 is attractive for extruded triangular meshes, boundary-layer-type regions, curved swept solids, and transition zones where wedge topology is natural. It can represent bending and through-thickness gradients better than C3D6, and it can fit curved edges more accurately when midside nodes are projected correctly to the geometry.

The element is still sensitive to mesh quality. A wedge can distort through triangular face shape, through side-face warping, or through inconsistent movement of midside nodes. Because the interpolation is higher order, a bad geometric mapping can have a stronger effect than it would in a linear wedge. The two triangular faces should remain consistently oriented, and the side faces should not twist excessively.

Use C3D15 when wedge topology is needed and the result quality justifies the additional cost. It is a good candidate for refined swept regions, but not a cure for poor mesh topology. If the geometry can be meshed cleanly with quadratic hexahedra, C3D20 may be more efficient for smooth fields. If the geometry is highly unstructured, C3D10 may be easier to generate robustly.

6.11 C3D20: 20-node quadratic hexahedral solid



Element	C3D20
Type	solid
Nodes	20
Faces	6
Edges	12
DOF / node	3
Total DOF	60
Shape functions	quadratic serendipity hexahedral
Integration points	27

Figure 6.9: C3D20 quadratic hexahedral solid element.

Syntax

```
*ELEMENT, TYPE=C3D20, ELSET=<set>
<id>, <n1>, ..., <n20>
```

C3D20 is the full higher-order hexahedral solid element. It adds midside nodes to the standard hexahedral topology and uses quadratic interpolation with higher-order integration. In a high-quality structured mesh, this is one of the most accurate continuum element choices available in FEMaster.

The element is well suited to smooth stress fields, bending-dominated solids, curved geometry, and problems where stress gradients matter. Compared with C3D8, it can represent curvature and deformation modes with fewer elements, especially when the physical solution is smooth and the mesh follows the geometry. It is a strong choice for block-like parts, swept volumes, thick plates, and local refinement regions where a structured quadratic mesh can be maintained.

The extra nodes and integration points increase cost. Each element contributes larger matrices and requires more memory and assembly work than a linear hexahedron. This cost is often worthwhile when it reduces the number of elements required for convergence, but it can be wasteful in regions where the solution is nearly uniform or where the mesh is too distorted to benefit from higher-order interpolation.

Mesh quality is the deciding factor. A well-shaped C3D20 mesh can produce excellent results, but a distorted higher-order hexahedron can produce misleading stresses. Midside nodes should follow intended straight or curved edges consistently, and the element should not be warped into shapes that create poor Jacobians. Use this element when the geometry and meshing process can support it; otherwise, a cleaner lower-order or tetrahedral discretization may be more reliable.

6.12 C3D20R: 20-node reduced-integration hexahedral solid

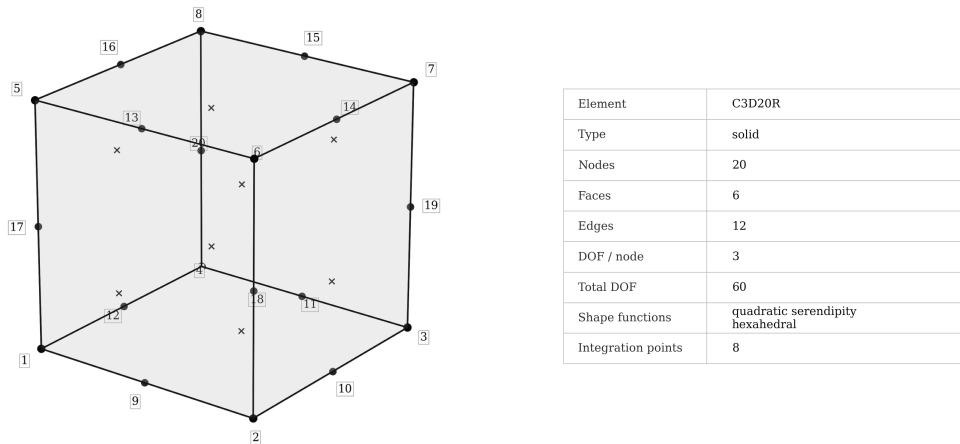


Figure 6.10: C3D20R reduced-integration quadratic hexahedral element.

Syntax

```
*ELEMENT, TYPE=C3D20R, ELSET=<set>
<id>, <n1>, ..., <n20>
```

C3D20R uses the same 20-node quadratic hexahedral topology as C3D20, but applies reduced stiffness integration. Reduced integration lowers the cost of stiffness assembly and can improve behaviour in situations where a fully integrated element is prone to excessive stiffness, such as bending-dominated or locking-sensitive states.

This element is useful when a high-quality quadratic hexahedral mesh is available and the model benefits from reduced integration. It can provide good bending behaviour with less assembly cost than the fully integrated element. The mass and stiffness integration choices should be understood as part of the element formulation rather than as a simple performance switch: they change the numerical character of the element.

The main risk is the appearance of nonphysical deformation patterns. Reduced integration can introduce zero-energy or hourglass-like modes if the formulation, constraints, or mesh quality do not control them sufficiently. Such modes may not be obvious from solver convergence alone. They often show up as unexpected displacement shapes, unrealistic local distortion, poor reaction balance, or stress fields that do not match the expected load path.

Use C3D20R with engineering checks. Inspect deformed shapes, reaction forces, strain energy distribution, and stress smoothness, especially for coarse or distorted meshes. If the model is sensitive, compare against C3D20 or a refined mesh. Reduced integration is powerful when used deliberately, but it should not be selected only because it is cheaper.

6.13 Shell element family

Shell elements describe thin-walled structures using translational and rotational degrees of freedom. They require `*SHELLSECTION`, which supplies material, thickness, optional orientation, transverse shear stiffness, and offsets. A shell model replaces a thin three-dimensional body with a two-dimensional mid-surface plus section behaviour. This makes shells much more efficient than solid meshes for plates, skins, panels, webs, pressure-vessel walls, and thin-walled mechanical components.

The shell idealization is appropriate when the thickness is small compared with the in-plane dimensions and when through-thickness stress details are not the main modelling target. Shells recover membrane behaviour, bending curvature, transverse shear behaviour, and section resultants. They are not intended to resolve local three-dimensional stress concentrations through thickness unless the model is enriched with local solid detail.

Shell quality depends on surface mesh quality, normal consistency, aspect ratio, warping, and thickness assumptions. A shell element can be geometrically thin but numerically poor if it is badly warped or if the local normals flip across the mesh. For thin bending-dominated structures, the choice between triangular, quadrilateral, low-order, and higher-order shells can matter as much as mesh density.

Shell element recommendation

The legacy shell elements `S3`, `S4`, `MITC4`, `S6`, `S8`, and `MITC8` should not be used for production calculations. They remain documented to describe accepted input types and for comparison with older example decks, but new shell analyses should use the finite-rotation shell family: `MITC3FRT`, `MITC4FRT`, `MITC6FRT`, or `MITC8FRT`.

6.14 S3: 3-node triangular shell

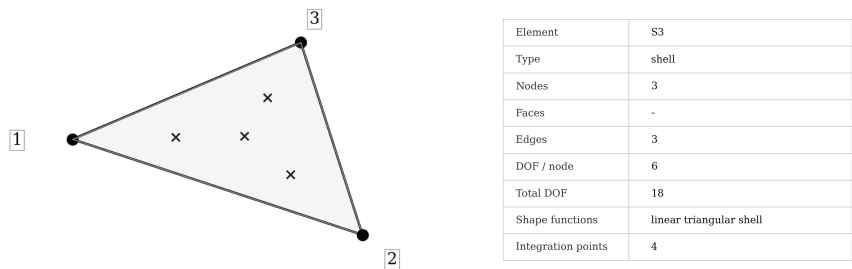


Figure 6.11: S3 triangular shell element.

Syntax

```
*ELEMENT, TYPE=S3, ELSET=<set>  
<id>, <n1>, <n2>, <n3>
```

`S3` is a three-node triangular shell element. It is geometrically flexible and can discretize complex surfaces, imported shell meshes, transition regions, and local details where quadrilateral meshing is difficult. Because

any polygonal surface can be decomposed into triangles, **S3** is often the easiest shell element to generate automatically.

Do not use **S3** for actual analysis work. It is retained as a legacy element and as a reference topology. Use **MITC3FRT** instead when a three-node triangular shell is required.

The legacy element should be viewed only as a compatibility path, not as a recommended bending element. Triangular shells can be stiffer than good quadrilateral shells in bending-dominated situations, and they may require substantial refinement to produce smooth curvature and stress-resultant fields. This is especially true on broad regular panels where a structured quadrilateral mesh could align with the primary bending and membrane directions.

S3 may still appear in legacy decks, imported shell meshes, or topology-conversion tests. In new analysis decks, use the corresponding **FRT** element and refine the mesh enough that local element orientation does not dominate the result.

Prefer quadrilateral **FRT** shell elements in smooth, regular regions when possible. Use triangular **FRT** shells only where triangular topology solves a real meshing problem. A mixed shell mesh can be reasonable if triangles are kept in transition zones and quadrilaterals carry the main panel behaviour.

6.15 MITC3FRT: finite-rotation 3-node MITC shell

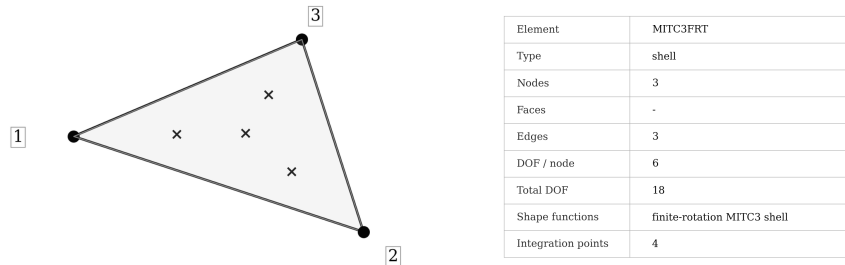


Figure 6.12: MITC3FRT triangular finite-rotation shell element.

Syntax

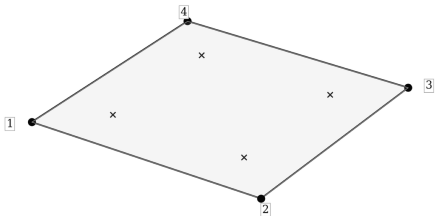
```
*ELEMENT, TYPE=MITC3FRT, ELSET=<set>
<id>, <n1>, <n2>, <n3>
```

MITC3FRT uses the same three-node triangular topology as **S3**, but follows the finite-rotation shell kernel used by the nonlinear MITC shell family. Its nodal degrees of freedom are translations and rotations, and its midsurface is integrated with cubic triangular quadrature. The element applies an assumed covariant transverse-shear field from three edge-midpoint tying positions.

Use this element when triangular shell topology is needed in a geometrically nonlinear model with large rotations. It is best suited to transition regions, irregular surface patches, and meshes where a triangular finite-rotation element is required for connectivity. It should not be selected as a high-accuracy bending element when a clean quadrilateral finite-rotation mesh is available.

As with all triangular shells, refinement and mesh-quality checks matter. Inspect rotations, reactions, and shell resultants in nonlinear analyses because convergence alone does not prove that a coarse triangular shell mesh gives a reliable bending response.

6.16 S4: 4-node quadrilateral shell



Element	S4
Type	shell
Nodes	4
Faces	-
Edges	4
DOF / node	6
Total DOF	24
Shape functions	bilinear quadrilateral shell
Integration points	4

Figure 6.13: S4 quadrilateral shell element.

Syntax

```
*ELEMENT, TYPE=S4, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>
```

S4 is the legacy four-node quadrilateral shell element. Quadrilateral topology aligns naturally with plates, panels, webs, skins, and other structural surfaces where two in-plane directions are meaningful, but this legacy formulation should not be used as the analysis element for new decks.

Do not use S4 for actual analysis work. It is retained as a legacy element and as a reference topology. Use MITC4FRT instead for four-node quadrilateral shell analyses.

Compared with triangular shells, quadrilateral shells usually provide smoother membrane and bending fields for the same number of elements, provided the mesh quality is good. For new models this topology should be represented with MITC4FRT or MITC8FRT, not with S4.

The limitations are connected to low-order interpolation, linear shell kinematics, and mesh distortion. A coarse S4 mesh can be too stiff in thin bending problems, and strongly warped or skewed quadrilaterals can reduce both stiffness accuracy and stress recovery quality. Use the FRT family instead of trying to make S4 suitable through refinement alone.

When converting old S4 decks, preserve node order, aspect ratio, warping, and normal direction while changing to MITC4FRT. For large panel models, refine gradually and compare reactions, deflections, and section resultants rather than relying only on local peak stress values.

6.17 MITC4: quadrilateral shell with tied shear interpolation

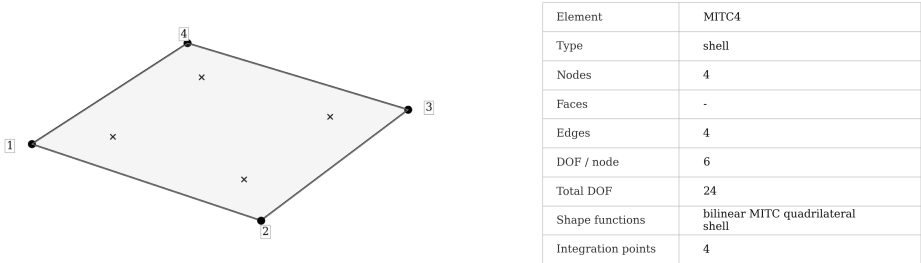


Figure 6.14: MITC4 shell element.

Syntax

```
*ELEMENT, TYPE=MITC4, ELSET=<set>  
<id>, <n1>, <n2>, <n3>, <n4>
```

MITC4 has the same four-node quadrilateral topology as S4, but modifies the transverse shear interpolation. FEMaster evaluates shear-related quantities at tied locations and interpolates them back into the element. The purpose is to reduce shear locking, which can otherwise make low-order shell elements artificially stiff in thin bending-dominated structures.

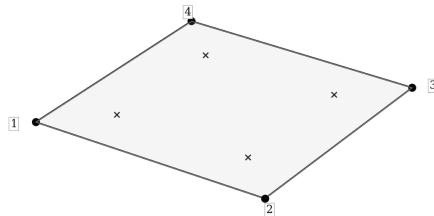
Do not use MITC4 for actual analysis work. It is retained mainly for legacy comparison and regression decks. Use the finite-rotation MITC4FRT element instead.

Historically this element was used as the safer low-order quadrilateral shell when the structure was thin, the mesh was relatively coarse, or bending response was more important than membrane response. New decks should use MITC4FRT for that role.

MITC4 does not remove the need for a good mesh. Severe warping, poor aspect ratios, inconsistent normals, and abrupt size changes can still damage the solution. It also remains a low-order element, so smooth curvature and stress gradients still require refinement. The tied shear interpolation addresses a specific numerical weakness; it does not make the element insensitive to modelling choices.

When comparing S4 and MITC4, check global displacement, reaction forces, and section-resultant patterns. If S4 appears too stiff in a thin-shell bending problem, MITC4 is usually worth trying. If both elements give similar behaviour on a refined mesh, the simpler quadrilateral shell may be sufficient.

6.18 MITC4FRT: finite-rotation 4-node MITC shell



Element	MITC4FRT
Type	shell
Nodes	4
Faces	-
Edges	4
DOF / node	6
Total DOF	24
Shape functions	finite-rotation MITC4 shell
Integration points	4

Figure 6.15: MITC4FRT quadrilateral finite-rotation shell element.

Syntax

```
*ELEMENT, TYPE=MITC4FRT, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>
```

MITC4FRT is the geometrically nonlinear finite-rotation counterpart of the four-node MITC quadrilateral shell. It uses bilinear quadrilateral interpolation, cubic quadrilateral area integration, and the classical MITC4 transverse-shear interpolation between four edge-midpoint tying positions. The assumed shear values and their B-matrix rows use the same interpolation, while geometric-tangent resultants are pulled back with the transposed operator.

This element is the low-order quadrilateral choice for large-rotation shell analyses. It is useful for thin panels, skins, webs, and other shell structures where a standard linear kinematics assumption is not adequate but a robust four-node topology is desired. It also provides a practical nonlinear reference for comparing S4, MITC4, and higher-order shell meshes.

The formulation addresses shear locking, not all mesh pathologies. Warping, skew, inconsistent normals, abrupt thickness changes, and coarse discretization can still dominate the result. In nonlinear work, validate load paths with incremental deformation shapes, reactions, and shell-resultant signs.

6.19 S6: 6-node quadratic triangular shell

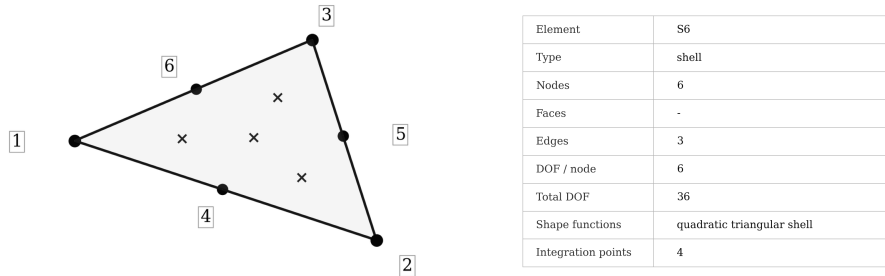


Figure 6.16: S6 quadratic triangular shell element.

Syntax

```
*ELEMENT, TYPE=S6, ELSET=<set>
<id>, <n1>, ..., <n6>
```

S6 is the legacy quadratic triangular shell element. It extends the triangular shell topology with midside nodes and higher-order interpolation, but it should be treated as a documented compatibility type rather than as a recommended analysis element.

Do not use S6 for actual analysis work. It is retained as a legacy element and as a reference topology. Use MITC6FRT instead when a six-node triangular shell is required.

In old models this element may appear where triangular topology is unavoidable but the response requires better bending or stress-resultant quality than a low-order triangle can provide. For new models, use MITC6FRT for curved triangular shell surfaces, irregular imported meshes, and local features where quadrilateral meshing is not practical.

The cost and geometric sensitivity are higher than for S3. Midside-node placement must be consistent with the intended surface; otherwise, the element can become geometrically distorted even if the corner triangle looks acceptable. A poor higher-order triangle can be less reliable than a cleaner low-order element.

Use MITC6FRT when the surface geometry or solution field justifies quadratic triangular interpolation. The legacy S6 type is not a reason to ignore mesh quality or convergence checks. For broad regular shell regions, a high-quality quadrilateral FRT mesh may still be more efficient.

6.20 MITC6FRT: finite-rotation 6-node MITC6-b shell

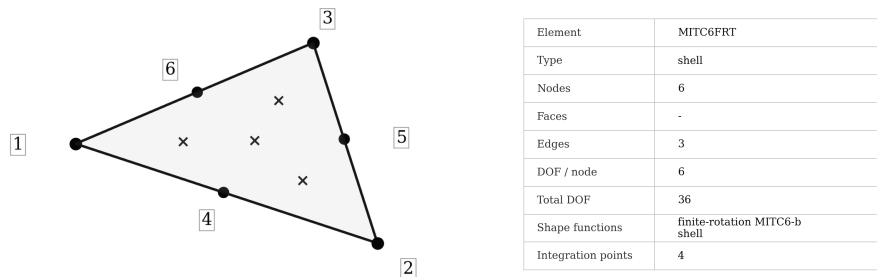


Figure 6.17: MITC6FRT quadratic triangular finite-rotation shell element.

Syntax

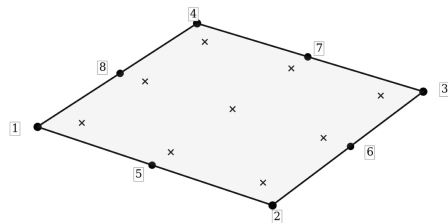
```
*ELEMENT, TYPE=MITC6FRT, ELSET=<set>
<id>, <n1>, ..., <n6>
```

MITC6FRT is a six-node finite-rotation triangular shell with quadratic triangular interpolation. Its assumed in-plane strain field follows the isotropic triangular MITC interpolation, while transverse shear uses the MITC6-b linear interpolation with two tying positions on each edge. Membrane strain and curvature are treated independently through the same in-plane operator.

Use this element for geometrically nonlinear shell models where triangular topology is unavoidable but a higher-order midsurface representation is needed. It is especially relevant for curved triangular shell meshes, transition zones, and imported nonlinear shell models that cannot be represented cleanly with quadrilateral elements.

The element is more capable than MITC3FRT, but also more sensitive to midside-node placement and curved-surface mapping. A distorted quadratic triangle can still give poor Jacobians or unreliable shell resultants. Check element quality and compare against refined meshes when bending or stress recovery is important.

6.21 S8: 8-node quadratic quadrilateral shell



Element	S8
Type	shell
Nodes	8
Faces	-
Edges	4
DOF / node	6
Total DOF	48
Shape functions	quadratic serendipity shell
Integration points	9

Figure 6.18: S8 quadratic quadrilateral shell element.

Syntax

```
*ELEMENT, TYPE=S8, ELSET=<set>
<id>, <n1>, ..., <n8>
```

S8 is the legacy higher-order quadrilateral shell element. It uses midside nodes and quadratic interpolation, but it should be treated as a documented compatibility type rather than as a recommended analysis element.

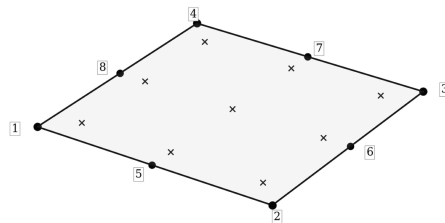
Do not use S8 for actual analysis work. It is retained as a legacy element and as a reference topology. Use MITC8FRT instead when an eight-node quadrilateral shell is required.

In a high-quality mesh, the eight-node quadrilateral topology can represent curvature, displacement gradients, and section-resultant variation more efficiently than a four-node topology. For new shell analyses, use MITC8FRT to get that topology with the finite-rotation shell formulation.

The disadvantages are cost and sensitivity to element shape. Warped or distorted higher-order quadrilaterals can produce unreliable results, and incorrectly placed midside nodes can damage the geometric mapping. A higher-order element is not automatically better if the mesh quality is poor.

Use MITC8FRT when the shell surface is smooth and the mesh generator can produce clean curved quadrilaterals. For rough imported meshes, abrupt topology changes, or strongly irregular panels, a refined lower-order FRT mesh may be easier to validate. As with solid higher-order elements, convergence should be checked using global and integrated quantities as well as local stress measures.

6.22 MITC8: 8-node quadratic shell with tied shear interpolation



Element	MITC8
Type	shell
Nodes	8
Faces	-
Edges	4
DOF / node	6
Total DOF	48
Shape functions	quadratic MITC8 shell
Integration points	9

Figure 6.19: Quadratic eight-node topology used by the MITC8 shell element.

Syntax

```
*ELEMENT, TYPE=MITC8, ELSET=<set>
<id>, <n1>, ..., <n8>
```

MITC8 combines the quadratic eight-node serendipity topology of **S8** with tied transverse-shear interpolation. For each covariant shear component, four values are evaluated at edge tying points and a fifth value is formed as the mean of two internal tying points. These values are interpolated in the natural element coordinates and transformed back into the local shell coordinates. Their order is γ_{13}, γ_{23} , energetically conjugate to Q_{13}, Q_{23} .

Do not use MITC8 for actual analysis work. It is retained mainly for legacy comparison and validation decks. Use MITC8FRT instead for eight-node MITC shell analyses.

The legacy element was intended for smooth quadratic shell meshes where transverse-shear locking would otherwise affect bending-dominated results. For new decks, use MITC8FRT; it provides the corresponding eight-node MITC shell path in the finite-rotation family.

The current MITC8 implementation uses the linear shell formulation. Geometrically nonlinear analyses should use an element with an explicitly supported nonlinear shell path. Mesh quality remains important: warped elements, poor midside-node placement, inconsistent normals, and abrupt element-size changes can still degrade the solution.

Use the same node order as for **S8**: four corner nodes followed by the four midside nodes around the perimeter. Validate new models with the FRT family by comparing displacement, reactions, and shell resultants against refinement or a solid reference model.

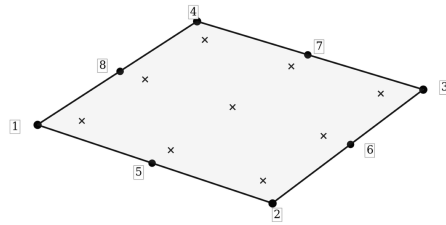
The example decks 039 through 043 compare **S4**, **MITC4**, **MITC4FRT**, **S8**, and **MITC8** on a ten-element cantilever strip. For $L = 1000$ mm, $b = 100$ mm, $t = 10$ mm, $E = 210000$ N/mm², $\nu = 0$, and $P = 100$ N, Timoshenko beam theory predicts a tip displacement magnitude of 19.0487619 mm. The calculated values are summarized below.

Element	Tip displacement [mm]	Error [%]	Tip rotation [rad]
S4	0.446455	-97.6563	0.00066964
MITC4	19.001140	-0.2500	0.02857143
MITC4FRT	19.001140	-0.2500	0.02857143
S8	19.006240	-0.2232	0.02857143
MITC8	19.048760	-0.0000	0.02857143

Table 6.2: Ten-element shell-strip cantilever compared with Timoshenko beam theory.

The comparison exposes severe transverse-shear locking in the coarse **S4** mesh. The tied-shear and quadratic formulations closely reproduce the analytical bending response, while **MITC8** also recovers the analytical root moment resultant and extreme-fibre stress to the printed result precision. The **MITC4FRT** displacement and reactions agree with **MITC4**, but its recovered bending resultants and top/bottom bending stresses currently have the opposite sign convention.

6.23 MITC8FRT: finite-rotation 8-node MITC shell



Element	MITC8FRT
Type	shell
Nodes	8
Faces	-
Edges	4
DOF / node	6
Total DOF	48
Shape functions	finite-rotation MITC8 shell
Integration points	9

Figure 6.20: MITC8FRT quadratic quadrilateral finite-rotation shell element.

Syntax

```

*ELEMENT, TYPE=MITC8FRT, ELSET=<set>
<id>, <n1>, ..., <n8>

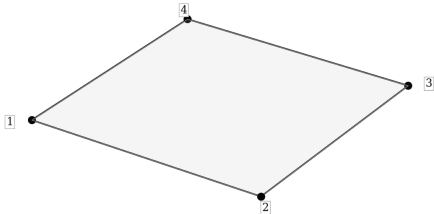
```

MITC8FRT is the eight-node finite-rotation member of the nonlinear MITC shell family. It uses quadratic serendipity interpolation on the same node ordering as **S8** and **MITC8**. The element applies the classical MITC8 construction: in-layer sampling tensors are interpolated over the reduced square, and the two transverse-shear fields use boundary sampling plus one internal coefficient formed from interior sampling values.

This is the preferred FRT shell when a clean quadratic quadrilateral mesh is available and the analysis involves large rotations. The midside nodes improve geometric representation and bending-field smoothness, while the finite-rotation kernel provides the nonlinear kinematics missing from the linear **MITC8** path.

MITC8FRT is still not a cure for poor shell meshes. Warped high-order quads, poor midside-node placement, inconsistent normals, or severe element distortion can produce misleading nonlinear stiffness and shell-resultant recovery. Validate large-rotation models with deformation-shape checks, reaction balance, and mesh refinement.

6.24 QSPT: quadrilateral shear-panel type



Element	QSPT
Type	shell
Nodes	4
Faces	-
Edges	4
DOF / node	3
Total DOF	12
Shape functions	quadrilateral shear-panel formulation
Integration points	-

Figure 6.21: QSPT shear-panel element.

Syntax

```

*ELEMENT, TYPE=QSPT, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>

```

QSPT is a specialized four-node quadrilateral panel element nach Bäß, the inventor of the element developed at SLA. It is intended for shear-flow-oriented structural idealizations and should not be treated as a general shell replacement. Its purpose is to represent panel-like shear transfer in simplified structural models where shear flow or effective panel stiffness is the main quantity of interest.

This kind of element is useful in aircraft-style thin-walled structures, webs, simplified stiffened panels, or preliminary models where the detailed shell kinematics are less important than the panel shear path. It can provide a compact representation of shear transfer without requiring a full shell formulation for every panel feature.

The limitation is generality. QSPT is not meant to capture the same bending, rotational, and shell-resultant behaviour as S4, MITC4, or S8. If the model depends on bending stiffness, drilling behaviour, local shell curvature, or detailed shell stresses, a standard shell element is the more appropriate choice.

Use QSPT only when the engineering idealization matches the element behaviour. It is valuable when a panel is intended to carry shear in a simplified structural model. It is risky when used simply because it has four nodes and resembles a quadrilateral shell in the mesh.

6.25 Beam and truss element family

Line elements reduce a three-dimensional structural member to a one-dimensional representation with section properties. They are efficient for slender members because the cross-section is described by a section definition rather than by a solid or shell mesh. This makes them useful for frames, rods, stiffeners, trusses, spars, and reinforcement members.

The price of this efficiency is the modelling assumption. Local three-dimensional stress states, connection details, bolt holes, weld geometry, local buckling modes, and cross-section deformation are not resolved unless the model adds local detail elsewhere. Line elements are best used when member forces and global stiffness are the design quantities.

6.26 B33: 3D beam element

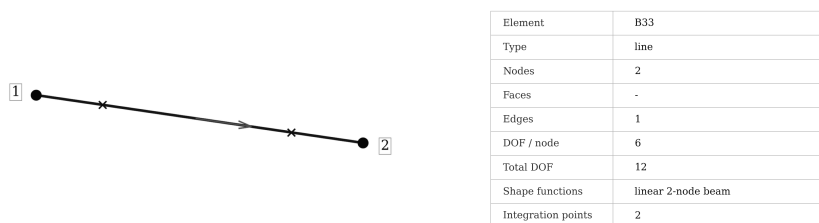


Figure 6.22: B33 beam element.

Syntax

```
*ELEMENT, TYPE=B33, ELSET=<set>
<id>, <n1>, <n2>, <orientation-node>
```

B33 is a two-node 3D beam element with six degrees of freedom per node. It represents axial deformation, bending, torsion, and section orientation through a beam-section description. The optional third connectivity entry is an orientation node used to define the local beam-axis system.

The element is appropriate for slender structural members where section forces are the natural engineering output. Frames, stiffeners, spars, ribs, support structures, and idealized reinforcements can often be modelled with far fewer degrees of freedom than a comparable shell or solid model. Because the section properties are defined explicitly, the model can represent axial stiffness, bending inertias, torsional stiffness, principal-axis orientation, and offsets without meshing the full cross-section.

The main limitation is the beam assumption. B33 does not resolve local stress concentrations, cross-section warping beyond the supported formulation, detailed joint flexibility, bolt patterns, weld geometry, or local load introduction effects. If the member is short and thick, if the joint detail dominates the response, or if local shell or solid stresses are required, a beam idealization may be too coarse.

The local axis definition must be robust. The orientation direction should not be parallel to the beam axis, and adjacent beams should use consistent orientations when section-force interpretation matters. After building a beam model, inspect local axes, reactions, and internal forces before relying on stress-like quantities derived from the section.

6.27 T3: 2-node truss

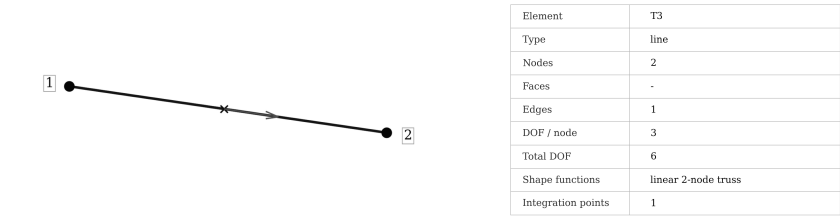


Figure 6.23: T3 truss element.

Syntax

```

*ELEMENT, TYPE=T3, ELSET=<set>
<id>, <n1>, <n2>
    
```

T3 is a two-node truss element. It carries axial force along its line direction and uses translational degrees of freedom only. The element requires `*TRUSSSECTION`, which supplies material and cross-sectional area.

The element is appropriate for members intended to carry tension or compression without bending moment transfer. Pin-jointed structures, rods, links, braces, idealized cables, and simplified support members are typical use cases. Because the formulation is simple, truss models can be very efficient and easy to interpret: the important output is usually axial force, axial strain, and the corresponding stress based on section area.

The limitations are deliberate. T3 has no bending stiffness, no torsional stiffness, and no rotational continuity. It cannot represent frame action, fixed-end moments, eccentric connections, or transverse stiffness except through the geometry of the truss network. If a physical member can transfer moment or has important bending response, use a beam or shell model instead.

Truss models are sensitive to structural mechanism behaviour. A collection of axial-only members must be geometrically stable under the applied supports and loads. If the model contains mechanisms, the solver may report singularity or produce unrealistic displacements. Before using truss results for design, check that the assumed pin-jointed idealization is actually consistent with the structure being represented.

6.28 Choosing an element

Element selection should follow the physical idealization before it follows meshing convenience. Use solids when the stress state is three-dimensional or when local geometry and through-thickness behaviour matter. Use shells when the structure is thin-walled and section resultants are meaningful. Use beams when members are slender and section forces are the primary design quantities. Use trusses only when axial force is the intended behaviour.

For solid meshes, prefer hexahedra when a high-quality structured mesh is available, use quadratic tetrahedra for serious stress work on complex geometry, and treat pyramids mainly as transition elements. For shell meshes, use the finite-rotation family: prefer quadrilateral FRT shells in regular regions, use triangular FRT shells where topology requires them, and choose **MITC8FRT** when clean higher-order quadrilateral meshing is available. For line models, verify that the connection assumptions match the physical structure.

No element choice removes the need for validation. Check reactions, displacement shapes, load paths, mesh sensitivity, and stress convergence. A good element used outside its assumptions can be less reliable than a simpler element used in a model that matches the physics.

6.29 Shell directors and prescribed normals

Shell elements require a director field that defines the local through-thickness direction. If no explicit director field is supplied, FEMaster constructs shell normals from the element geometry and equalises neighbouring directions where required by the shell-normal build procedure.

For imported or deliberately oriented shell meshes, the director can be prescribed with an element-nodal field. The field is defined first and then selected with ***NORMAL**.

Syntax

```
*FIELD, NAME=<name>, TYPE=ELEMENT_NODAL, COLS=3, FILL=NAN
<element-id>, <local-node>, <nx>, <ny>, <nz>

*NORMAL, FIELD=<name>
```

The local node index is zero-based. The selected field must use the **ELEMENT_NODAL** domain and contain exactly three components. ***NORMAL** does not create, normalize, or average the supplied vectors; it only selects the field that is used during shell-normal construction. Using **FILL=NAN** is useful for partially prescribed director fields because entries that are not supplied remain distinguishable from explicitly prescribed values and can be completed by the geometric shell-normal construction.

Example

```
*FIELD, NAME=DIRECTORS, TYPE=ELEMENT_NODAL, COLS=3, FILL=NAN
10, 0, 0.0, 0.0, 1.0
10, 1, 0.0, 0.0, 1.0
10, 2, 0.0, 0.0, 1.0
10, 3, 0.0, 0.0, 1.0
*NORMAL, FIELD=DIRECTORS
```

The field must be defined before ***NORMAL** in the input deck. The parser resolves enumeration-dependent fields after element topology has been built and before the final model data pass, so the selected director field is available when shell normals are completed.

7 Surfaces

Surfaces describe loadable or connectable element sides. They are used for pressure loads, distributed surface loads, tie constraints, surface couplings, and surface-to-surface contact. A surface is a selection of element sides; it is not a material or section property.

7.1 Surface from one element side

Syntax

```
*SURFACE, NAME=<surface-set>  
<surface-id>, <element-id>, <side>
```

The side can be written as a number or as a token such as **S1** or **S2**. For shells, **SPOS** and **SNEG** select the positive or negative shell side.

7.2 Surface from an element set

Syntax

```
*SURFACE, NAME=<surface-set>, TYPE=ELEMENT  
<element-set-or-id>, <side>
```

If the target is an element set, FEMaster creates surfaces for the selected side of all elements in the set. Otherwise the target is interpreted as a single element ID.

7.3 Orientation and sign

Surface loads and contact depend on the chosen side and its normal direction. The normal follows the local face orientation implied by the element connectivity and selected side. If pressure or traction direction is unexpected, check the selected side, element orientation, and shell side. For shells, positive and negative sides share the same geometric midsurface but have opposite normals.

Surface orientation is particularly important for ***CONTACT**. The dual-mortar formulation uses current master and slave facet normals when constructing the common-plane segmentation and signed normal gap. A master surface that is oriented opposite to the intended contact direction can be corrected for the contact pair with **FLIP=YES**; this does not change the underlying element connectivity or surface definition.

Contact surfaces

The current ***CONTACT** interface requires both **MASTER** and **SLAVE** to be surface sets. Node-set slaves and the older search-distance form are not supported by the current dual-mortar contact DSL.

8 Sets

Sets are named collections of model IDs. They are one of the most important readability tools in a FEMaster input deck because most higher-level commands should target named sets instead of repeating long lists of node, element, or surface IDs. A set gives a modelling meaning to otherwise anonymous mesh numbers.

The mesh IDs themselves are usually not stable engineering concepts. They can change when a part is remeshed, when an external preprocessor exports a new deck, or when local refinement is added. A set name such as `FIXED_EDGE`, `WING_SKIN`, `LOAD_PATCH`, or `BOLT_SURFACES` is much more meaningful than a raw list of IDs. The name records why those entities are grouped, not only which numbers happened to exist in one version of the mesh.

Sets are also useful because FEMaster commands often operate on groups. Supports, loads, sections, fields, surface interactions, couplings, and diagnostics all become easier to audit when their targets are named explicitly. A deck that assigns a shell section to `PANEL_TOP_SKIN` or a support to `ROOT_CLAMP_NODES` can be reviewed without constantly cross-checking individual IDs against the mesh.

8.1 *NSET: node sets

Syntax

```
*NSET, NAME=<name>, GENERATE  
<id-start>, <id-end>, <increment>
```

A node set groups node IDs. If `GENERATE` is present, FEMaster interprets the data line as a range with start ID, end ID, and increment. Without `GENERATE`, the following data lines are read as explicit node IDs. Generated sets are compact and convenient for regular ID ranges, while explicit sets are better for arbitrary selections imported from a preprocessor.

Node sets are used wherever the model needs to refer to points in the discretization. Supports and concentrated loads commonly target node sets. Couplings, rigid-body-style constraints, connector definitions, point masses, diagnostic output, and result extraction workflows also use them. A good node set name should describe the physical or modelling role of the nodes, such as `FIXED_ROOT`, `LOAD_NODES`, `COUPLING_SLAVES`, or `MASS_POINTS`.

Node sets should be chosen with the intended degree-of-freedom behaviour in mind. A set applied to solid-only nodes usually affects translational degrees of freedom. A set applied to shell or beam nodes may involve rotations as well. If a support or load appears to have no effect, verify that the targeted node set contains the expected nodes and that those nodes actually own the degrees of freedom referenced by the command.

It is normal for one node to belong to several node sets. For example, a node can be part of a geometric set, a load-introduction set, and a diagnostic output set at the same time. This is not a problem by itself. The important question is whether the commands using those sets create consistent boundary conditions and loads. Overlapping sets become dangerous only when they accidentally apply contradictory constraints or duplicate loads.

Example

```
*NSET, NAME=FIXED_ROOT, GENERATE  
1, 41, 1
```

Example

```
*NSET, NAME=LOAD_PATCH  
105, 106, 112, 113, 119, 120
```

8.2 *ELSET: element sets

Syntax

```
*ELSET, NAME=<name>, GENERATE  
<id-start>, <id-end>, <increment>
```

An element set groups element IDs. As with node sets, **GENERATE** creates a compact range, while explicit data lines list individual element IDs. Element sets are central to physical model definition because sections are assigned to element sets. Without meaningful element sets, material and section assignment becomes difficult to review.

Element sets are used by solid, shell, beam, and truss sections, volume loads, inertial loads, topology fields, RBM constraints, and many postprocessing or diagnostic workflows. They should normally reflect physical regions or modelling roles: **SOLID_CORE**, **SHELL_SKIN**, **SPAR_BEAMS**, **TRUSS_BRACES**, **TOPO_DOMAIN**, or **GRAVITY_ELEMENTS**. Names like these make it clear why the elements are grouped and which later commands are expected to reference them.

One element may belong to several element sets. This is useful when the same elements need to be referenced for different reasons. A shell panel might belong to **PANEL_A**, **ALL_SHELLS**, and **AERO_LOAD_REGION**. A topology-design domain might also belong to a material section set. This kind of overlap is expected as long as the commands using the sets are compatible.

The main risk with element sets is unintended multiple physical assignment. If the same element is assigned to two different sections that are both meant to define its physical stiffness, the deck becomes ambiguous from a modelling perspective. Even if the parser accepts the input order, the model is difficult to audit. Keep physical section sets clear and non-overlapping unless there is a deliberate reason and the command semantics support it.

Generated element sets are convenient for simple meshes, but they can be fragile after remeshing. If element IDs change, a generated range may still exist but refer to the wrong region. For production decks, prefer sets exported from the preprocessor by named geometric selections when possible. The model should be organized around physical regions, not around accidental ID ranges.

Example

```
*ELSET, NAME=SOLID_CORE, GENERATE  
1, 2400, 1
```

Example

```
*ELSET, NAME=SPAR_BEAMS  
5101, 5102, 5103, 5104
```

8.3 *SFSET: surface sets

Syntax

```
*SFSET, NAME=<name>, GENERATE  
<id-start>, <id-end>, <increment>
```

A surface set groups surface IDs. Surface IDs are created by surface definitions and represent element faces, shell sides, or other surface-like model entities depending on the surrounding input. A surface set is therefore one level above the raw element topology: it names the boundary or interface region that later commands should use.

Surface sets are typically used by pressure loads, distributed loads, ties, contact-like workflows, and surface couplings. Commands such as *PLOAD, *DLOAD, *TIE, and coupling definitions become much clearer when they target names like PRESSURE_SIDE, MASTER_SURFACE, SLAVE_SURFACE, BOLT_CONTACT, or LOAD_INTRODUCTION_SURFACE.

The orientation of a surface matters. A surface is not only a collection of IDs; it also carries a geometric meaning through the underlying face orientation and normal direction. For pressure loads and interface constraints, the selected side of an element can change the physical interpretation. If a pressure has the wrong sign or a tie behaves unexpectedly, inspect both the surface set membership and the underlying surface orientation.

Surface sets can overlap, but overlapping surfaces should be reviewed carefully. A surface might reasonably belong to both a broad diagnostic set and a specific load set. It is less reasonable for the same surface region to accidentally receive two unrelated pressure definitions or to act as both sides of an interface constraint. Use narrow, descriptive names for operational surface sets so that accidental reuse is easier to spot.

As with element sets, generated surface ranges are compact but not inherently meaningful. They are useful for controlled meshes and hand-written examples. In larger workflows, named geometric selections exported from preprocessing are usually safer because they preserve the modelling intent when IDs change.

Example

```
*SFSET, NAME=PRESSURE_FACE, GENERATE  
1001, 1040, 1
```

Example

```
*SFSET, NAME=TIE_SLAVE_SURFACE  
2201, 2204, 2210, 2212
```

8.4 Generated and explicit sets

The **GENERATE** option is a compact way to define regular ID ranges. It is well suited to small examples, controlled test decks, and meshes whose numbering scheme is intentionally structured. A generated set is easy to read when the range corresponds to a physical region, such as all nodes along a simple edge or all elements in a block.

Explicit sets are more verbose, but they are often safer for arbitrary selections. If a preprocessor exports a named group of nodes, elements, or surfaces, the IDs may not form a simple range. In that case, an explicit list preserves exactly what was selected. Explicit sets are also useful for small hand-picked groups such as coupling reference nodes, load introduction nodes, or selected diagnostic elements.

The main disadvantage of both forms is that they still depend on numeric IDs. If the mesh is regenerated, the set content must be regenerated or checked. A generated range is especially easy to misuse after remeshing because it may still look plausible. Treat set definitions as part of the mesh data, not as permanent engineering truth independent of the mesh.

8.5 Naming conventions

Set names should describe function and physical meaning rather than only geometry. A name like **NODES_1** or **SET_A** provides almost no review value. A name like **FIXED_ROOT_NODES**, **PRESSURE_UPPER_SKIN**, **BOLT_HOLE_SURFACES**, or **SPAR_CAP_ELEMENTS** tells the reader what the set is for and why later commands reference it.

Names should also be specific enough to avoid accidental reuse. If a model has several load patches, **LOAD** is too broad. Use names such as **LOAD_PATCH_FRONT**, **LOAD_PATCH_REAR**, or **ACTUATOR_LOAD_NODES**. If a set is used only for output or diagnostics, include that in the name, for example **TIP_OUTPUT_NODES**. This prevents diagnostic sets from being mistaken for physical boundary-condition sets.

Consistency matters more than any one naming scheme. Choose a convention that distinguishes node sets, element sets, and surface sets clearly. Some decks use suffixes such as **_NODES**, **_ELEMENTS**, and **_SURFACES**; others rely on command context. Suffixes are verbose but helpful in large models because they make accidental cross-use easier to see.

8.6 Checking set usage

Set definitions should be reviewed whenever the mesh or preprocessing workflow changes. The solver can only operate on the IDs it receives; it cannot know whether a set still represents the physical region its name implies. A remeshed model may preserve a set name while changing its membership, or it may preserve numeric ranges while changing their geometric meaning.

Useful checks include verifying that every physical element region is covered by the intended section set, that every support and load set contains the expected nodes or surfaces, and that no operational set is empty. Empty sets are especially dangerous because they can make a command appear present in the deck while having no physical effect. Very small or unexpectedly large sets should also be treated as suspicious until confirmed.

For complex decks, it is often helpful to keep broad organizational sets separate from operational sets. A broad set such as **ALL_SHELLS** can be useful for output or global checks, while a narrow set such as **ROOT_CLAMP_NODES** should be used for a specific support. This separation makes the input deck easier to audit and reduces the risk of applying a command to more of the model than intended.

9 Sections and Profiles

Sections connect element sets to the physical data that the element topology alone does not contain. An element defines interpolation, degrees of freedom, and numerical integration; a section tells FEMaster which material, thickness, area, profile constants, or local orientation should be used for a particular element set. Without a matching section, elements may exist geometrically in the model, but they cannot contribute the intended stiffness, mass, stress recovery, or section-force output.

The separation between elements and sections is deliberate. A mesh can be grouped into named element sets, and each set can then receive the section definition that matches its physical role. This allows the same element type to represent different parts, materials, thicknesses, or idealizations without changing the mesh connectivity. It also makes the input deck easier to audit: element commands describe topology, while section commands describe physical interpretation.

Section definitions are normally applied to element sets through **ELSET**. The referenced set should contain only elements compatible with the section type. A solid section is meaningful for solid elements, a shell section for shell elements, a beam section for beam elements, and a truss section for truss elements. Mixing unrelated element families in the same section set is usually a modelling error even if the parser can read the command.

9.1 *SOLIDSECTION: solid sections

Syntax

```
*SOLIDSECTION, ELSET=<elset>, MATERIAL=<material>, ORIENTATION=<orientation>
```

A solid section assigns material behaviour to a set of three-dimensional continuum elements. It is used with the solid element family, such as C3D4, C3D8, C3D8R, C3D10, C3D20, and related transition elements. The section does not define a thickness or area because those quantities are already represented by the volume mesh. The physical volume, element shape, and integration rules provide the geometric contribution; the section supplies the constitutive interpretation.

MATERIAL is the central parameter. It references a material definition that provides elastic stiffness, density, or other material data used by the selected analysis type. The material assigned through the solid section affects stiffness assembly, mass assembly where density is available, stress recovery, and any loadcase that depends on material response.

ORIENTATION is optional. For isotropic materials it often has no practical effect because the material response is direction-independent. For orthotropic, anisotropic, or otherwise direction-dependent material models, however, orientation is part of the physical model. It defines how the material axes are placed relative to the element or global coordinate system. A wrong orientation can produce a model that is numerically stable but physically incorrect, especially in composite-like solids, layered inserts, printed structures, or directional materials.

Solid sections are usually simple, but they are easy to misuse in large models. The most common problem is an incomplete element-set assignment: some elements exist in the mesh but are not part of any solid section set, or are assigned to the wrong material. Another common issue is accidental reuse of a generic

set name when different regions should receive different materials. For serious models, inspect the section coverage and verify that each solid region has exactly the intended material and orientation.

Example

```
*SOLIDSECTION, ELSET=BLOCK, MATERIAL=STEEL
```

Example

```
*SOLIDSECTION, ELSET=ORTHO_CORE, MATERIAL=CORE_MAT, ORIENTATION=CORE_AXES
```

9.2 *SHELLSECTION: shell sections

Syntax

```
*SHELLSECTION, ELSET=<elset>, TYPE=INTEGRATED, MATERIAL=<material>, THICKNESS=<t>,
    ORIENTATION=<orientation>, CSYSAXIS=<1|2|3>
<thickness>

*SHELLSECTION, ELSET=<elset>, TYPE=ABD, THICKNESS=<t>, ORIENTATION=<orientation>, CSYSAXIS
    =<1|2|3>
<abd11>, <abd12>, ..., <abd66>, <s11>, <s12>, <s21>, <s22>
```

A shell section assigns a section formulation to a set of shell elements. Shell elements represent a thin structure through a two-dimensional surface mesh, so the section carries information that would otherwise be present in a full three-dimensional solid mesh. **TYPE** selects how the generalized membrane, bending and transverse-shear response is obtained. **INTEGRATED** is the default and evaluates an assigned material through the physical thickness. **ABD** reads prescribed generalized section stiffness matrices directly from the section command.

TYPE=INTEGRATED requires **MATERIAL**. The data line following the command supplies the physical shell thickness. The keyword **THICKNESS** also defaults to 1.0, but the data-line form is the authoritative thickness for the integrated variant. The material is evaluated at through-thickness points and integrated into generalized shell resultants and a consistent section tangent.

TYPE=ABD reads 40 values after the command. The first 36 values form a 6×6 ABD matrix in row-major order. The final 4 values form the 2×2 transverse-shear stiffness matrix in row-major order. These values may be distributed over up to eight data lines. **MATERIAL** is optional for ABD sections because the prescribed matrices already define the generalized elastic response; a material may still be referenced when shared material data such as density is desired.

Thickness is not just a display property. For integrated shells, membrane stiffness generally scales with thickness, while bending stiffness is much more sensitive because it depends on the thickness cubed in classical plate behaviour. For ABD shells, the prescribed ABD and shear matrices are already section stiffnesses; FEMaster does not multiply them by **THICKNESS**. The thickness is still used for shell mass, geometric thickness coordinates, and equivalent stress recovery. In all cases the thickness must be positive and expressed in the same length unit as the node coordinates and the rest of the model.

MATERIAL references the material used by an integrated shell section. For isotropic shells, the material and thickness together define the main stiffness behaviour. For orthotropic materials or ABD-type section descriptions, the shell orientation becomes part of the physical definition. It controls how material axes, laminate axes, or directional stiffness terms are aligned with the shell surface.

ORIENTATION is optional but important whenever the material or prescribed section stiffness is direction-dependent. **CSYSAXIS** selects which axis of the coordinate system is projected into the shell tangent plane to define the first local section axis; valid values are 1, 2, and 3. If the selected axis is perpendicular to a shell surface, FEMaster reports an error instead of silently choosing another direction. Without an explicit orientation, physical stress output is written in global Cartesian components, while shell resultants use a deterministic tangent output basis constructed from the global X axis and, if needed, the global Y axis.

Shell sections should be defined at the level of physical regions. If a panel has different thickness zones, each zone should normally receive its own element set and shell section. If a model uses offsets or more advanced section behaviour elsewhere in the input language, those settings should be kept consistent with the geometric idealization of the midsurface. A shell surface placed at a physical midplane behaves differently from a shell surface placed on an outer skin unless the section definition accounts for that modelling choice.

Example

```
*SHELLSECTION, ELSET=SKIN_TOP, TYPE=INTEGRATED, MATERIAL=AL2024  
1.6
```

Example

```
*SHELLSECTION, ELSET=PANEL_45, TYPE=INTEGRATED, MATERIAL=ORTHO_LAMINA, ORIENTATION=PANEL_AXES  
    , CSYSAXIS=1  
0.8
```

Example

```
*SHELLSECTION, ELSET=LAMINATE_PANEL, TYPE=ABD, THICKNESS=1.0, ORIENTATION=LAMINATE_AXES  
1.0e6, 2.5e5, 0.0, 0.0, 0.0, 0.0  
2.5e5, 1.0e6, 0.0, 0.0, 0.0, 0.0  
0.0, 0.0, 3.0e5, 0.0, 0.0, 0.0  
0.0, 0.0, 0.0, 1.0e3, 2.5e2, 0.0  
0.0, 0.0, 0.0, 2.5e2, 1.0e3, 0.0  
0.0, 0.0, 0.0, 0.0, 0.0, 3.0e2  
3.0e5, 0.0, 0.0, 3.0e5
```

9.3 *PROFILE: Beam profiles

Syntax

```
*PROFILE, NAME=<name>
<A>, <Iy>, <Iz>, <J>, <Iyz>, <y0>, <z0>, ...
```

A beam profile stores the cross-section constants used by beam sections. Beam elements reduce a three-dimensional member to a line, so the shape of the cross-section must be represented by section properties rather than by mesh geometry. The profile is the place where those properties are collected and named.

The most important values are the cross-sectional area A , the bending inertias I_y and I_z , the torsional constant J , and the product inertia I_{yz} . Area controls axial stiffness and mass contribution when density is available. The bending inertias control bending stiffness about the local section axes. The torsional constant controls twist stiffness. The product inertia is relevant when the chosen local axes are not principal axes of the section.

The coordinates y_0 and z_0 , where used, describe section reference offsets in the local beam-section coordinate system. Offsets matter when the beam line does not pass through the intended section reference point. They can change coupling between axial, bending, and torsional behaviour and are especially important for idealized stiffeners, eccentric reinforcements, and members attached to shell surfaces.

The profile definition is intentionally independent from the beam element set. This allows a single profile to be reused by multiple beam sections, possibly with different materials or orientations. It also makes it possible to audit cross-section data separately from mesh topology. In larger decks, profile names should be descriptive enough to distinguish units and physical meaning; a profile named only P1 is easy to confuse once several member types exist.

Profile constants must be in the unit system of the model. If length is measured in millimetres, area is in square millimetres and second moments of area are in fourth powers of millimetres. Unit mistakes in beam profiles are common because the exponents differ between area, bending inertia, and torsion. A section can appear connected and stable while still being wrong by orders of magnitude. FEMaster requires $A > 0$, $I_y > 0$, $I_z > 0$, and $J > 0$. The bending inertia matrix must also be positive definite, which requires

$$I_y I_z - I_{yz}^2 > 0.$$

Example

```
*PROFILE, NAME=RECT_20_40
800.0, 106666.7, 26666.7, 90000.0, 0.0, 0.0, 0.0
```

9.4 *BEAMSECTION: beam sections

Syntax

```
*BEAMSECTION, ELSET=<elset>, MATERIAL=<material>, PROFILE=<profile>  
<nx>, <ny>, <nz>
```

A beam section assigns a material and profile to a set of beam elements. The element connectivity defines the beam line, while the beam section defines how the line behaves as a physical member. The section references a *PROFILE for cross-section constants and a material for constitutive behaviour. Together they define axial stiffness, bending stiffness, torsional stiffness, mass contribution where applicable, and section-force recovery.

The direction line following the command defines a local section direction. It is used to construct the beam's local coordinate system and therefore determines which profile inertia acts about which physical direction. This direction must not be parallel to the beam axis. If it is nearly parallel, the local axis construction becomes ill-conditioned and the resulting section orientation may be unstable or unexpected.

Beam orientation is a modelling decision, not a cosmetic detail. A rectangular or otherwise non-axisymmetric profile has different bending stiffness about its local section axes. Rotating the section by 90 degrees can produce a completely different structural response. For open, offset, or asymmetric profiles, the orientation also affects how section forces and moments should be interpreted.

Beam sections are most reliable when the beam element set contains members with a consistent intended orientation. If a single set contains beams running in many directions, one shared orientation vector may not describe all members well. In that case, split the beams into separate sets or define orientations carefully so that the local axes match the engineering model. After building a beam model, inspect local axes and section forces before treating beam stresses as design quantities.

Example

```
*BEAMSECTION, ELSET=FRAME_LONGERONS, MATERIAL=AL2024, PROFILE=RECT_20_40  
0.0, 0.0, 1.0
```

9.5 *TRUSSSECTION: truss sections

Syntax

```
*TRUSSSECTION, ELSET=<elset>, MATERIAL=<material>, AREA=<A>
```

A truss section assigns material and cross-sectional area to truss elements. Truss elements carry axial force only, so the section is correspondingly simple. The material provides the axial constitutive response, and **AREA** scales axial stiffness, mass contribution where density is available, and stress recovery.

The simplicity is useful, but it is also the main modelling constraint. A truss section does not define bending inertia, torsional stiffness, offsets, or rotational behaviour. If a physical member transfers moment, resists bending, or has important connection eccentricity, a truss section cannot represent that behaviour. In those cases a beam, shell, or solid idealization is more appropriate.

Area must be positive and chosen consistently with the model unit system and with the intended structural idealization. For a rod-like member, it may correspond to the actual cross-sectional area. For an idealized brace, link, or equivalent stiffness member, it may represent an effective area chosen to match axial stiffness. In either case, the interpretation should be clear because truss stresses are based directly on axial force divided by area.

Truss sections should be assigned to element sets that are intended to behave as pin-jointed axial members. The surrounding support and connection model must provide stability. A collection of truss elements can easily form a mechanism if the geometry or constraints do not prevent rigid-body or hinge-like motion. When debugging truss models, check global stability, reaction balance, and axial force signs before focusing on local stresses.

Example

```
*TRUSSSECTION, ELSET=BRACES, MATERIAL=STEEL, AREA=25.0
```

9.6 Choosing and checking sections

A good section assignment should answer three questions: which elements receive the section, what physical data are assigned to them, and how local directions are interpreted. If any of these are unclear, the model is difficult to review and easy to misuse. Use meaningful element-set names, material names, profile names, and orientation names so the input deck remains readable after the mesh grows.

For solid sections, check material coverage and orientation for directional materials. For shell sections, check thickness, normals, and material axes. For beam sections, check profile units and local section directions. For truss sections, check that the member is really intended to carry axial force only. These checks are often faster than solving the model and can catch the most expensive modelling mistakes before assembly begins.

Sections should be reviewed whenever the mesh changes. A remeshed part may keep the same visual geometry but lose element-set membership, split a region into new sets, or change local element orientation. A section definition that was correct for an old mesh can silently become incomplete after preprocessing. Treat section coverage as part of model validation, not as a one-time setup step.

10 Features

Features are additional model objects that are not defined through element connectivity. They add concentrated properties or auxiliary behaviour to the model.

10.1 Point Mass

Syntax

```
*POINTMASS, NSET=<node-set>  
<m>, <Ixx>, <Iyy>, <Izz>, <Kx>, <Ky>, <Kz>, <Krx>, <Kry>, <Krz>
```

***POINTMASS** assigns concentrated mass, rotational inertia, and optional spring stiffness to the nodes of a node set. It is useful for equipment masses, simplified component masses, sensors, local inertias, or concentrated replacement springs.

The data line contains four groups: scalar mass, three rotational inertias, three translational spring stiffnesses, and three rotational spring stiffnesses. Missing values are interpreted as zero.

Point masses can contribute to mass and inertia. They are therefore important for eigenfrequency analysis, transient analysis, inertial loads, and inertia relief. In a purely static loadcase without inertia relief, mass does not automatically mean weight; gravity or acceleration must be introduced through loads.

***INERTIALOAD** and ***INERTIARELIEF** use the option **CONSIDER_POINT_MASSES** to decide whether point-mass features are included in the mass and inertia balance.

11 Material Models

Material definitions provide density, thermal expansion, and elastic or hyperelastic constitutive behaviour. Sections reference materials by name; elements receive their actual stiffness and mass only after their section has been assigned. A material can therefore be defined once and reused by several shell, solid, beam, or truss sections.

11.1 Material context: *MATERIAL

Syntax

```
*MATERIAL, NAME=<material>
  *ELASTIC, TYPE=<model>
  ...
  *HYPERELASTIC, NEO HOOKE
  ...
  *DENSITY
  ...
  *THERMALEXPANSION
  ...
```

***MATERIAL** activates or creates a named material entry. The material name is supplied through **MATERIAL** or the alias **NAME**. Subcommands such as ***ELASTIC**, ***HYPERELASTIC**, ***DENSITY**, and ***THERMALEXPANSION** apply to the currently active material. ***ELASTIC** and ***HYPERELASTIC** are alternative constitutive-model assignments; a material definition normally uses one of them.

11.2 Density: *DENSITY

Syntax

```
*DENSITY
<rho>
```

***DENSITY** sets the nonnegative mass density ρ . Density is used when FEMaster builds mass matrices, lumped masses from continuum or structural elements, inertia loads, inertia relief mass properties, and dynamic equations. In a consistent unit system, ρ must match the length, force, and time units used in the deck. For example, if length is in meters and force in Newtons, density is normally given in kg/m³.

For a continuum element, the mass contribution is based on

$$M_e = \int_{\Omega_e} \rho N^T N d\Omega$$

or on the corresponding lumped form used by the selected analysis. For shells and beams, the section thickness or section area converts density into mass per area or mass per length.

11.3 Thermal expansion: *THERMALEXPANSION

Syntax

```
*THERMALEXPANSION
<alpha>
```

*THERMALEXPANSION sets the scalar thermal expansion coefficient α . Thermal loads use this coefficient together with the temperature difference $T - T_0$. For isotropic expansion, the thermal strain vector is

$$\varepsilon_{\text{th}} = \alpha(T - T_0) [1 \quad 1 \quad 1 \quad 0 \quad 0 \quad 0]^T.$$

If a material has no thermal expansion coefficient, a thermal load can still be present in the deck, but it has no meaningful elastic thermal strain contribution for that material.

11.4 Isotropic elasticity

Syntax

```
*ELASTIC, TYPE=ISOTROPIC
<E>, <nu>
```

Accepted type values are ISO and ISOTROPIC. The data line contains Young's modulus E and Poisson's ratio ν . The shear modulus is not entered independently; it is derived as

$$G = \frac{E}{2(1 + \nu)}.$$

FEMaster requires $E > 0$ and $-1 < \nu < 0.5$.

For three-dimensional stress states, FEMaster uses the isotropic stiffness matrix

$$D = \frac{E}{(1 + \nu)(1 - 2\nu)} \begin{bmatrix} 1 - \nu & \nu & \nu & 0 & 0 & 0 \\ \nu & 1 - \nu & \nu & 0 & 0 & 0 \\ \nu & \nu & 1 - \nu & 0 & 0 & 0 \\ 0 & 0 & 0 & (1 - 2\nu)/2 & 0 & 0 \\ 0 & 0 & 0 & 0 & (1 - 2\nu)/2 & 0 \\ 0 & 0 & 0 & 0 & 0 & (1 - 2\nu)/2 \end{bmatrix}.$$

For two-dimensional plane-stress-style element formulations, the in-plane matrix is

$$D_{2D} = \frac{E}{1 - \nu^2} \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & (1 - \nu)/2 \end{bmatrix}.$$

11.5 Generalised isotropic elasticity

Syntax

```
*ELASTIC, TYPE=GENISO
<E>, <nu>, <G>
```

Accepted type values are `GENERALISEDISOTROPIC`, `GENERALISED_ISOTROPIC`, and `GENISO`. This model keeps the normal isotropic stiffness terms from E and ν , but uses the supplied G as an independent shear stiffness. In 3D, the normal block is identical to the isotropic material, while the shear diagonal entries are

$$D_{44} = D_{55} = D_{66} = G.$$

FEMaster requires $E > 0$, $-1 < \nu < 0.5$, and $G > 0$. In the 2D matrix, the shear entry is also set directly:

$$D_{2D,33} = G.$$

This material is intentionally not a physically general isotropic law when $G \neq E/(2(1 + \nu))$. It breaks the isotropic relationship between Young's modulus, Poisson's ratio, and shear modulus. That can be useful deliberately, especially for shell studies where the user wants to tune, reduce, or effectively ignore shear stiffness while keeping isotropic membrane and bending terms. It should not be used as a physical isotropic material unless the supplied G satisfies the isotropic relation.

11.6 Neo-Hooke hyperelasticity

Syntax

```
*HYPERELASTIC, NEO HOOKE
<C10>, <D1>
```

`*HYPERELASTIC` assigns the compressible Abaqus-style Neo-Hooke model to the active material. The accepted model flags normalize to `NEOHOOKE`; `NEO_HOOKE` and `NEO-HOOKE` are equivalent spellings. The model is a standalone material subcommand: use `*HYPERELASTIC, NEO HOOKE`, not the former `*ELASTIC, TYPE=NEOHOOKE` syntax.

The strain-energy density is

$$W = C_{10}(\bar{I}_1 - 3) + \frac{(J - 1)^2}{D_1}.$$

C_{10} has stress units, D_1 has inverse-stress units, and both parameters must be positive. The corresponding small-strain shear and bulk moduli are

$$\mu = 2C_{10}, \quad K = \frac{2}{D_1}.$$

The material supplies linearized and Green–Lagrange response paths for axial, solid, and integrated-shell formulations. In nonlinear axial evaluation the lateral stretches are solved so that the lateral surfaces are traction-free. In nonlinear shell evaluation the thickness strain is iterated to enforce plane stress and the material tangent is condensed consistently.

Example

```
*MATERIAL, NAME=RUBBER
*HYPERELASTIC, NEO HOOKE
0.35, 0.02
```

11.7 Orthotropic elasticity

Syntax

```
*ELASTIC, TYPE=ORTHOTROPIC
<E1>, <E2>, <E3>, <G23>, <G13>, <G12>, <nu23>, <nu13>, <nu12>
```

Accepted type values are ORTHO and ORTHOTROPIC. Orthotropic elasticity defines three material axes with separate Young's moduli E_1, E_2, E_3 , shear moduli G_{23}, G_{13}, G_{12} , and Poisson ratios. It is appropriate for laminates, wood-like materials, unidirectional composites, and any material whose stiffness depends on direction.

The 3D compliance matrix is assembled as

$$S = \begin{bmatrix} 1/E_1 & -\nu_{21}/E_2 & -\nu_{31}/E_3 & 0 & 0 & 0 \\ -\nu_{12}/E_1 & 1/E_2 & -\nu_{32}/E_3 & 0 & 0 & 0 \\ -\nu_{13}/E_1 & -\nu_{23}/E_2 & 1/E_3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1/G_{23} & 0 & 0 \\ 0 & 0 & 0 & 0 & 1/G_{13} & 0 \\ 0 & 0 & 0 & 0 & 0 & 1/G_{12} \end{bmatrix}, \quad D = S^{-1}.$$

Reciprocal Poisson ratios are derived from modulus ratios. In particular, FEMaster derives

$$\nu_{31} = \nu_{13} \frac{E_3}{E_1}.$$

The input column is named NU13, but the material construction uses the reciprocal value needed by the compliance matrix. Orthotropic data should be checked carefully because inconsistent modulus and Poisson-ratio combinations can produce a non-positive or ill-conditioned stiffness matrix.

11.8 Prescribed ABD shell stiffness

Syntax

```
*SHELLSECTION, ELSET=<elset>, TYPE=ABD, THICKNESS=<t>, ORIENTATION=<orientation>, CSYSAXIS
=<1|2|3>
<abd11>, <abd12>, ..., <abd66>,
<s11>, <s12>, <s21>, <s22>
```

Prescribed ABD stiffness is a shell-section formulation, not an *ELASTIC material subcommand. It is documented in this chapter because it replaces a material law with already homogenized generalized section stiffnesses. The data belong to *SHELLSECTION with TYPE=ABD. The command reads 40 values. The first 36 values are a 6×6 ABD matrix in row-major order. The last 4 values are a 2×2 transverse shear matrix in row-major order. The values may be distributed over up to eight input lines.

The ABD matrix is a shell or laminate stiffness representation:

$$\begin{bmatrix} N \\ M \end{bmatrix} = \begin{bmatrix} A & B \\ B & D \end{bmatrix} \begin{bmatrix} \varepsilon_0 \\ \kappa \end{bmatrix}.$$

A is membrane stiffness, B is membrane-bending coupling, and D is bending stiffness. This model is appropriate when the section stiffness has already been homogenized outside FEMaster or when a laminate should be represented directly by engineering stiffness resultants rather than by individual plies.

The final four input values form the transverse shear stiffness matrix

$$S = \begin{bmatrix} s_{11} & s_{12} \\ s_{21} & s_{22} \end{bmatrix}, \quad \begin{bmatrix} Q_x \\ Q_y \end{bmatrix} = S \begin{bmatrix} \gamma_{xz} \\ \gamma_{yz} \end{bmatrix}.$$

Thus the complete generalized shell law consists of the 6×6 ABD matrix and the separate 2×2 matrix S . FEMaster uses the transverse-shear component order (xz, yz) , so the four values following the 36 ABD entries map exactly to $s_{11}, s_{12}, s_{21}, s_{22}$. For **TYPE=ABD**, these are already section stiffnesses: FEMaster does not multiply the supplied ABD or S matrix by shell thickness and does not apply an additional shear-correction factor.

Both matrices must contain finite values, be symmetric, and be positive definite. These conditions are checked when the ABD shell section is constructed. **MATERIAL** is optional for ABD shell sections because the generalized stiffness is supplied directly. When no orientation is assigned, the matrices act in the geometric shell basis. With **ORIENTATION**, FEMaster projects the selected **CSYSAXIS** into the shell plane, evaluates the prescribed matrices in that section basis, and rotates the element-facing resultants and tangent back into the geometric shell basis.

Example

```
*MATERIAL, NAME=STEEL
*ELASTIC, TYPE=ISOTROPIC
210000000000.0, 0.3
*DENSITY
7850.0
*THERMALEXPANSION
1.2e-5
```

12 Coordinate Systems and Orientations

***ORIENTATION** defines named local coordinate systems. These names can be referenced by sections, loads, supports, and connectors. The purpose is always the same: values that are convenient or physically meaningful in a local basis are transformed into the global basis used by the assembled finite-element equations.

Let the local basis vectors be the columns of a rotation matrix

$$R = \begin{bmatrix} e_1 & e_2 & e_3 \end{bmatrix}.$$

A local vector v_ℓ is transformed to global coordinates by

$$v_g = Rv_\ell.$$

For displacements, forces, rotations, and moments, the same directional transformation is used for the three-component translational block and the three-component rotational block.

12.1 Rectangular systems

Syntax

```
*ORIENTATION, NAME=<name>, TYPE=RECTANGULAR  
<x1>, <x2>, <x3>, <y1>, <y2>, <y3>, <z1>, <z2>, <z3>
```

RECTANGULAR describes a Cartesian local basis. The command accepts three practical input forms:

- 9 values: three direction vectors are supplied.
- 6 values: two direction vectors are supplied and the third is completed.
- 3 values: one primary direction is supplied and the remaining directions are completed.

The safest input is two or three non-collinear vectors. FEMaster normalizes and completes the basis, but it cannot infer a robust local frame from ambiguous geometry. If two supplied vectors are almost parallel, the cross product becomes small and the resulting local axes may be unstable. In formula form, a typical completion from two vectors a and b is

$$e_1 = \frac{a}{\|a\|}, \quad e_3 = \frac{e_1 \times b}{\|e_1 \times b\|}, \quad e_2 = e_3 \times e_1.$$

The exact input variant decides which vector is treated as primary, but the modelling implication is the same: provide directions that define a clear right-handed frame.

12.2 Cylindrical systems

Syntax

```
*ORIENTATION, NAME=<name>, TYPE=CYLINDRICAL
<origin-x>, <origin-y>, <origin-z>,
<axis-x>, <axis-y>, <axis-z>,
<reference-x>, <reference-y>, <reference-z>
```

CYLINDRICAL defines a coordinate system from an origin, an axis direction, and a reference direction. The axial unit vector is

$$e_z = \frac{a}{\|a\|}.$$

At a point x , the radial direction is obtained by projecting $x - o$ onto the plane normal to e_z :

$$r = (x - o) - ((x - o) \cdot e_z)e_z, \quad e_r = \frac{r}{\|r\|}.$$

The tangential direction follows from

$$e_\theta = e_z \times e_r.$$

Near the cylindrical axis, the radial direction is not well defined. The reference vector is used to establish a stable angular reference, but users should still avoid applying cylindrical local directions exactly on the axis unless the intended direction is unambiguous.

12.3 Where orientations are used

Command	Use of the orientation
*SUPPORT	Prescribed support components are interpreted in the local basis before they become constraint equations.
*CLOAD	Nodal force and moment components are entered locally and transformed to global generalized forces.
*DLOAD	Surface traction components are interpreted in the local basis.
*VLOAD	Body-force components are interpreted in the local basis.
*SOLIDSECTION	Optional material axes for solid elements.
*SHELLSECTION	Optional shell material, ABD, stress, and resultant axes; CSYSAXIS selects the projected coordinate-system axis.
*CONNECTOR	Local connector axes define which relative translations or rotations are constrained.

Near-collinear directions

Coordinate systems, beam sections, couplings, and connectors are sensitive to near-collinear direction definitions. A nearly zero cross product creates poorly defined axes and can produce unintuitive constraints or stiffness directions. Prefer clear, normalized, geometrically separated reference directions.

13 Fields

***FIELD** defines generic tabular data. Fields may be defined at root level or inside a loadcase block, depending on their intended use. The field domain determines how rows are associated with model entities.

13.1 Supported domains

For node, element, integration-point, and material-point fields the general form is

Syntax

```
*FIELD, NAME=<name>, TYPE=NODE|ELEMENT|ELEMENT_IP|ELEMENT_MP, COLS=<n>, FILL=ZERO|NAN
<row-id>, <v1>, <v2>, ...
```

Element-nodal fields use an element identifier and zero-based local node index instead of one flat row identifier:

Syntax

```
*FIELD, NAME=<name>, TYPE=ELEMENT_NODAL, COLS=<n>, FILL=ZERO|NAN
<element-id>, <local-node>, <v1>, <v2>, ...
```

COLS is required and must be greater than zero. FEMaster limits fields to 64 components. **FILL** defaults to **ZERO**; **NAN** initializes unspecified values with NaN. Individual values may be omitted; omitted components preserve the initialization value already stored in the field.

Domain	Meaning
NODE	One row per node ID. Used for temperature fields, initial velocities, or other prescribed nodal data.
ELEMENT	One row per element ID. Used for topology density and orientation fields.
ELEMENT_NODAL	One row per element-local node. Input rows are addressed by element ID and zero-based local node index. Used, for example, for prescribed shell director fields.
ELEMENT_IP	One row per enumerated element integration point. The alias IP is accepted.
ELEMENT_MP	One row per enumerated material point. The alias MP is accepted. This domain is also used by nonlinear material-state infrastructure.

Enumeration-dependent domains require the corresponding element topology and local enumeration to exist. FEMaster therefore creates these fields after topology construction during its field parsing stage.

Data lines may omit individual values; omitted values do not overwrite existing entries. The tokens **NAN**, **+NAN**, **-NAN**, **INF**, **+INF**, and **-INF** are accepted.

13.2 Typical references

Use	Domain/Cols	Referencing command
Temperature load	NODE, 1	*TLOAD with TEMPERATUREFIELD.
Initial velocity	NODE, 6	*INITIALVELOCITY.
Topology density	ELEMENT, 1	*TOPODENSITY with FIELD=<name>.
Topology orientation	ELEMENT, 3	*TOPOORIENT with FIELD=<name>.
Shell directors	ELEMENT_NODAL, 3	*NORMAL with FIELD=<name>.

13.3 Shell normal selection: *NORMAL

Syntax

```
*NORMAL, FIELD=<element-nodal-field>
```

***NORMAL** selects an existing three-component **ELEMENT_NODAL** field as the shell-normal field. It is valid at model root scope. The command does not allocate the field and does not create, normalize, or average the supplied vectors; geometric completion remains part of shell-normal construction.

A typical partially prescribed field uses **FILL=NAN** so unspecified element-local directions remain distinguishable from explicitly supplied vectors:

Example

```
*FIELD, NAME=DIRECTORS, TYPE=ELEMENT_NODAL, COLS=3, FILL=NAN
10, 0, 0.0, 0.0, 1.0
10, 1, 0.0, 0.0, 1.0
*NORMAL, FIELD=DIRECTORS
```

The ***FIELD** definition must appear before the referencing ***NORMAL** command. Further shell-specific interpretation is described in Section 6.29.

14 Loads, Supports, and Amplitudes

Loads and supports are not activated directly by their definition command. They are assigned to named collectors first. A support collector is a named list of support entries. A load collector is a named list of load entries. A loadcase later selects one or more of these collectors with ***SUPPORTS** and ***LOADS**.

This collector model is important for reusable decks. For example, one support collector may contain all fixture supports, another may contain symmetry supports, and a third may contain temporary stabilization supports. A static loadcase can activate only the fixture collector, while a modal loadcase can activate fixture and symmetry collectors. Loads work the same way: each ***CLOAD**, ***DLOAD**, ***PLOAD**, ***VLOAD**, ***TLOAD**, or ***INERTIALLOAD** entry declares which **LOAD_COLLECTOR** it belongs to; the loadcase decides which collectors are assembled.

14.1 Support collectors

Syntax

```
*SUPPORT, SUPPORT_COLLECTOR=<name>, ORIENTATION=<orientation>  
<node-set-or-id>, <ux>, <uy>, <uz>, <rx>, <ry>, <rz>
```

***SUPPORT** adds one support entry to the collector named by **SUPPORT_COLLECTOR**. The collector is created implicitly when the first support uses its name. The data line targets either a node set or a single node id and then gives prescribed values for the six nodal components **Ux**, **Uy**, **Uz**, **Rx**, **Ry**, **Rz**. In most models the prescribed values are zero, but nonzero entries can be used for enforced displacement or rotation.

If **ORIENTATION** is omitted, the six values are interpreted in the global coordinate system. If an orientation is given, the translational and rotational components are interpreted in that local coordinate system and transformed into the global system before the constraint equations are built. This is the usual way to describe supports normal to a cylindrical surface, sliding conditions in a skew system, or fixtures that are easier to express in local axes than in global axes.

The support value is a kinematic prescription. In matrix form, a scalar support can be read as

$$e_i^T u = \bar{u}_i,$$

where e_i selects the constrained degree of freedom and $\bar{u}_i$ is the prescribed value from the support line. Multiple support entries in the same collector are assembled into one set of equations.

In **LINEARSTATIC**, the full prescribed value is applied. In **NONLINEARSTATIC** load control, a nonzero prescribed value is ramped proportionally with the load factor: it is zero at $\lambda = 0$ and reaches the entered value at $\lambda = 1$. Arc-length control currently supports only homogeneous constraints and therefore rejects nonzero prescribed displacements or rotations.

14.2 Concentrated loads

Syntax

```
*CLOAD, LOAD_COLLECTOR=<name>, ORIENTATION=<orientation>, AMPLITUDE=<amp>
<node-set-or-id>, <Fx>, <Fy>, <Fz>, <Mx>, <My>, <Mz>
```

***CLOAD** adds nodal forces and moments to the collector named by **LOAD_COLLECTOR**. The target may be a node set or a single node id. The six numeric values correspond to nodal generalized force components conjugate to U_x , U_y , U_z , R_x , R_y , R_z . Translational components are forces; rotational components are moments.

If an orientation is specified, the force and moment components are interpreted in the local system and transformed to global components during assembly. If an amplitude is specified, the load entry is multiplied by the amplitude value in transient analyses:

$$f_j(t) = a(t)f_j.$$

In static analyses, the load is used as its defined value unless the loadcase explicitly evaluates time-dependent data.

14.3 Surface traction and pressure

Syntax

```
*DLOAD, LOAD_COLLECTOR=<name>, ORIENTATION=<orientation>, AMPLITUDE=<amp>
<surface-set-or-id>, <Fx>, <Fy>, <Fz>
```

Syntax

```
*PLOAD, LOAD_COLLECTOR=<name>, AMPLITUDE=<amp>
<surface-set-or-id>, <pressure>
```

***DLOAD** defines a vector traction on surfaces. Conceptually, the load contribution is

$$f_e = \int_{\Gamma_e} N^T t \, d\Gamma,$$

where t is the traction vector and N contains the element shape functions on the loaded surface. ***PLOAD** defines a scalar pressure. A positive pressure acts into the selected surface, opposite to its normal:

$$t = -pn.$$

The selected surface side and its normal direction therefore determine the pressure sign convention. For shell surfaces, **SPOS** and **SNeg** select opposite normals on the same geometric midsurface.

Use ***DLOAD** when the traction direction is known as a vector. Use ***PLOAD** when the load should follow the surface normal. Both commands add entries to a load collector and are activated only when that collector is referenced by a loadcase.

14.4 Volume loads

Syntax

```
*VLOAD, LOAD_COLLECTOR=<name>, ORIENTATION=<orientation>, AMPLITUDE=<amp>
<element-set-or-id>, <Fx>, <Fy>, <Fz>
```

***VLOAD** defines a body-force density over an element region. The assembled contribution follows

$$f_e = \int_{\Omega_e} N^T b d\Omega,$$

where b is the supplied volume load vector. This is appropriate for load densities that are already expressed per unit volume. If a gravitational acceleration should be converted into force, the material density and the intended unit convention must be reflected in the supplied values or in the load formulation used for the model.

14.5 Thermal loads

Syntax

```
*TLOAD, LOAD_COLLECTOR=<name>, TEMPERATUREFIELD=<field>, REFERENCETEMPERATURE=<T0>
```

***TLOAD** adds thermal strain loading to a load collector. The command references a nodal temperature field and a reference temperature. For a material with thermal expansion coefficient α , the thermal strain is based on

$$\varepsilon_{th} = \alpha(T - T_0) [1 \ 1 \ 1 \ 0 \ 0 \ 0]^T$$

for isotropic expansion. The corresponding equivalent mechanical load is assembled from the element constitutive law and the thermal strain. Thermal loads therefore require both a temperature field and material definitions with thermal expansion data.

14.6 Inertial loads

Syntax

```
*INERTIALOAD, LOAD_COLLECTOR=<name>, CONSIDER_POINT_MASSES=0|1
<elset>, <cx>, <cy>, <cz>, <a0x>, <a0y>, <a0z>, <wx>, <wy>, <wz>, <alphax>, <alphay>, <alphaz>
```

***INERTIALOAD** creates equivalent nodal loads from a prescribed rigid-body acceleration field. The first vector c is the reference point. The second vector a_0 is the translational acceleration of that point. The third vector ω is angular velocity and the fourth vector α is angular acceleration. At a material point x , the rigid-body acceleration is

$$a(x) = a_0 + \alpha \times (x - c) + \omega \times (\omega \times (x - c)).$$

The equivalent inertial load is assembled over the selected element set. **CONSIDER_POINT_MASSES** controls whether point-mass features are included as additional concentrated inertia contributions.

14.7 Amplitudes

Syntax

```
*AMPLITUDE, NAME=<name>, TYPE=LINEAR|STEP|NEAREST  
<time>, <value>
```

An amplitude is a scalar time function. Load entries that reference an amplitude are scaled by its value during transient analysis. **LINEAR** interpolates linearly between data points. **STEP** keeps the previous value until the next point is reached. **NEAREST** uses the value of the closest tabulated time. Amplitudes are useful when the spatial load pattern is fixed but the intensity changes over time.

15 Constraints

Constraints define kinematic equations between degrees of freedom. They are different from supports in modelling intent: a support prescribes motion of selected degrees of freedom, while a constraint relates several nodes, surfaces, or reference points to each other. FEMaster assembles supports and constraints into one algebraic equation set before the active loadcase is solved.

Most constraint equations can be written as

$$Cu = d.$$

u is the vector of active nodal degrees of freedom, C is the constraint matrix, and d is the right-hand side. Homogeneous constraints have $d = 0$. The constraint definitions in this chapter describe which equations are generated. The numerical handling of those equations, for example Nullspace reduction or Lagrange multipliers, is selected by `*CONSTRAINTMETHOD` and described in Chapter 16.

15.1 Rigid-body motion suppression: *RBM

Syntax

```
*RBM, ELSET=<elset>
```

`*RBM` generates up to six homogeneous equations for the nodes used by the selected element set. `ELSET` also accepts the alias `SET`; if omitted, the default is `EALL`. The command is meant to suppress free rigid-body modes without fixing an arbitrary node. This is especially useful for stabilization, diagnostic models, and inertia-relief workflows.

FEMaster first collects all nodes that belong to the selected structural elements. It computes the center of gravity x_0 of the selected element region and uses the relative position

$$r_i = x_i - x_0$$

for every participating node i . With n participating nodes and $w = 1/n$, the three translational equations are

$$\sum_i w u_{ix} = 0, \quad \sum_i w u_{iy} = 0, \quad \sum_i w u_{iz} = 0.$$

The three rotational equations suppress the average rigid rotation measured by the first moment of the displacement field:

$$\sum_i w (r_i \times u_i)_x = 0, \quad \sum_i w (r_i \times u_i)_y = 0, \quad \sum_i w (r_i \times u_i)_z = 0.$$

Expanded component-wise, these are

$$\sum_i w (r_{iy} u_{iz} - r_{iz} u_{iy}) = 0,$$

$$\sum_i w(r_{iz}u_{ix} - r_{ix}u_{iz}) = 0,$$

$$\sum_i w(r_{ix}u_{iy} - r_{iy}u_{ix}) = 0.$$

Only active translational degrees of freedom are used. If a required component does not exist for a node, it is skipped for that equation.

15.2 Coupling: *COUPLING

Syntax

```
*COUPLING, MASTER=<master-nset>, TYPE=KINEMATIC|STRUCTURAL, SLAVE=<slave-nset>
<Ux>, <Uy>, <Uz>, <Rx>, <Ry>, <Rz>
```

Syntax

```
*COUPLING, MASTER=<master-nset>, TYPE=KINEMATIC|STRUCTURAL, SFSET=<surface-set>
<Ux>, <Uy>, <Uz>, <Rx>, <Ry>, <Rz>
```

***COUPLING** connects one master node set to either a slave node set or a slave surface set. Exactly one of **SLAVE** and **SFSET** must be given. In practical models the master set should normally contain one reference node. Larger master sets can over-constrain the model because the generated equations are based on a single master node.

The data line contains six flags for **Ux**, **Uy**, **Uz**, **Rx**, **Ry**, **Rz**. Values greater than zero activate the corresponding coupling component. **KINEMATIC** generates constraint equations. **STRUCTURAL** does not generate kinematic equations; it distributes generalized loads from the master node to slave nodes in a work-equivalent way.

15.2.1 Kinematic coupling equations

For a slave node s , master node m , and offset

$$r = x_s - x_m,$$

the translational kinematic relation is the small-rotation rigid-body relation

$$u_s = u_m + \theta_m \times r.$$

FEMaster writes this component-wise as homogeneous equations. For example,

$$u_{sx} - u_{mx} - r_z\theta_{my} + r_y\theta_{mz} = 0,$$

$$u_{sy} - u_{my} - r_x\theta_{mz} + r_z\theta_{mx} = 0,$$

$$u_{sz} - u_{mz} - r_y\theta_{mx} + r_x\theta_{my} = 0.$$

Rotational coupling is direct:

$$\theta_{sx} - \theta_{mx} = 0, \quad \theta_{sy} - \theta_{my} = 0, \quad \theta_{sz} - \theta_{mz} = 0.$$

Only the components enabled by the six flags are generated, and only if the involved degrees of freedom exist.

15.2.2 Structural coupling load distribution

For **STRUCTURAL**, a generalized master load

$$b = [F_x \quad F_y \quad F_z \quad M_x \quad M_y \quad M_z]^T$$

is distributed to slave translational forces. FEMaster builds the equilibrium operator

$$A = \begin{bmatrix} I & I & \dots & I \\ [r_1]_{\times} & [r_2]_{\times} & \dots & [r_n]_{\times} \end{bmatrix},$$

where $[r_i]_{\times} f_i = r_i \times f_i$. The distributed force vector is computed in a least-norm, work-equivalent form from

$$G\lambda = b, \quad G = AA^T, \quad f = A^T\lambda.$$

The resulting slave forces reproduce the master resultant force and moment as closely as the slave geometry permits. The master load is then consumed because it has been represented by slave forces.

15.3 Tie: *TIE

Syntax

```
*TIE, MASTER=<master>, SLAVE=<slave>, ADJUST=NO|YES, DISTANCE=<search-distance>
```

***TIE** binds slave nodes to a master surface set or master line set. The slave region may be a node set or a surface set. For each slave node, FEMaster searches the closest master geometry within **DISTANCE**. If no candidate is found, no equation is generated for that slave node. If **ADJUST=YES**, the slave node coordinate is moved to the projected master position before the equations are assembled.

Let s be a slave node projected onto a master surface or line at local coordinates ξ . Let $N_a(\xi)$ be the shape function of master node a at that projected point. For every active degree of freedom, the compatibility equation is

$$u_s - \sum_a N_a(\xi) u_a = 0.$$

This ties the slave degree of freedom to the interpolated master motion. For surface masters the interpolation uses the surface shape functions; for line masters it uses the line shape functions. The same equation form is used for translational and rotational degrees of freedom when those degrees of freedom exist on both slave and all master nodes.

15.4 Connector: *CONNECTOR

Syntax

```
*CONNECTOR, TYPE=<type>, NSET1=<nset1>, NSET2=<nset2>, COORDINATESYSTEM=<orientation>
```

***CONNECTOR** creates idealized relative-motion constraints between two node sets in a named coordinate system. Supported types are **BEAM**, **HINGE**, **CYLINDRICAL**, **TRANSLATOR**, **JOIN**, and **JOINRX**. The connector type maps to a six-component mask that decides which local translations and rotations are constrained.

For each constrained local component e_α , FEMaster transforms the local direction into the global system:

$$g_\alpha = R e_\alpha.$$

For a constrained translation, the generated equation is

$$g_{\alpha}^T(u_1 - u_2) = 0.$$

For a constrained rotation, the equation is

$$g_{\alpha}^T(\theta_1 - \theta_2) = 0.$$

The connector therefore constrains relative motion along selected local axes rather than simply tying global components. This is why the referenced coordinate system is part of the connector definition and should be chosen carefully.

15.5 Constraint summary

Syntax

```
*LOADCASE, TYPE=LINEARSTATIC
*CONSTRAINTSUMMARY
```

***CONSTRAINTSUMMARY** requests a diagnostic listing of the internally generated constraint equations for the active loadcase. The output is grouped by constraint origin, for example supports, RBM equations, couplings, ties, and connectors. This is useful when a model is unexpectedly singular, over-constrained, or when a surface tie did not find as many slave projections as intended. It is a diagnostic report of generated equations, not a modelling command that changes the equations.

Master sets

Many coupling and connector workflows are intended for one reference node in the master set. Larger master sets may over-constrain the model or produce a kinematic interpretation that is different from the intended reference-point behaviour.

16 Analysis and Solver Controls

This chapter describes commands that are used inside loadcases to activate collectors, choose the linear solver, select the constraint backend, and configure analysis-specific numerical options. Constraint definitions themselves are described in Chapter 15; the commands here decide how those definitions are used in a particular analysis.

16.1 Activating support and load collectors

Syntax

```
*SUPPORTS
<support-collector-1>, <support-collector-2>, ...

*LOADS
<load-collector-1>, <load-collector-2>, ...
```

***SUPPORTS** activates named support collectors that were populated by ***SUPPORT**. ***LOADS** activates named load collectors that were populated by ***CLOAD**, ***DLOAD**, ***PLOAD**, ***VLOAD**, ***TLOAD**, or ***INERTIALOAD**. The loadcase does not activate every support or load in the model automatically. It assembles only the collectors listed in these commands.

Mathematically, active support collectors contribute equations to the global constraint set $Cu = d$. Active load collectors contribute to the right-hand side vector f . In a transient loadcase, time-dependent amplitudes are evaluated during time integration, so the active load vector is a function $f(t)$.

16.2 Solver

Syntax

```
*SOLVER, DEVICE=CPU|GPU, METHOD=DIRECT|INDIRECT
```

DEVICE selects the execution device. **CPU** is the standard path. **GPU** is intended for configurations where the corresponding solver routines and data structures are available. **METHOD** selects the strategy for linear systems.

DIRECT solves a system by factorization. For a static reduced system

$$Aq = b,$$

the matrix A is factorized and the solution is obtained by triangular solves or an equivalent direct procedure. Direct solvers are robust and predictable for small and medium models. Their main limitation is memory growth, especially for large three-dimensional meshes.

INDIRECT uses an iterative solution path. Instead of building a full direct factorization, an approximate solution is improved until the residual is sufficiently small. Iterative methods can be attractive for large systems because they avoid some of the memory cost of direct factorization. They are more sensitive to conditioning, scaling, constraint handling, and matrix definiteness. In FEMaster the indirect path should be used with NULLSPACE constraints, because the reduced system keeps the standard structural form. It is not intended for the saddle-point system produced by Lagrange multipliers.

Constraint	Backend	DIRECT	INDIRECT
NULLSPACE	CPU MKL	Yes	Yes
NULLSPACE	CPU Eigen	Yes	Yes
NULLSPACE	GPU	Yes	Yes
NULLSPACE	GPU cuDSS	Yes	Yes
LAGRANGE	CPU MKL	Yes	No
LAGRANGE	CPU Eigen	Limited	No
LAGRANGE	GPU	No	No
LAGRANGE	GPU cuDSS	Yes	No
ELIMINATION	CPU MKL	Yes	Yes
ELIMINATION	CPU Eigen	Yes	Yes
ELIMINATION	GPU	Yes	Yes
ELIMINATION	GPU cuDSS	Yes	Yes

Table 16.1: Supported static solver and constraint combinations.

16.3 Constraint method

Syntax

```
*CONSTRAINTMETHOD, TYPE=NULLSPACE|LAGRANGE|ELIMINATION
```

The active supports and constraints define

$$Cu = d.$$

*CONSTRAINTMETHOD selects how this equation set is introduced into the analysis equations. The command is supported inside LINEARSTATIC, LINEARSTATICTOPO, and NONLINEARSTATIC loadcases. The nonlinear static loadcase currently accepts only NULLSPACE; linear static loadcases may use NULLSPACE, LAGRANGE, or ELIMINATION.

16.3.1 Nullspace method

NULLSPACE parameterizes all admissible displacements as

$$u = u_p + Tq.$$

u_p is a particular displacement vector satisfying the inhomogeneous part of the constraints. T is a basis for the nullspace of C , so

$$CT = 0, \quad Cu_p = d.$$

q contains only independent degrees of freedom. A static system

$$Ku = f$$

is transformed into

$$T^T K T q = T^T (f - K u_p).$$

The matrix

$$A = T^T K T$$

is the reduced stiffness matrix. This approach removes constrained degrees of freedom from the solve and is well suited for direct and iterative solvers. It also avoids adding Lagrange multipliers to the unknown vector.

For eigenvalue and transient problems the same projection is applied to the operators:

$$K_r = T^T K T, \quad M_r = T^T M T, \quad C_r = T^T C T.$$

Recovered displacements are mapped back with $u = u_p + Tq$, while velocities and accelerations use the homogeneous part $T\dot{q}$ and $T\ddot{q}$.

16.3.2 Lagrange method

LAGRANGE keeps the original displacement vector and adds one multiplier per constraint equation. The static system becomes

$$\begin{bmatrix} K & C^T \\ C & -\epsilon I \end{bmatrix} \begin{bmatrix} u \\ \lambda \end{bmatrix} = \begin{bmatrix} f \\ d \end{bmatrix}.$$

λ contains the Lagrange multipliers, which can be interpreted as generalized constraint reactions. ϵ is a possible small regularization term when configured by the solver path; conceptually, the pure Lagrange system has $\epsilon = 0$.

This formulation is direct and preserves the original displacement variables, but it creates a saddle-point problem. Such systems are indefinite and more demanding numerically. In FEMaster, **LAGRANGE** requires the direct linear solver for linear static analysis; see Table 16.1 for supported backend combinations. Use **NULLSPACE** when an indirect solver is desired.

16.3.3 Elimination method

ELIMINATION directly selects dependent degrees of freedom from the independent constraint rows and expresses them through the remaining master degrees of freedom. The resulting affine map has the same recovered-displacement form

$$u = u_p + Tq$$

as the nullspace method, but T is built by explicit substitution rather than by a numerical nullspace factorization. Each selected slave degree of freedom is written as

$$u_s = u_{p,s} + \sum_j X_{sj} q_j.$$

FEMaster applies substitutions row by row, avoids cyclic dependencies, and treats redundant rows consistently with the reported constraint build diagnostics.

Because the final solve is again performed in the reduced coordinates q , **ELIMINATION** keeps the structural system form and is compatible with both direct and indirect static solvers. It is best suited to equation sets where a clear dependent degree of freedom can be selected from each independent row, for example support equations and many direct coupling equations. For dense or strongly coupled equation systems, **NULLSPACE** may be the more robust choice.

16.4 Inertia relief

Syntax

```
*INERTIARELIEF, CONSIDER_POINT_MASSES=0|1
```

Inertia relief is used for static analysis of free or nearly free structures. A free body under an unbalanced external load has no static equilibrium in an inertial frame; it accelerates as a rigid body. Inertia relief computes an equivalent rigid-body acceleration and adds inertia loads that balance the external resultant force and moment. The remaining static solve describes elastic deformation relative to that accelerating rigid-body motion.

This is not a substitute for missing real supports. If the physical model has bearings, bolts, fixtures, or symmetry conditions, they should be modelled as supports or constraints. Inertia relief is appropriate when the analysed body is intentionally free, for example an aircraft component under manoeuvre load or a structure in a load case where fixed ground supports would be artificial.

CONSIDER_POINT_MASSES controls whether point-mass features are included in the mass properties used by inertia relief. If point masses represent relevant equipment or lumped mass, set this option to 1. If they are purely numerical or should not affect the inertial balance, leave them out.

16.5 Load rebalancing

Syntax

```
*REBALANCELOADS
```

Load rebalancing modifies the external load balance so that the resultant force and moment vanish. It is a numerical balancing tool, not a physical rigid-body acceleration model. It is useful when a load is intended to be self-equilibrated but small residual resultants appear because of discretization, surface selection, rounding, or simplified load introduction.

The distinction from inertia relief is important. Inertia relief says: the structure is free, the external load would accelerate it, and inertia loads should represent that acceleration. Load rebalancing says: the load should be balanced, and the residual imbalance should be corrected. For real free-body operating loads, inertia relief is usually the more physical interpretation. For small unintended imbalance in an otherwise self-equilibrated load, rebalancing is often the simpler diagnostic option.

16.6 Matrix and diagnostic requests

Syntax

```
*REQUESTSTIFFNESS, FILE=<base-file>  
*REQUESTSTGEOM, FILE=<base-file>  
*CONSTRAINTSUMMARY
```

***REQUESTSTIFFNESS** requests stiffness matrix output for diagnostics or external comparison. ***REQUESTSTGEOM** requests geometric stiffness output in buckling analysis. ***CONSTRAINTSUMMARY** prints the internally generated constraint equations grouped by origin, such as supports, RBM equations, couplings, ties, and connectors.

These commands should be treated as diagnostic output controls. They do not change the physical model. They are helpful when validating assemblies, comparing reduced matrices, debugging constraint generation, or checking why a loadcase is singular or over-constrained.

16.7 Eigenvalue parameters

Syntax

***NUMEIGENVALUES**

<count>

***SIGMA**

<shift>

***NUMEIGENVALUES** sets how many eigenpairs are requested for eigenfrequency or buckling analysis. The value must be positive.

***SIGMA** is used in linear buckling. The reduced buckling problem is assembled as

$$A\phi = \lambda(-B)\phi,$$

where

$$A = T^T K T, \quad B = T^T K_G T.$$

The reported buckling factors are sorted ascending after the eigensolve. The shift σ guides the buckling eigenvalue search and is the smallest buckling factor region that should be targeted for output. It must be chosen carefully: a poor shift can make the eigensolver converge to an unintended part of the spectrum or miss the factors of engineering interest.

If ***SIGMA** is set to 0, FEMaster generates a shift automatically. It first tries a Rayleigh estimate from the pre-buckling displacement q_{pre} :

$$\lambda_{\text{raw}} = \frac{q_{\text{pre}}^T A q_{\text{pre}}}{q_{\text{pre}}^T (-B) q_{\text{pre}}}.$$

If that estimate is finite and positive, the generated shift is

$$\sigma = \frac{\lambda_{\text{raw}}}{10^6}.$$

If the Rayleigh estimate is not usable, FEMaster falls back to a diagonal ratio estimate

$$\lambda_i = \frac{A_{ii}}{(-B)_{ii}}$$

for positive finite diagonal entries and uses a conservative lower quartile of those values. If no valid estimate exists, the fallback shift is 10^{-12} .

16.8 Nonlinear static controls

Syntax

```
*NONLINEAR, CONTROL=LOAD|ARC_LENGTH,
  INCREMENTS=<n>, MAX_INCREMENTS=<n>,
  INITIAL_INCREMENT=<value>, MINIMUM_INCREMENT=<value>,
  MAXIMUM_INCREMENT=<value>, ARC_LENGTH_PSI=<value>,
  ADAPTIVE=ON|OFF, GROWTH_FACTOR=<value>,
  CUTBACK_FACTOR=<value>, FAST_ITERATIONS=<n>,
  SLOW_ITERATIONS=<n>, MAXIMUM_CUTBACKS=<n>,
  MAXITER=<n>, TOL=<value>,
  REGULARIZE_ZERO_ROWS=ON|OFF,
  REGULARIZATION_ALPHA=<value>
```

***NONLINEAR** configures a **NONLINEARSTATIC** loadcase. It combines the path-control method, the increment controller, the Newton iteration limit, and optional regularization of weak tangent rows. All arguments are optional; omitted arguments use the defaults in Table 16.2.

Table 16.2: Options of ***NONLINEAR**.

Option	Default	Meaning
CONTROL	LOAD	Select LOAD control or ARC_LENGTH path following.
INCREMENTS	—	Legacy shorthand that sets INITIAL_INCREMENT to $1/n$. It does not prescribe the number of accepted increments. An explicitly supplied INITIAL_INCREMENT takes precedence.
MAX_INCREMENTS	100	Maximum number of accepted increments.
INITIAL_INCREMENT	0.1	Initial step scale. Under load control this is $\Delta\lambda$; under arc-length control it scales the initial reference radius.
MINIMUM_INCREMENT	10^{-4}	Smallest step scale permitted after cutback.
MAXIMUM_INCREMENT	1.0	Largest step scale permitted after growth.
ARC_LENGTH_PSI	1.0	Nonnegative weight of the load-factor term in the arc-length constraint. Ignored by load control.
ADAPTIVE	ON	Enable step growth and cutback. With OFF , every attempt uses INITIAL_INCREMENT ; a rejected increment terminates the loadcase.
GROWTH_FACTOR	1.5	Multiplier applied after fast convergence.
CUTBACK_FACTOR	0.5	Multiplier applied after slow convergence or a rejected attempt; it must lie strictly between 0 and 1.
FAST_ITERATIONS	6	An accepted increment using no more than this number of Newton iterations is enlarged.
SLOW_ITERATIONS	10	An accepted increment using at least this number of Newton iterations is reduced.
MAXIMUM_CUTBACKS	20	Maximum consecutive cutbacks or final-step adjustments for one increment.
MAXITER	20	Maximum Newton iterations per attempted increment.
TOL	10^{-8}	Relative equilibrium-residual tolerance; arc-length control also applies it to the normalized path constraint.
REGULARIZE_ZERO_ROWS	ON	Add diagonal stiffness to tangent rows whose row norm is below the configured threshold.
REGULARIZATION_ALPHA	10^{-4}	Scale used to derive the minimum row stiffness from the mean nonzero tangent diagonal, or from the mean nonzero entry if no usable diagonal exists.

The increment bounds must satisfy

$$0 < \Delta s_{\min} \leq \Delta s_{\text{initial}} \leq \Delta s_{\max}.$$

All count limits must be positive. This applies to **MAX_INCREMENTS**, **MAXITER**, **FAST_ITERATIONS**, and **MAXIMUM_CUTBACKS**. In addition, **SLOW_ITERATIONS** must not be smaller than **FAST_ITERATIONS**.

16.8.1 Load control

With **CONTROL=LOAD**, FEMaster prescribes the load factor at each attempt:

$$f_{\text{ext}} = \lambda f_{\text{ref}}, \quad \lambda_{i+1} = \min(\lambda_i + \Delta\lambda, 1).$$

The Newton iterations enforce equilibrium at the prescribed λ_{i+1} . The last step is shortened so that the analysis ends at $\lambda = 1$, provided the remaining step does not fall below **MINIMUM_INCREMENT**. Load control

is appropriate while the equilibrium path remains representable as a single-valued function of the load factor. It generally cannot pass a limit point at which increasing displacement requires a decreasing load factor.

After two accepted states are available, the initial reduced displacement for a new attempt is predicted by linear secant extrapolation:

$$q_{i+1}^{(0)} = q_i + \frac{\lambda_{i+1} - \lambda_i}{\lambda_i - \lambda_{i-1}}(q_i - q_{i-1}).$$

The load factor itself is not predicted; it remains the prescribed target λ_{i+1} . The first increment starts from the accepted initial state. Rejected attempts restore the last accepted state, leave the secant history unchanged, and form a new predictor for the reduced target increment after cutback.

For inhomogeneous constraints, load control recovers the full displacement as

$$u(\lambda) = \lambda u_p + Tq.$$

Prescribed support values therefore ramp proportionally from zero to their entered value and are satisfied exactly at every target load factor.

16.8.2 Arc-length control

With `CONTROL=ARC_LENGTH`, displacement and load factor are corrected together. For one increment FEMaster enforces

$$\|\Delta q\|^2 + \psi^2 s_f^2 (\Delta \lambda)^2 = r^2,$$

where Δq is the reduced-coordinate increment, $\Delta \lambda$ is the load-factor increment, $s_f = \|\Delta q_f\|$ is a fixed load scale obtained from the first tangent predictor, and r is the current arc radius. The configured increment is a normalized radius scale: the first predictor establishes a fixed reference radius and subsequent growth or cutback multiplies that reference.

`ARC_LENGTH_PSI` is dimensionless. $\psi = 1$ gives the default scaled weighting. $\psi = 0$ removes the load-factor term and produces a cylindrical constraint. Increasing ψ restricts load-factor changes more strongly for a fixed radius. Arc-length control selects the continuation direction from the previous accepted path increment and can therefore follow branches through limit points where λ temporarily decreases. The loadcase completes when the continued path reaches $\lambda = 1$; the final radius is adjusted when necessary to hit that value within `TOL`.

16.8.3 Adaptation and convergence

After an accepted increment, adaptive control multiplies the next step by `GROWTH_FACTOR` when the solve used at most `FAST_ITERATIONS`, or by `CUTBACK_FACTOR` when it used at least `SLOW_ITERATIONS`. The result is clamped to the configured minimum and maximum. A rejected attempt restores the last accepted state and applies a cutback. Exceeding the minimum step, the cutback limit, or the maximum accepted-increment count terminates the loadcase.

For equilibrium, FEMaster forms the reduced residual

$$r = T^T(\lambda f_{\text{ref}} - f_{\text{int}})$$

and compares its RMS value with the larger of the reduced external-force RMS, reduced internal-force RMS, and 1. Under arc-length control, the convergence measure is the larger of this relative equilibrium residual and the normalized arc-constraint error. The displacement-correction norm printed in the solver log is diagnostic; equilibrium convergence is controlled by `TOL`.

Current restrictions

`NONLINEARSTATIC` currently requires `METHOD=DIRECT` and `TYPE=NULLSPACE`. Arc-length control additionally requires homogeneous constraints and rejects nonzero prescribed displacements or rotations. Zero-row regularization changes the tangent and should be enabled only when its numerical stabilization is intended. It is not a physical stiffness contribution.

16.9 Transient-specific controls

Syntax

```
*TIME
<t_start>, <t_end>, <dt>
```

***TIME** defines the time interval and fixed time step for **LINEARTRANSIENT**. It accepts either three values ($t_{\text{start}}, t_{\text{end}}, \Delta t$) or two values ($t_{\text{end}}, \Delta t$), in which case the start time is 0. The number of integration steps is based on the interval length and Δt . A smaller time step gives better temporal resolution but increases the number of linear solves and result states.

Syntax

```
*NEWMARK
<beta>, <gamma>
```

***NEWMARK** sets the Newmark integration parameters β_N and γ_N . The transient equation is

$$M\ddot{u} + C\dot{u} + Ku = f(t).$$

In reduced coordinates FEMaster solves the projected form

$$M_r\ddot{q} + C_r\dot{q} + K_rq = r(t).$$

The common values $\beta_N = 0.25$ and $\gamma_N = 0.5$ correspond to the average-acceleration method for linear systems. Other values should be chosen deliberately because they change numerical damping, stability, and accuracy.

Syntax

```
*DAMPING, TYPE=RAYLEIGH
<alpha>, <beta>
```

***DAMPING** currently supports Rayleigh damping:

$$C = \alpha M + \beta K.$$

α controls mass-proportional damping and has stronger influence at low frequencies. β controls stiffness-proportional damping and has stronger influence at high frequencies. In reduced form FEMaster builds

$$C_r = \alpha M_r + \beta K_r.$$

Syntax

```
*WRITEEVERY, TYPE=STEPS|TIME
<value>
```

***WRITEEVERY** controls transient result cadence. With **TYPE=STEPS**, the value is rounded to an integer and results are written every N steps. With **TYPE=TIME**, the value is a physical output interval; FEMaster converts it to a step stride by dividing by the analysis time step. The final time state is written even when it does not fall exactly on the stride.

Syntax

```
*INITIALVELOCITY, FIELD=<node-field>
```

***INITIALVELOCITY** references a node field with six components. The field is reduced into the active coordinate space and used as the initial velocity. If no initial velocity is specified, the initial reduced velocity is zero. Initial acceleration is computed internally from the initial state, the active matrices, and the load at the start time.

16.10 Linear harmonic response controls

16.10.1 Frequency sweep

Syntax

```
*FREQUENCIES, SCALE=LINEAR
<start-frequency>, <end-frequency>, <number-of-points>
```

***FREQUENCIES** defines the excitation frequencies of a **LINEARHARMONIC** loadcase. Frequencies are specified in cycles per unit time; the harmonic solver converts them internally to circular frequency with

$$\omega = 2\pi f.$$

The start frequency must be non-negative, the end frequency must be greater than or equal to the start frequency, and the number of points must be a positive integer. The current implementation supports a linearly spaced sweep. For more than one point, the increment is

$$\Delta f = \frac{f_{\text{end}} - f_{\text{start}}}{n - 1}.$$

With one point, the single requested frequency is the start frequency.

16.10.2 Rayleigh damping in harmonic analysis

The existing Rayleigh damping command is available in both **LINEARTRANSIENT** and **LINEARHARMONIC** loadcases:

Syntax

```
*DAMPING, TYPE=RAYLEIGH
<alpha>, <beta>
```

The damping matrix is

$$C = \alpha M + \beta K.$$

In harmonic analysis this matrix enters the frequency-domain operator through the term $i\omega C$. The mass-proportional coefficient α primarily affects the low-frequency range, while the stiffness-proportional coefficient β has increasing influence at higher frequencies.

Current harmonic solver restrictions

The current **LINEARHARMONIC** implementation requires **METHOD=DIRECT**, **TYPE=NULLSPACE** for ***CONSTRAINTMETHOD**, and a homogeneous constraint system. These restrictions belong to the present solver implementation rather than to the frequency-domain formulation itself.

17 Loadcases

A loadcase defines which analysis is executed on the previously defined model. The model describes what exists: nodes, elements, surfaces, sets, materials, sections, features, loads, supports, constraints, and fields. The loadcase describes which equation is assembled, which collectors are active, how constraints are handled, which solver is used, and which analysis-specific parameters are applied.

17.1 General syntax

Syntax

```
*LOADCASE, TYPE=<type>, NAME=<optional-name>  
  <loadcase commands>
```

TYPE is required. NAME is optional and is used for readability in the input deck and result output. Loadcases are executed in input order. Result blocks are written in the same order.

17.2 Common equation structure

Linear loadcases use global matrices assembled from element contributions. The most common operators are stiffness K , mass M , damping C , and geometric stiffness K_G . Active load collectors produce a right-hand side f . Active supports and constraints produce equations

$$C_c u = d.$$

The selected constraint method then either reduces the problem with $u = u_p + Tq$ or augments it with Lagrange multipliers. The loadcase formulas below are written in their physical full-space form first; the actual solved system may be the reduced or augmented form described in Chapter 16.

17.3 Linear Static

LINEARSTATIC solves small-displacement linear static equilibrium. It is the standard analysis for displacements, reactions, strains, stresses, and static load paths.

17.3.1 Governing equation

The physical equation is

$$Ku = f.$$

After Nullspace constraint reduction, FEMaster solves

$$T^T K T q = T^T (f - K u_p), \quad u = u_p + T q.$$

With Lagrange constraints, the static system is

$$\begin{bmatrix} K & C_c^T \\ C_c & 0 \end{bmatrix} \begin{bmatrix} u \\ \lambda \end{bmatrix} = \begin{bmatrix} f \\ d \end{bmatrix}$$

up to possible numerical regularization.

17.3.2 Typical DSL

Example

```
*LOADCASE, TYPE=LINEARSTATIC, NAME=static-load
*SUPPORTS
FIXED
*LOADS
FORCE
*SOLVER, DEVICE=CPU, METHOD=DIRECT
*CONSTRAINTMETHOD, TYPE=NULLSPACE
```

Common commands are *SUPPORTS, *LOADS, *SOLVER, and *CONSTRAINTMETHOD. Optional controls include *INERTIARELIEF, *REBALANCELOADS, *REQUESTSTIFFNESS, and *CONSTRAINTSUMMARY.

17.4 Linear Static Topology

LINEARSTATICTOPO solves a static problem whose element stiffness can be modified by topology fields. It is useful for topology studies, density-based parameter sweeps, and orientation-dependent stiffness experiments.

17.4.1 Governing equation

The equation is

$$K(\rho, \theta)u = f.$$

ρ is an element density-like field and θ is an element orientation-like field. The density field is typically used through a penalized stiffness contribution

$$K_e^{\text{eff}} = \rho_e^p K_e,$$

where p is supplied by *TOPOEXPONENT. Low densities reduce stiffness contribution; high densities approach the full element stiffness. The orientation field is used by element/material formulations that evaluate local orientation data.

17.4.2 Typical DSL

Syntax

```
*LOADCASE, TYPE=LINEARSTATICTOPO, NAME=<name>
*TOPODENSITY, FIELD=<element-field>
*TOPOORIENT, FIELD=<element-field>
*TOPOEXPONENT
<p>
```

***TOPODENSITY** requires **FIELD** to reference an element field with one component. ***TOPOORIENT** is optional and requires an element field with three components. Both commands also accept **NAME** as an alias for **FIELD**. ***TOPOEXPONENT** remains a data-line command; if it is omitted, $p = 1$. The density field itself is required. The static controls from **LINEARSTATIC** are used in the same way.

The compliance, density sensitivity, and orientation sensitivity derivations are collected in Appendix A.

17.5 Nonlinear Static

NONLINEARSTATIC solves quasi-static equilibrium with an updated geometry and a repeatedly assembled tangent stiffness. It supports load control and arc-length path following. Nonlinear contact contributions are assembled only by this loadcase.

17.5.1 Governing equation

At load factor λ , equilibrium is

$$f_{\text{int}}(u) = \lambda f_{\text{ref}}.$$

For a Newton correction at the current state, FEMaster assembles the tangent stiffness K_T and solves the constrained, reduced equation

$$T^T K_T T \Delta q = T^T (\lambda f_{\text{ref}} - f_{\text{int}}(u)), \quad u(\lambda) = \lambda u_p + T q.$$

This affine recovery applies to load control and ramps inhomogeneous prescribed displacements with the load factor. Arc-length control treats q and λ as coupled unknowns but currently requires homogeneous constraints, so $u_p = 0$ for that method.

The model positions are updated from the current displacement during nonlinear assembly. After the loadcase finishes, the original model positions are restored so that later loadcases start from the original geometry.

Load control prescribes λ incrementally. Arc-length control treats q and λ as coupled unknowns and adds a path constraint, allowing the solver to pass load limit points. Both methods and all ***NONLINEAR** options are described in Section 16.8.

17.5.2 Typical DSL

Example

```
*LOADCASE, TYPE=NONLINEARSTATIC, NAME=load-control
*SUPPORTS
FIXED
*LOADS
FORCE
*SOLVER, DEVICE=CPU, METHOD=DIRECT
*CONSTRAINTMETHOD, TYPE=NULLSPACE
*NONLINEAR, CONTROL=LOAD, INITIAL_INCREMENT=0.05, MINIMUM_INCREMENT=1e-5, MAXIMUM_INCREMENT
=0.1, MAX_INCREMENTS=100, ADAPTIVE=ON, MAXITER=30, TOL=1e-8
```

For an equilibrium path containing a limit point, select arc-length control:

Example

```

*LOADCASE, TYPE=NONLINEARSTATIC, NAME=arc-length
  *SUPPORTS
  FIXED
  *LOADS
  FORCE
  *SOLVER, DEVICE=CPU, METHOD=DIRECT
  *CONSTRAINTMETHOD, TYPE=NULLSPACE
  *NONLINEAR, CONTROL=ARC_LENGTH, ARC_LENGTH_PSI=1.0, INITIAL_INCREMENT=0.05,
    MINIMUM_INCREMENT=1e-6, MAXIMUM_INCREMENT=0.1, MAX_INCREMENTS=300, ADAPTIVE=ON, MAXITER
    =35, TOL=1e-7, REGULARIZE_ZERO_ROWS=OFF

```

Each accepted increment writes `DISPLACEMENT_i`, `STRESS_i`, and `LAMBDA_i`. These path frames are retained even if a later increment fails. On successful completion, FEMaster additionally writes the final displacement, strain, stress, internal-force, external-force, and reaction-force fields. `*REQUESTSTIFFNESS` writes the final full tangent and its reduced form.

17.6 Eigenfrequency

`EIGENFREQ` computes natural frequencies and mode shapes of the linearized model. Loads are not used because the free-vibration problem is defined by stiffness, mass, and boundary conditions.

17.6.1 Governing equation

The full-space eigenproblem is

$$K\phi_i = \omega_i^2 M\phi_i.$$

ϕ_i is the mode shape and ω_i is the circular frequency. The frequency in Hertz is

$$f_i = \frac{\omega_i}{2\pi}.$$

With Nullspace constraints, the reduced problem is

$$T^T K T \psi_i = \omega_i^2 T^T M T \psi_i, \quad \phi_i = T \psi_i.$$

17.6.2 Typical DSL

Syntax

```

*LOADCASE, TYPE=EIGENFREQ, NAME=<name>
  *SUPPORTS
  <support-collector>
  *NUMEIGENVALUES
  <count>

```

A free-free model can contain rigid-body modes with zero or near-zero frequency. A supported model computes modes relative to the active supports and constraints.

17.7 Linear Buckling

`LINEARBUCKLING` computes ideal linear buckling factors and buckling modes around a preloaded reference state. It first solves a static preload problem, then assembles geometric stiffness from the resulting stresses,

and finally solves a generalized eigenvalue problem.

17.7.1 Governing equations

The preload solve is

$$Ku_0 = f_{\text{pre}}.$$

With constraints, FEMaster solves the reduced preload equation

$$Aq_{\text{pre}} = b, \quad A = T^T K T, \quad b = T^T (f_{\text{pre}} - K u_p).$$

From the preload stresses, FEMaster builds the geometric stiffness K_G and reduces it to

$$B = T^T K_G T.$$

The buckling eigenproblem is solved as

$$A\phi_i = \lambda_i(-B)\phi_i.$$

λ_i is the buckling factor. For the preload pattern f_{pre} , the ideal linear critical load for mode i is

$$f_{\text{crit},i} = \lambda_i f_{\text{pre}}.$$

17.7.2 Sigma

***SIGMA** controls the buckling eigenvalue shift. It should be set carefully because it guides the part of the spectrum returned by the eigensolver. If ***SIGMA** is zero, FEMaster estimates a shift automatically from the preload displacement with the Rayleigh quotient

$$\lambda_{\text{raw}} = \frac{q_{\text{pre}}^T A q_{\text{pre}}}{q_{\text{pre}}^T (-B) q_{\text{pre}}}.$$

When this estimate is valid and positive, the generated shift is $\lambda_{\text{raw}}/10^6$. Otherwise a diagonal-ratio fallback is used; if that also fails, 10^{-12} is used.

17.7.3 Typical DSL

Syntax

```
*LOADCASE, TYPE=LINEARBUCKLING, NAME=<name>
  *SUPPORTS
  <support-collector>
  *LOADS
  <preload-collector>
  *NUMEIGENVALUES
  <count>
  *SIGMA
  <shift>
```

***LOADS** defines the preload. ***REQUESTSTIFFNESS** and ***REQUESTSTGEOM** can be used to write elastic and geometric stiffness matrices for diagnostics.

17.8 Linear Transient

LINEARTRANSIENT solves the linear dynamic response over time with implicit Newmark integration. The structure remains linear; contact, plasticity, and large deformation effects are not included in this loadcase.

17.8.1 Governing equation

The physical equation is

$$M\ddot{u}(t) + C\dot{u}(t) + Ku(t) = f(t).$$

With Nullspace constraints, this becomes

$$M_r\ddot{q}(t) + C_r\dot{q}(t) + K_rq(t) = r(t),$$

with

$$M_r = T^T M T, \quad C_r = T^T C T, \quad K_r = T^T K T.$$

Loads with amplitudes are assembled as

$$f(t) = \sum_j a_j(t) f_j.$$

For Rayleigh damping,

$$C = \alpha M + \beta K.$$

17.8.2 Typical DSL

Syntax

```
*LOADCASE, TYPE=LINEARTRANSIENT, NAME=<name>
  *SUPPORTS
  <support-collector>
  *LOADS
  <load-collector>
  *TIME
  <t_start>, <t_end>, <dt>
  *NEWMARK
  <beta>, <gamma>
  *DAMPING, TYPE=RAYLEIGH
  <alpha>, <beta>
  *WRITEEVERY, TYPE=STEPS
  <n>
```

The transient-specific keywords `*TIME`, `*NEWMARK`, `*DAMPING`, `*WRITEEVERY`, and `*INITIALVELOCITY` are described in Chapter 16.

17.9 Typical command overview

Loadcase	Typical commands
LINEARSTATIC	<code>*SUPPORTS</code> , <code>*LOADS</code> , <code>*SOLVER</code> , <code>*CONSTRAINTMETHOD</code> , optional <code>*INERTIARELIEF</code> , <code>*REBALANCELOADS</code> , <code>*REQUESTSTIFFNESS</code> , <code>*CONSTRAINTSUMMARY</code> .
LINEARSTATICTOPO	Linear-static commands plus <code>*TOPODENSITY</code> , <code>*TOPOORIENT</code> , <code>*TOPOEXPONENT</code> .
NONLINEARSTATIC	<code>*SUPPORTS</code> , <code>*LOADS</code> , <code>*SOLVER</code> , <code>*CONSTRAINTMETHOD</code> , <code>*NONLINEAR</code> , optional <code>*REQUESTSTIFFNESS</code> and <code>*CONSTRAINTSUMMARY</code> .
EIGENFREQ	<code>*SUPPORTS</code> , <code>*NUMEIGENVALUES</code> ; Nullspace constraints are applied internally.

Loadcase	Typical commands
LINEARBUCKLING	*SUPPORTS, *LOADS, *NUMEIGENVALUES, optional *SIGMA, *REQUESTSTIFFNESS, *REQUESTSTGEOM; Nullspace constraints are applied internally.
LINEARTRANSIENT	*SUPPORTS, *LOADS, *SOLVER, *TIME, *NEWMARK, *DAMPING, *WRITEEVERY, *INITIALVELOCITY; Nullspace constraints are applied internally.

17.10 Linear Harmonic

LINEARHARMONIC computes the steady-state response of a linear structural model to a harmonic load over a prescribed frequency sweep. The load amplitudes are defined by the active load collectors and are currently interpreted as real-valued harmonic amplitudes.

17.10.1 Governing equation

At circular excitation frequency ω , the complex displacement amplitude $\hat{u}$ satisfies

$$(K - \omega^2 M + i\omega C) \hat{u} = \hat{f}.$$

The complex response is written as

$$\hat{u} = u_{\text{real}} + i u_{\text{imag}}.$$

Rayleigh damping uses

$$C = \alpha M + \beta K.$$

FEMaster applies the active homogeneous constraints with the Nullspace transformation and solves the reduced frequency-domain problem. To reuse the existing real sparse solver backends, the complex system is written as the real block system

$$\begin{bmatrix} A & -B \\ B & A \end{bmatrix} \begin{bmatrix} u_{\text{real}} \\ u_{\text{imag}} \end{bmatrix} = \begin{bmatrix} f \\ 0 \end{bmatrix},$$

with

$$A = K - \omega^2 M, \quad B = \omega C.$$

17.10.2 Typical DSL

Example

```
*LOADCASE, TYPE=LINEARHARMONIC, NAME=FREQUENCY_RESPONSE
*SUPPORTS
BC
*LOADS
EXCITATION
*SOLVER, DEVICE=CPU, METHOD=DIRECT
*CONSTRAINTMETHOD, TYPE=NULLSPACE
*FREQUENCIES, SCALE=LINEAR
0.0, 500.0, 201
*DAMPING, TYPE=RAYLEIGH
0.0, 1.0e-5
```

*FREQUENCIES is required and is described in Section 16.10. The current implementation requires the direct solver, Nullspace constraint handling, and homogeneous supports and constraints.

17.10.3 Result fields

For every requested frequency FEMaster writes separate real and imaginary response fields:

- DISPLACEMENT_REAL and DISPLACEMENT_IMAG,
- STRESS_REAL and STRESS_IMAG,
- STRAIN_REAL and STRAIN_IMAG.

The harmonic loadcase passes the excitation frequency to the result writers as the frame value. The native `.res` writer currently does not encode this frame value in the field header, so harmonic `.res` blocks are associated with frequencies by their sweep order; see the Result Format chapter. Component-wise response magnitude and phase can be recovered from

$$|\hat{u}| = \sqrt{u_{\text{real}}^2 + u_{\text{imag}}^2}, \quad \varphi = \text{atan2}(u_{\text{imag}}, u_{\text{real}}).$$

The same interpretation applies to complex stress and strain components.

18 Contact

Contact introduces unilateral interaction between separate surface regions. Unlike a tie constraint, contact can open and close during the nonlinear solution and transfers only compressive normal interaction in the current frictionless formulation. FEMaster currently exposes one contact formulation: frictionless dual-mortar surface-to-surface contact for **NONLINEARSTATIC** loadcases.

18.1 Contact definition

Syntax

```
*CONTACT, MASTER=<master-surface>, SLAVE=<slave-surface>, PENALTY=<value>, CLEARANCE=<value>,  
        FLIP=NO|YES
```

MASTER, **SLAVE**, and **PENALTY** are required. **CLEARANCE** defaults to 0 and **FLIP** defaults to **NO**. Both master and slave must refer to surface sets; node-set slaves and a geometric search-distance parameter are not part of the current contact interface.

***CONTACT** is defined at model root level. The contact contribution is assembled only during **NONLINEARSTATIC**. A linear static, eigenfrequency, buckling, transient, or harmonic loadcase does not activate nonlinear contact.

18.2 Master and slave surfaces

The contact pair consists of one master surface region and one slave surface region. During assembly, current slave and master facets are projected to a common local plane and their physical overlap is segmented and integrated. The contact integration is therefore surface-to-surface rather than a node-to-surface collocation rule.

The selected element side and its orientation matter. Surface normals follow the element-side convention described in Chapter 6 and the surface definitions described in the Surfaces chapter. If the master normal is opposite to the intended contact orientation, **FLIP=YES** reverses the master normal used by the contact formulation.

Surface orientation

An incorrect side selection or reversed master normal can make an otherwise correct contact pair appear inactive or produce an unintended gap sign. Check element connectivity and surface-side numbering before increasing the penalty parameter to compensate for apparent non-contact.

18.3 Gap and clearance

Contact is unilateral. Separated surfaces carry no normal contact traction, while closed configurations can develop a compressive contact reaction. **CLEARANCE** modifies the normal gap used by the contact law and can be used to represent a prescribed normal offset without moving the mesh. The default is zero clearance.

There is no **DISTANCE** or search-radius keyword in the current DSL. The active surface pair is supplied explicitly, and the current master facets are considered during mortar segmentation. Older input decks that used a node slave or **DISTANCE** must be updated to the current surface-to-surface syntax.

18.4 Dual-mortar discretization

For every overlapping master/slave facet pair, FEMaster constructs a common-plane segmentation and integrates the overlap with a mortar formulation. The slave interpolation is paired with a dual basis so the contact residual is distributed consistently over the participating slave nodes while the master side receives the equal and opposite contribution.

This construction has several practical consequences:

- the slave region must be a surface set rather than a node set,
- contact force is integrated over projected surface overlap,
- contact can remain geometrically projected while still carrying zero force when the normal gap is open,
- master and slave contact resultants should be equal and opposite up to solver tolerance.

The method is frictionless; only normal contact is currently represented. Tangential frictional traction is not part of the exposed contact model.

18.5 Penalty and augmented-Lagrange update

PENALTY defines the starting normal mortar penalty stiffness. A larger value reduces penetration but also increases the stiffness contrast of the nonlinear system. The penalty should therefore be chosen large enough to control penetration without making the tangent unnecessarily ill-conditioned.

FEMaster combines the penalty contribution with augmented-Lagrange normal multipliers. The multiplier state is transactional: rejected nonlinear increments and temporary line-search evaluations can be rolled back without modifying the last accepted contact state. Between converged nonlinear solves the effective penalty may be increased internally when penetration reduction stagnates. The user-provided **PENALTY** value is the starting value of this continuation procedure.

18.6 Nonlinear solution behaviour

Contact changes the tangent and residual as surfaces open and close, so it is solved together with the structural equilibrium iterations of **NONLINEARSTATIC**. Contact state participates in the same accepted-increment and rollback lifecycle as other nonlinear history data.

The usual nonlinear controls in Section 16.8 therefore remain important. In particular, smaller increments can help when contact is first established or when the active overlap changes rapidly. If a model converges poorly, inspect the following before increasing iteration limits:

- master and slave surface orientation,
- initial gap and requested clearance,

- whether the intended surfaces actually overlap in projection,
- penalty magnitude relative to structural stiffness,
- support conditions and rigid-body freedom before contact closes.

18.7 Current limitations

The current user-facing contact implementation is intentionally narrow:

- frictionless normal contact only,
- dual-mortar surface-to-surface pairs only,
- contact contributions only in `NONLINEARSTATIC`,
- no node-to-surface slave definition,
- no user-facing geometric search-distance parameter,
- no tangential friction law.

These limitations should be treated as interface constraints of the present implementation, not as general limitations of the mortar method.

18.8 Minimal example

The following fragment defines one master and one slave face and applies surface-to-surface contact:

Example

```
*SURFACE, NAME=MASTER_FACE
1, 1, S2
*SURFACE, NAME=SLAVE_FACE
2, 2, S1

*CONTACT, MASTER=MASTER_FACE, SLAVE=SLAVE_FACE, PENALTY=1.0e5

*LOADCASE, TYPE=NONLINEARSTATIC, NAME=CONTACT_STEP
*SUPPORTS
BC
*SOLVER, DEVICE=CPU, METHOD=DIRECT
*CONSTRAINTMETHOD, TYPE=NULLSPACE
*NONLINEAR, CONTROL=LOAD, INITIAL_INCREMENT=0.1, MAXIMUM_INCREMENT=0.1, MAX_INCREMENTS=10
*END
```

A complete two-block regression model is included in the Complete Examples chapter and in `examples/10_contact/103_co`

19 Result Format

FEMaster writes two result formats by default. The native `.res` file is a text format that preserves all fields written by the loadcases. The `.frd` file is an ASCII CalculiX/CGX-style result file for supported nodal result visualization.

Both files use the same job base name. For example, an input deck `model.inp` produces `model.res` and `model.frd` unless `-output` changes the base name.

19.1 Native `.res` files

The `.res` file is organized into loadcases and fields. Each loadcase starts with a loadcase header followed by any number of field blocks.

19.1.1 Loadcase blocks

Syntax

```
LC <id>
```

The loadcase ID follows execution order. The following field blocks belong to that loadcase until the next loadcase header appears.

19.1.2 Dense fields

Syntax

```
FIELD, NAME=<name>, TYPE=<type>, COLS=<cols>, ROWS=<rows>  
<v11> <v12> ...  
END FIELD
```

TYPE is always written and is one of `NODE`, `ELEMENT`, `ELEMENT_NODAL`, `ELEMENT_IP`, `ELEMENT_MP`, or `UNKNOWN`. ROWS gives the number of rows. COLS gives the number of values per row. For nodal fields, a row typically corresponds to a node-position row. For element fields, a row corresponds to an element-position row.

19.1.3 Indexed fields

Syntax

```
FIELD, NAME=<name>, TYPE=<type>, INDEX_COLS=<i>, VALUE_COLS=<v>, ROWS=<rows>
<index...> <value...>
END FIELD
```

Element-local fields are written with explicit location indices. For `ELEMENT_NODAL`, `ELEMENT_IP`, and `ELEMENT_MP` fields, the native writer uses two index columns: the element ID and the zero-based local row number within that element. These are followed by the field values.

19.1.4 Common field names

Loadcase	Typical fields
Linear Static	DISPLACEMENT, STRAIN, STRESS, STRESS_TOP, STRESS_BOT, SHELL_RESULTANTS, EXTERNAL_FORCES, REACTION_FORCES, LOCAL_SECTION_FORCES, and optional SHEAR_FLOW.
Linear Static Topology	Static fields plus COMPLIANCE, DENS_GRAD, VOLUME, DENSITY, optional SHEAR_FLOW, and optional ORIENTATION_GRAD and ORIENTATION.
Nonlinear Static	Per accepted increment: DISPLACEMENT_i, STRESS_i, LAMBDA_i. Final state: DISPLACEMENT, STRAIN, STRESS, STRESS_TOP, STRESS_BOT, EXTERNAL_FORCES, INTERNAL_FORCES, and REACTION_FORCES.
Eigenfrequency	MODE_SHAPE_i, PARTICIPATION_i, EIGENVALUES, EIGENFREQUENCIES, FREQUENCIES.
Linear Buckling	BUCKLING_MODE_i, BUCKLING_FACTORS.
Linear Transient	DISPLACEMENT_k, VELOCITY_k, ACCELERATION_k for written time steps.
Linear Harmonic	For every requested frequency: DISPLACEMENT_REAL, DISPLACEMENT_IMAG, STRESS_REAL, STRESS_IMAG, STRAIN_REAL, and STRAIN_IMAG.

A six-component nodal motion field uses `Ux`, `Uy`, `Uz`, `Rx`, `Ry`, `Rz`. A six-component stress or strain field uses the engineering order x, y, z, yz, zx, xy . These conventions must not be confused.

19.1.5 Linear harmonic result fields

A `LINEARHARMONIC` loadcase writes the complex steady-state response as separate real and imaginary nodal fields for every requested excitation frequency. If a harmonic response quantity is written as

$$\hat{q} = q_{\text{real}} + iq_{\text{imag}},$$

then the corresponding `*_REAL` and `*_IMAG` fields contain these two components of the complex amplitude.

Field	Domain	Components
DISPLACEMENT_REAL	NODE	Real part of the harmonic nodal displacement amplitude in the mechanical nodal order <code>Ux</code> , <code>Uy</code> , <code>Uz</code> , <code>Rx</code> , <code>Ry</code> , <code>Rz</code> .

Field	Domain	Components
DISPLACEMENT_IMAG	NODE	Imaginary part of the harmonic nodal displacement amplitude in the mechanical nodal order $U_x, U_y, U_z, R_x, R_y, R_z$.
STRESS_REAL	NODE	Real part of the recovered harmonic nodal stress amplitude in engineering order x, y, z, yz, zx, xy .
STRESS_IMAG	NODE	Imaginary part of the recovered harmonic nodal stress amplitude in engineering order x, y, z, yz, zx, xy .
STRAIN_REAL	NODE	Real part of the recovered harmonic nodal strain amplitude in engineering order x, y, z, yz, zx, xy .
STRAIN_IMAG	NODE	Imaginary part of the recovered harmonic nodal strain amplitude in engineering order x, y, z, yz, zx, xy .

For any scalar component, amplitude and phase can be reconstructed from the stored real and imaginary values as

$$|\hat{q}| = \sqrt{q_{\text{real}}^2 + q_{\text{imag}}^2}, \quad \varphi = \text{atan2}(q_{\text{imag}}, q_{\text{real}}).$$

FEMaster currently writes the real/imaginary field blocks in frequency-sweep order, but the native `.res` writer does not encode the supplied frequency frame value in the field header. The associated excitation frequency is therefore determined from the `*FREQUENCIES` sequence and the block order.

19.2 FRD files

The `.frd` writer emits the model mesh once and then writes supported nodal result blocks. It is intended for visualization in CalculiX/CGX-compatible tools. Element-local fields, integration-point fields, material-point fields, participation factors, eigenvalue tables, and scalar history fields remain available in the native `.res` file but are not written as FRD result blocks.

FRD node and element numbers are normally the internal IDs. If the mesh contains ID 0, the FRD writer shifts those IDs by one because FRD consumers generally expect positive numbering.

Supported structural elements are written with FRD element type codes for common solid, shell, beam, and truss topologies. Higher-order wedge and hexahedral connectivity is reordered to match the FRD convention before it is written.

FEMaster field	FRD block	Components
DISPLACEMENT, MODE_SHAPE_i, BUCKLING_MODE_i	DISP	D1, D2, D3, D4, D5, D6, ALL.
VELOCITY_i	VELO	V1, V2, V3, ALL.
ACCELERATION_i	ACCE	A1, A2, A3, ALL.
REACTION_FORCES	FORC	F1, F2, F3, F4, F5, F6, ALL.
EXTERNAL_FORCES	EXTFORC	F1, F2, F3, F4, F5, F6, ALL.
INTERNAL_FORCES	INTFORC	F1, F2, F3, F4, F5, F6, ALL.
STRESS	STRESS	SXX, SYX, SZZ, SYZ, SZX, SXY.
STRAIN	TOSTRAIN	EXX, EYX, EZZ, EYZ, EZX, EXY.
MECHANICAL_STRAIN	MESTRAIN	MEXX, MEYX, MEZZ, MEYZ, MEZX, MEXY.
STRESS_TOP	STRPOS	SXX, SYX, SZZ, SYZ, SZX, SXY.
STRESS_BOT	STRNEG	SXX, SYX, SZZ, SYZ, SZX, SXY.
SHELL_RESULTANTS	SHR	NXX, NYX, NXY, MXX, MYX, MXY, QX, QY.

The harmonic names DISPLACEMENT_REAL, DISPLACEMENT_IMAG, STRESS_REAL, STRESS_IMAG, STRAIN_REAL,

and **STRAIN_IMAG** are not currently registered as FRD field aliases. Use the native **.res** output for the complete harmonic real/imaginary response until dedicated FRD mappings are added.

For shell resultants, an explicitly assigned ***SHELLSECTION** orientation also defines the output basis. Without an orientation, FEMaster writes physical shell stresses in global Cartesian components and rotates shell resultants into a deterministic tangent basis obtained by projecting global *X* into the shell plane, falling back to global *Y* when necessary.

For transient results, the frame value written to FRD is the physical time passed by the transient loadcase. For eigenfrequency and buckling results, the frame value is the frequency or buckling factor associated with the written mode.

20 Complete Examples

Every example in this chapter is a complete input deck. It includes geometry, properties, boundary conditions, loads where applicable, and the loadcase. The examples can therefore be copied into separate `.inp` files and run directly.

20.1 Linear static truss

This first model is a single axial bar in N-mm units. The guide support at the tip removes the two transverse rigid-body modes without constraining axial extension.

Example

```

*MODEL, NAME=LINEAR_TRUSS

*NODE, NSET=NALL
1, 0.0, 0.0, 0.0
2, 1000.0, 0.0, 0.0

*ELEMENT, TYPE=T3, ELSET=BAR
1, 1, 2

*NSET, NAME=FIXED
1
*NSET, NAME=TIP
2

*MATERIAL, NAME=STEEL
*ELASTIC, TYPE=ISOTROPIC
210000.0, 0.3

*TRUSSSECTION, ELSET=BAR, MATERIAL=STEEL
100.0

*SUPPORT, SUPPORT_COLLECTOR=BC
FIXED, 0.0, 0.0, 0.0, NAN, NAN, NAN
TIP, NAN, 0.0, 0.0, NAN, NAN, NAN

*CLOAD, LOAD_COLLECTOR=AXIAL_LOAD
TIP, 1000.0, 0.0, 0.0, 0.0, 0.0, 0.0

*LOADCASE, TYPE=LINEARSTATIC, NAME=AXIAL_TENSION
*SUPPORTS
BC
*LOADS
AXIAL_LOAD
*SOLVER, DEVICE=CPU, METHOD=DIRECT
*CONSTRAINTMETHOD, TYPE=NULLSPACE
*END

```

20.2 Topology-aware linear static solid

The density and orientation fields are defined at root level and referenced by keyword. ***TOPODENSITY** requires **FIELD**; the data-line form is not valid. The orientation angles are Euler angles in radians and are optional. Density $\rho_e = 0.8$ with $p = 3$ scales this element's stiffness by 0.8^3 .

Example

```

*MODEL, NAME=TOPOLOGY_CUBE

*NODE, NSET=NALL
1, 0.0, 0.0, 0.0
2, 0.0, 1.0, 0.0
3, 0.0, 0.0, 1.0
4, 0.0, 1.0, 1.0
5, 1.0, 0.0, 0.0
6, 1.0, 1.0, 0.0
7, 1.0, 0.0, 1.0
8, 1.0, 1.0, 1.0

*ELEMENT, TYPE=C3D8, ELSET=DESIGN_DOMAIN
1, 1, 5, 6, 2, 3, 7, 8, 4

*NSET, NAME=FIXED_FACE
1, 2, 3, 4
*NSET, NAME=LOADED_FACE
5, 6, 7, 8

*MATERIAL, NAME=ORTHOTROPIC_SOLID
*ELASTIC, TYPE=ORTHOTROPIC
100000.0, 50000.0, 50000.0, 20000.0, 20000.0, 25000.0, 0.25, 0.25, 0.25

*SOLIDSECTION, ELSET=DESIGN_DOMAIN, MATERIAL=ORTHOTROPIC_SOLID

*FIELD, NAME=RHO, TYPE=ELEMENT, COLS=1, FILL=ZERO
1, 0.8

*FIELD, NAME=ANGLES, TYPE=ELEMENT, COLS=3, FILL=ZERO
1, 0.0, 0.0, 0.0

*SUPPORT, SUPPORT_COLLECTOR=BC
FIXED_FACE, 0.0, 0.0, 0.0, NAN, NAN, NAN

*CLOAD, LOAD_COLLECTOR=TENSION
LOADED_FACE, 250.0, 0.0, 0.0, 0.0, 0.0, 0.0

*LOADCASE, TYPE=LINEARSTATICTOPO, NAME=TOPO_STATIC
*SUPPORTS
BC
*LOADS
TENSION
*SOLVER, DEVICE=CPU, METHOD=DIRECT
*CONSTRAINTMETHOD, TYPE=NULLSPACE
*TOPODENSITY, FIELD=RHO
*TOPOORIENT, FIELD=ANGLES
*TOPOEXPONENT
3.0
*END

```

20.3 Shell with explicit ABD and transverse shear stiffness

The data following `*SHELLSECTION` with `TYPE=ABD` contain the 6×6 ABD matrix in row-major order followed by the four entries of S , also in row-major order. The example uses a symmetric uncoupled laminate, but the parser accepts all 36 ABD entries independently.

Example

```

*MODEL, NAME=ABD_CANTILEVER

*NODE, NSET=NALL
1, 0.0, 0.0, 0.0
2, 1000.0, 0.0, 0.0
3, 1000.0, 500.0, 0.0
4, 0.0, 500.0, 0.0

*ELEMENT, TYPE=S4, ELSET=PANEL
1, 1, 2, 3, 4

*NSET, NAME=ROOT
1, 4
*NSET, NAME=TIP
2, 3

*SHELLSECTION, ELSET=PANEL, TYPE=ABD, THICKNESS=1.0
1.0e6, 2.5e5, 0.0, 0.0, 0.0, 0.0
2.5e5, 1.0e6, 0.0, 0.0, 0.0, 0.0
0.0, 0.0, 3.0e5, 0.0, 0.0, 0.0
0.0, 0.0, 0.0, 1.0e3, 2.5e2, 0.0
0.0, 0.0, 0.0, 2.5e2, 1.0e3, 0.0
0.0, 0.0, 0.0, 0.0, 0.0, 3.0e2
3.0e5, 0.0, 0.0, 3.0e5

*SUPPORT, SUPPORT_COLLECTOR=BC
ROOT, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0

*CLOAD, LOAD_COLLECTOR=TIP_LOAD
TIP, 0.0, 0.0, -100.0, 0.0, 0.0, 0.0

*LOADCASE, TYPE=LINEARSTATIC, NAME=ABD_BENDING
*SUPPORTS
BC
*LOADS
TIP_LOAD
*SOLVER, DEVICE=CPU, METHOD=DIRECT
*CONSTRAINTMETHOD, TYPE=NULLSPACE
*END

```

20.4 Nonlinear load control

The two initially horizontal bars have no linearized vertical stiffness at their midpoint. Zero-row regularization supplies the initial perturbation; after deflection, geometric stiffness carries the load.

Example

```

*MODEL, NAME=TWO_BAR_MECHANISM

*NODE, NSET=NALL
1, -1000.0, 0.0, 0.0
2, 0.0, 0.0, 0.0
3, 1000.0, 0.0, 0.0

*ELEMENT, TYPE=T3, ELSET=BARS
1, 1, 2
2, 2, 3

*NSET, NAME=END_SUPPORTS
1, 3
*NSET, NAME=MIDPOINT
2

*MATERIAL, NAME=STEEL
*ELASTIC, TYPE=ISOTROPIC
210000.0, 0.3

*TRUSSSECTION, ELSET=BARS, MATERIAL=STEEL
100.0

*SUPPORT, SUPPORT_COLLECTOR=BC
END_SUPPORTS, 0.0, 0.0, 0.0, NAN, NAN, NAN
MIDPOINT, NAN, NAN, 0.0, NAN, NAN, NAN

*CLOAD, LOAD_COLLECTOR=MIDPOINT_LOAD
MIDPOINT, 0.0, -1000.0, 0.0, 0.0, 0.0, 0.0

*LOADCASE, TYPE=NONLINEARSTATIC, NAME=LOAD_CONTROL
*SUPPORTS
BC
*LOADS
MIDPOINT_LOAD
*SOLVER, DEVICE=CPU, METHOD=DIRECT
*CONSTRAINTMETHOD, TYPE=NULLSPACE
*NONLINEAR, CONTROL=LOAD, INCREMENTS=20, MAXITER=40, TOL=1e-8, ADAPTIVE=ON,
    REGULARIZE_ZERO_ROWS=ON, REGULARIZATION_ALPHA=1e-4
*END

```

20.5 Snap-through with arc-length control

The 20 kN reference load exceeds the first limit load of the shallow two-bar arch. Arc-length control can follow the equilibrium path while the load factor temporarily decreases.

Example

```

*MODEL, NAME=SNAP_THROUGH_TRUSS

*NODE, NSET=NALL
1, -1000.0, 0.0, 0.0
2, 0.0, 100.0, 0.0
3, 1000.0, 0.0, 0.0

*ELEMENT, TYPE=T3, ELSET=BARS
1, 1, 2
2, 2, 3

*NSET, NAME=END_SUPPORTS
1, 3
*NSET, NAME=APEX
2

*MATERIAL, NAME=STEEL
*ELASTIC, TYPE=ISOTROPIC
210000.0, 0.3

*TRUSSSECTION, ELSET=BARS, MATERIAL=STEEL
100.0

*SUPPORT, SUPPORT_COLLECTOR=BC
END_SUPPORTS, 0.0, 0.0, 0.0, NAN, NAN, NAN
APEX, 0.0, NAN, 0.0, NAN, NAN, NAN

*CLOAD, LOAD_COLLECTOR=APEX_LOAD
APEX, 0.0, -20000.0, 0.0, 0.0, 0.0, 0.0

*LOADCASE, TYPE=NONLINEARSTATIC, NAME=ARC_LENGTH
*SUPPORTS
BC
*LOADS
APEX_LOAD
*SOLVER, DEVICE=CPU, METHOD=DIRECT
*CONSTRAINTMETHOD, TYPE=NULLSPACE
*NONLINEAR, CONTROL=ARC_LENGTH, ARC_LENGTH_PSI=1.0, MAX_INCREMENTS=200, INITIAL_INCREMENT
    =0.1, MINIMUM_INCREMENT=1e-5, MAXIMUM_INCREMENT=0.1, ADAPTIVE=ON, MAXITER=30, TOL=1e-8,
    REGULARIZE_ZERO_ROWS=OFF
*END

```

20.6 Eigenfrequency with a point mass

The two-element bar has two free axial degrees of freedom; the loadcase requests the lowest mode. Its distributed density and the concentrated tip mass both contribute to the mass matrix.

Example

```

*MODEL, NAME=AXIAL_EIGENFREQUENCY

*NODE, NSET=NALL
1, 0.0, 0.0, 0.0
2, 500.0, 0.0, 0.0
3, 1000.0, 0.0, 0.0

*ELEMENT, TYPE=T3, ELSET=BAR
1, 1, 2
2, 2, 3

*NSET, NAME=FIXED
1
*NSET, NAME=GUIDED
2, 3
*NSET, NAME=TIP
3

*MATERIAL, NAME=STEEL
*ELASTIC, TYPE=ISOTROPIC
210000.0, 0.3
*DENSITY
7.85e-9

*TRUSSSECTION, ELSET=BAR, MATERIAL=STEEL
100.0

*POINTMASS, NSET=TIP
0.01, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0

*SUPPORT, SUPPORT_COLLECTOR=BC
FIXED, 0.0, 0.0, 0.0, NAN, NAN, NAN
GUIDED, NAN, 0.0, 0.0, NAN, NAN, NAN

*LOADCASE, TYPE=EIGENFREQ, NAME=AXIAL_MODE
*SUPPORTS
BC
*NUMEIGENVALUES
1
*END

```

20.7 Linear buckling of a shallow truss

The preload places both bars in compression. Linear buckling then evaluates the geometric stiffness of that preloaded state and reports the requested factor and mode.

Example

```
*MODEL, NAME=SHALLOW_TRUSS_BUCKLING

*NODE, NSET=NALL
1, -1000.0, 0.0, 0.0
2, 0.0, 100.0, 0.0
3, 1000.0, 0.0, 0.0

*ELEMENT, TYPE=T3, ELSET=BARS
1, 1, 2
2, 2, 3

*NSET, NAME=ENDS
1, 3
*NSET, NAME=APEX
2

*MATERIAL, NAME=STEEL
*ELASTIC, TYPE=ISOTROPIC
210000.0, 0.3

*TRUSSSECTION, ELSET=BARS, MATERIAL=STEEL
100.0

*SUPPORT, SUPPORT_COLLECTOR=BC
ENDS, 0.0, 0.0, 0.0, NAN, NAN, NAN
APEX, NAN, NAN, 0.0, NAN, NAN, NAN

*CLOAD, LOAD_COLLECTOR=PRELOAD
APEX, 0.0, -1000.0, 0.0, 0.0, 0.0, 0.0

*LOADCASE, TYPE=LINEARBUCKLING, NAME=LIMIT_ESTIMATE
*SUPPORTS
BC
*LOADS
PRELOAD
*SOLVER, DEVICE=CPU, METHOD=DIRECT
*NUMEIGENVALUES
1
*SIGMA
0.0
*END
```

20.8 Linear transient ramp load

The amplitude ramps the axial force over 0.1 seconds and then holds it constant. Results are written every ten integration steps.

Example

```
*MODEL, NAME=TRANSIENT_TRUSS

*NODE, NSET=NALL
1, 0.0, 0.0, 0.0
2, 1000.0, 0.0, 0.0

*ELEMENT, TYPE=T3, ELSET=BAR
1, 1, 2

*NSET, NAME=FIXED
1
*NSET, NAME=TIP
2

*MATERIAL, NAME=STEEL
*ELASTIC, TYPE=ISOTROPIC
210000.0, 0.3
*DENSITY
7.85e-9

*TRUSSSECTION, ELSET=BAR, MATERIAL=STEEL
100.0

*SUPPORT, SUPPORT_COLLECTOR=BC
FIXED, 0.0, 0.0, 0.0, NAN, NAN, NAN
TIP, NAN, 0.0, 0.0, NAN, NAN, NAN

*AMPLITUDE, NAME=RAMP, TYPE=LINEAR
0.0, 0.0
0.1, 1.0
1.0, 1.0

*CLOAD, LOAD_COLLECTOR=DYNAMIC_LOAD, AMPLITUDE=RAMP
TIP, 1000.0, 0.0, 0.0, 0.0, 0.0, 0.0

*LOADCASE, TYPE=LINEARTRANSIENT, NAME=RAMP_RESPONSE
*SUPPORTS
BC
*LOADS
DYNAMIC_LOAD
*SOLVER, DEVICE=CPU, METHOD=DIRECT
*TIME
0.0, 1.0, 0.01
*NEWMARK
0.25, 0.5
*DAMPING, TYPE=RAYLEIGH
0.0, 1e-5
*WRITEEVERY, TYPE=STEPS
10
*END
```

20.9 Linear harmonic response of a cantilever beam

This example excites the tip of a cantilever beam with a harmonic force and evaluates 201 frequencies between 0 and 500. Rayleigh damping regularizes the resonance peaks. The response is written as separate real and imaginary displacement, stress, and strain fields for every frequency.

Example

```
*MODEL, NAME=HARMONIC_BEAM
*NODE, NSET=NALL
1, 0, 0, 0
2, 250, 0, 0
3, 500, 0, 0
4, 750, 0, 0
5, 1000, 0, 0
*ELEMENT, TYPE=B33, ELSET=BEAM
1, 1, 2
2, 2, 3
3, 3, 4
4, 4, 5
*NSET, NAME=ROOT
1
*NSET, NAME=TIP
5
*MATERIAL, NAME=STEEL
*ELASTIC, TYPE=ISOTROPIC
210000.0, 0.3
*DENSITY
7.85e-9
*PROFILE, NAME=RECT_20x10
200.0, 6666.7, 1666.7, 8333.3
*BEAMSECTION, ELSET=BEAM, MATERIAL=STEEL, PROFILE=RECT_20x10
0.0, 0.0, 1.0
*SUPPORT, SUPPORT_COLLECTOR=BC
ROOT, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
*CLOAD, LOAD_COLLECTOR=EXCITATION
TIP, 0.0, 0.0, -100.0, 0.0, 0.0, 0.0
*LOADCASE, TYPE=LINEARHARMONIC, NAME=FREQUENCY_RESPONSE
*SUPPORTS
BC
*LOADS
EXCITATION
*SOLVER, DEVICE=CPU, METHOD=DIRECT
*CONSTRAINTMETHOD, TYPE=NULLSPACE
*FREQUENCIES, SCALE=LINEAR
0.0, 500.0, 201
*DAMPING, TYPE=RAYLEIGH
0.0, 1.0e-5
*END
```

20.10 Dual-mortar contact between two C3D8 blocks

This regression model isolates the current surface-to-surface contact formulation. The lower block is fixed. The upper block starts with a 0.5 gap, is guided laterally, and receives a prescribed downward displacement at its top face. Contact begins when the initial gap has closed; before that point the upper block can translate without developing a normal contact reaction.

Example

```

*MODEL, NAME=CONTACT_MORTAR_PARALLEL_BLOCKS_C3D8
*NODE, NSET=NALL
1, 0.0, 0.0, -1.0
2, 1.0, 0.0, -1.0
3, 1.0, 1.0, -1.0
4, 0.0, 1.0, -1.0
5, 0.0, 0.0, 0.0
6, 1.0, 0.0, 0.0
7, 1.0, 1.0, 0.0
8, 0.0, 1.0, 0.0
9, 0.0, 0.0, 0.5
10, 1.0, 0.0, 0.5
11, 1.0, 1.0, 0.5
12, 0.0, 1.0, 0.5
13, 0.0, 0.0, 100.5
14, 1.0, 0.0, 100.5
15, 1.0, 1.0, 100.5
16, 0.0, 1.0, 100.5

*ELEMENT, TYPE=C3D8, ELSET=LOWER_BLOCK
1, 1, 2, 3, 4, 5, 6, 7, 8
*ELEMENT, TYPE=C3D8, ELSET=UPPER_BLOCK
2, 9, 10, 11, 12, 13, 14, 15, 16

*NSET, NAME=LOWER_ALL
1, 2, 3, 4, 5, 6, 7, 8
*NSET, NAME=UPPER_ALL
9, 10, 11, 12, 13, 14, 15, 16
*NSET, NAME=UPPER_TOP
13, 14, 15, 16

*SURFACE, NAME=MASTER_FACE
1, 1, S2
*SURFACE, NAME=SLAVE_FACE
2, 2, S1

*MATERIAL, NAME=LINEAR
*ELASTIC, TYPE=ISOTROPIC
1000.0, 0.0
*SOLIDSECTION, ELSET=LOWER_BLOCK, MATERIAL=LINEAR
*SOLIDSECTION, ELSET=UPPER_BLOCK, MATERIAL=LINEAR

*CONTACT, MASTER=MASTER_FACE, SLAVE=SLAVE_FACE, PENALTY=1.0e5

*SUPPORT, SUPPORT_COLLECTOR=BC
LOWER_ALL, 0.0, 0.0, 0.0, NAN, NAN, NAN
UPPER_ALL, 0.0, 0.0, NAN, NAN, NAN, NAN
UPPER_TOP, NAN, NAN, -1.0, NAN, NAN, NAN

*LOADCASE, TYPE=NONLINEARSTATIC, NAME=MORTAR_BENCHMARK
*SUPPORTS
BC
*SOLVER, DEVICE=CPU, METHOD=DIRECT
*CONSTRAINTMETHOD, TYPE=NULLSPACE
*NONLINEAR, CONTROL=LOAD, INITIAL_INCREMENT=0.1, MINIMUM_INCREMENT=0.1, MAXIMUM_INCREMENT
=0.1, MAX_INCREMENTS=10, ADAPTIVE=OFF, MAXITER=30, TOL=1e-10, REGULARIZE_ZERO_ROWS=ON
*END

```

For this one-face/one-face benchmark the projected overlap area is 1, contact must remain inactive while the gap is positive, and total master and slave contact resultants must be equal and opposite. The repository additionally contains `examples/10_contact/103_contact_mortar_parallel_blocks_c3d8_expected.md` with the analytic pre-contact and penalty-response checks used for regression testing.

A Mathematical Derivations

This appendix collects derivations that are useful for understanding the topology-related quantities written by FEMaster. The main reference chapters describe how to use the DSL commands. This appendix gives the mathematical background for compliance, density sensitivities, and orientation sensitivities.

A.1 Element stiffness

For a linear elastic finite element, the unmodified element stiffness is

$$K_{e,0} = \int_{\Omega_e} B^T D B d\Omega.$$

B is the strain-displacement matrix, D is the constitutive matrix, and Ω_e is the physical element domain. With an isoparametric mapping from natural coordinates $r = (\xi, \eta, \zeta)$, the integral is evaluated as

$$K_{e,0} = \int_{\Omega'_e} B(r)^T D B(r) |\det J(r)| d\xi d\eta d\zeta.$$

For numerical integration with quadrature points r_q and weights w_q , this becomes

$$K_{e,0} \approx \sum_{q=1}^Q w_q B(r_q)^T D B(r_q) |\det J(r_q)|.$$

A.2 Compliance

For a linear static analysis,

$$Ku = f.$$

The external work measure commonly used for topology optimization is the compliance

$$C = f^T u.$$

Using the equilibrium equation $f = Ku$, this can also be written as

$$C = u^T Ku.$$

Because the global stiffness matrix is assembled from element stiffness matrices, compliance can be decomposed into element contributions:

$$C = \sum_{e=1}^{n_e} u_e^T K_e u_e.$$

This expression is the basis for element-wise topology sensitivities. u_e is the element displacement vector extracted from the global solution.

A.3 Density penalization

In a density-based topology formulation, an element density variable ρ_e scales the stiffness contribution. With a penalization exponent p ,

$$K_e(\rho_e) = \rho_e^p K_{e,0}.$$

The exponent p makes intermediate densities less efficient and therefore encourages designs closer to a solid-void distribution.

The element compliance contribution is

$$C_e(\rho_e) = u_e^T K_e(\rho_e) u_e = \rho_e^p u_e^T K_{e,0} u_e.$$

If u_e is held fixed for the local sensitivity expression, the derivative is

$$\frac{\partial C_e}{\partial \rho_e} = p \rho_e^{p-1} u_e^T K_{e,0} u_e.$$

Depending on the optimization convention, the objective derivative may be reported with the opposite sign because minimizing compliance through the equilibrium equation often uses the adjoint result

$$\frac{dC}{d\rho_e} = -u_e^T \frac{\partial K_e}{\partial \rho_e} u_e = -p \rho_e^{p-1} u_e^T K_{e,0} u_e.$$

The sign convention should therefore be interpreted together with the optimizer interface and the field name written by the analysis.

A.4 Orientation-dependent stiffness

For orientation-based topology or material steering, the constitutive matrix is rotated before the element stiffness is evaluated. Let the material orientation be represented by three angles

$$\alpha = (\alpha_1, \alpha_2, \alpha_3).$$

The elementary rotation matrices are

$$R_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha_1 & -\sin \alpha_1 \\ 0 & \sin \alpha_1 & \cos \alpha_1 \end{bmatrix},$$

$$R_2 = \begin{bmatrix} \cos \alpha_2 & 0 & \sin \alpha_2 \\ 0 & 1 & 0 \\ -\sin \alpha_2 & 0 & \cos \alpha_2 \end{bmatrix}, \quad R_3 = \begin{bmatrix} \cos \alpha_3 & -\sin \alpha_3 & 0 \\ \sin \alpha_3 & \cos \alpha_3 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

For the convention used here, the combined rotation is

$$R = R_1 R_2 R_3.$$

The multiplication order must be used consistently. Different conventions can describe the same physical orientation with different angle triples.

A.5 Voigt transformation

The strain vector uses engineering shear strains,

$$\varepsilon = [\varepsilon_{xx} \quad \varepsilon_{yy} \quad \varepsilon_{zz} \quad 2\varepsilon_{yz} \quad 2\varepsilon_{xz} \quad 2\varepsilon_{xy}]^T,$$

and the stress vector is

$$\sigma = [\sigma_{xx} \quad \sigma_{yy} \quad \sigma_{zz} \quad \sigma_{yz} \quad \sigma_{xz} \quad \sigma_{xy}]^T.$$

The strain transformation is written as

$$\varepsilon_m = T(R)\varepsilon_g,$$

where T is the 6×6 Voigt transformation matrix built from the entries of R . The rotated constitutive matrix used in global strain coordinates is

$$D_{\text{rot}} = T^T D T.$$

The orientation-dependent element stiffness is therefore

$$K_e(\rho_e, \alpha_e) = \rho_e^p \int_{\Omega_e} B^T D_{\text{rot}}(\alpha_e) B d\Omega.$$

With quadrature,

$$K_e(\rho_e, \alpha_e) \approx \rho_e^p \sum_{q=1}^Q w_q B(r_q)^T T(\alpha_e)^T D T(\alpha_e) B(r_q) |\det J(r_q)|.$$

A.6 Orientation sensitivity

For one element, the compliance contribution is

$$C_e = \rho_e^p \sum_{q=1}^Q w_q \varepsilon_q^T T^T D T \varepsilon_q |\det J_q|, \quad \varepsilon_q = B(r_q) u_e.$$

Only T depends on the orientation angles, so the derivative with respect to angle α_j is

$$\frac{\partial C_e}{\partial \alpha_j} = \rho_e^p \sum_{q=1}^Q w_q \varepsilon_q^T \left(\frac{\partial T^T}{\partial \alpha_j} D T + T^T D \frac{\partial T}{\partial \alpha_j} \right) \varepsilon_q |\det J_q|.$$

If D is symmetric, the scalar expression can also be evaluated as

$$\frac{\partial C_e}{\partial \alpha_j} = 2\rho_e^p \sum_{q=1}^Q w_q \varepsilon_q^T T^T D \frac{\partial T}{\partial \alpha_j} \varepsilon_q |\det J_q|,$$

provided the same transformation convention is used.

A.7 Rotation derivatives

The derivatives of the elementary rotation matrices are

$$\begin{aligned} \frac{\partial R_1}{\partial \alpha_1} &= \begin{bmatrix} 0 & 0 & 0 \\ 0 & -\sin \alpha_1 & -\cos \alpha_1 \\ 0 & \cos \alpha_1 & -\sin \alpha_1 \end{bmatrix}, \\ \frac{\partial R_2}{\partial \alpha_2} &= \begin{bmatrix} -\sin \alpha_2 & 0 & \cos \alpha_2 \\ 0 & 0 & 0 \\ -\cos \alpha_2 & 0 & -\sin \alpha_2 \end{bmatrix}, \\ \frac{\partial R_3}{\partial \alpha_3} &= \begin{bmatrix} -\sin \alpha_3 & -\cos \alpha_3 & 0 \\ \cos \alpha_3 & -\sin \alpha_3 & 0 \\ 0 & 0 & 0 \end{bmatrix}. \end{aligned}$$

For $R = R_1 R_2 R_3$, the total rotation derivatives are

$$\frac{\partial R}{\partial \alpha_1} = \frac{\partial R_1}{\partial \alpha_1} R_2 R_3,$$

$$\frac{\partial R}{\partial \alpha_2} = R_1 \frac{\partial R_2}{\partial \alpha_2} R_3,$$

$$\frac{\partial R}{\partial \alpha_3} = R_1 R_2 \frac{\partial R_3}{\partial \alpha_3}.$$

The derivative $\partial T / \partial \alpha_j$ is obtained by differentiating every entry of $T(R)$ with the product rule. For example, an entry $R_{ab} R_{cd}$ contributes

$$\frac{\partial}{\partial \alpha_j} (R_{ab} R_{cd}) = \frac{\partial R_{ab}}{\partial \alpha_j} R_{cd} + R_{ab} \frac{\partial R_{cd}}{\partial \alpha_j}.$$

This product-rule structure is the key point of the orientation derivative: once R , $\partial R / \partial \alpha_j$, T , and $\partial T / \partial \alpha_j$ are known, the sensitivity follows directly from the quadrature expression above.